

CORSO JUNIOR DATA ENGINEER

Le Classi in Python

Programmazione a oggetti dalla teoria alla pratica

Edizione guidata

56 esempi pronti da incollare ed eseguire, uno dopo ogni argomento

Con output atteso, spiegazione e 20 esercizi con soluzioni commentate

Modulo 1 · Fondamenti di programmazione a oggetti

Indice

Come usare questa guida · le prove da incollare ed eseguire

Perché gli oggetti?

- 1.1 Un problema concreto: gestire i dati di un corso
- 1.2 Tutto quello che può andare storto
- 1.3 La stessa cosa, con gli oggetti
- 1.4 Il vocabolario minimo: classe, istanza, campo, metodo
- 1.5 La mappa del viaggio

Esercizio 1

Classi, oggetti, campi e metodi

- 2.1 La prima classe
- 2.2 Il costruttore `__init__`
- 2.3 Che cos'è davvero `self`
- 2.4 I campi: lo stato dell'oggetto
- 2.5 I metodi: il comportamento dell'oggetto
- 2.6 Tante istanze, tante vite separate
- 2.7 Stampare un oggetto: `__str__` e `__repr__`
- 2.8 Attributi di classe e attributi di istanza

Esercizi 2 – 5

Variabili che contengono oggetti

- 3.1 La variabile non contiene l'oggetto
- 3.2 Alias: due nomi, un solo oggetto
- 3.3 `is` contro `==`
- 3.4 Oggetti dentro le liste
- 3.5 Oggetti dentro altri oggetti: la composizione
- 3.6 Passare oggetti alle funzioni
- 3.7 Copiare davvero un oggetto

Esercizi 6 – 9

Incapsulamento

- 4.1 Il problema: nessuno controlla i dati
- 4.2 Convenzioni di visibilità: `_` e `__`
- 4.3 Le property: controllare senza cambiare la sintassi
- 4.4 L'interfaccia pubblica come contratto
- 4.5 Progettare il componente prima di scriverlo
- 4.6 Testare il componente dalla sua interfaccia

4.7 Da `assert` a `pytest`

Esercizi 10 – 13

Ereditarietà e overriding

5.1 Il codice duplicato che chiede aiuto

5.2 Estendere una classe

5.3 `super()` : riusare il costruttore del padre

5.4 Overriding: sovrascrivere un metodo

5.5 Estendere invece di sostituire

5.6 `isinstance` e la catena di ereditarietà

5.7 Quando l'ereditarietà è la scelta sbagliata

Esercizi 14 – 17

Generalizzazione e astrazione

6.1 Generalizzare: salire di un livello

6.2 Il polimorfismo in azione

6.3 Classi astratte con `abc`

6.4 Il duck typing pythonico

6.5 Quanto astrarre? Il costo dell'astrazione

Esercizi 18 – 20

Progetto finale: il tuo primo componente da Data Engineer

7.1 Il requisito

7.2 Passo 1: progettare l'interfaccia

7.3 Passo 2: la classe astratta

7.4 Passo 3: le implementazioni concrete

7.5 Passo 4: il codice cliente che non cambia mai

7.6 Passo 5: i test del componente

7.7 Dove andrà a finire questo codice

Appendice A · Soluzioni commentate degli esercizi

Appendice B · I dieci errori più comuni

Appendice C · Cheat sheet: la sintassi a colpo d'occhio

Perché gli oggetti?

IN QUESTO CAPITOLO

- Vedrai un programma scritto *senza* oggetti e capirai perché diventa ingestibile.
- Scoprirai che cosa risolvono le classi, prima ancora di imparare come si scrivono.
- Imparerai le quattro parole che useremo per tutto il resto della dispensa: classe, istanza, campo, metodo.

Quasi tutti i corsi di programmazione a oggetti iniziano mostrando la sintassi: `class`, `self`, `__init__`. È un errore, perché la sintassi risponde alla domanda «*come si scrive?*» quando la domanda che conta è un'altra: «*quale problema sto risolvendo?*». Se non hai mai sentito il dolore che gli oggetti curano, le classi sembrano solo un modo più complicato di fare cose che sapevi già fare.

Quindi partiamo dal dolore.

1.1 Un problema concreto: gestire i dati di un corso

Ti hanno chiesto un piccolo programma per gestire gli studenti di un corso: registrare nome, email e i voti dei test, calcolare la media, dire chi è promosso. Con gli strumenti che conosci già — dizionari, liste e funzioni — la soluzione naturale è questa.

```
# studenti come dizionari
studente1 = {"nome": "Anna Rossi", "email": "anna@example.com", "voti": [28, 30, 25]}
studente2 = {"nome": "Luca Bianchi", "email": "luca@example.com", "voti": [18, 21]}

def media(studente):
    if len(studente["voti"]) == 0:
        return 0
    return sum(studente["voti"]) / len(studente["voti"])

def aggiungi_voto(studente, voto):
    studente["voti"].append(voto)

def promosso(studente):
    return media(studente) >= 24

print(media(studente1))      # 27.666...
print(promosso(studente2))   # False
```

Versione procedurale: i dati stanno nei dizionari, il comportamento sta nelle funzioni. Sono due mondi separati.

Funziona. Per venti righe di codice va benissimo, e non c'è niente di male nello scriverlo così. Il problema nasce quando il programma cresce: altri tipi di dato, altri sviluppatori, altri sei mesi di

manutenzione.

PROVA TU · 1 Uno studente fatto di dizionari

Vediamo com'è fatto un programma che gestisce i dati *senza* usare le classi.

```
studente = {"nome": "Anna Rossi", "voti": [28, 30, 25]}

def media(s):
    return sum(s["voti"]) / len(s["voti"])

print(media(studente))
print(studente)
```

Che cosa compare quando lo esegui

```
27.666666666666668
{'nome': 'Anna Rossi', 'voti': [28, 30, 25]}
```

Perché: lo studente è un dizionario e la funzione `media()` sta per conto suo. Funziona, ma nota la seconda riga stampata: Python ti mostra il dizionario nudo, con le graffe. Non sa che quello è uno studente, perché per lui è solo un dizionario come tanti.

Prova a cambiare: aggiungi una nuova chiave scrivendo `studente["eta"] = 25` e ristampa `studente`. Python la accetta senza dire niente. Nessuno controlla che uno studente abbia le chiavi giuste: è comodo adesso, sarà un problema fra dieci pagine.

1.2 Tutto quello che può andare storto

Guardiamo cosa succede a questo codice nel mondo reale.

Nessuno garantisce la forma dei dati. Un dizionario accetta qualunque chiave. Se da qualche parte nel programma scrivi `{"nome": "Sara", "voti": []}` dimenticando l'email, Python non protesta: l'errore esploderà molto più tardi, in un punto del codice che con quella riga non c'entra niente.

```
studente3 = {"nome": "Sara Verdi", "voto": [30]}    # "voto", non "voti"
media(studente3)    # KeyError: 'voti' -- ma solo quando qualcuno chiama media()
```

I dati e le funzioni che li usano vivono lontani. La funzione `media()` ha senso solo per gli studenti, ma nel file sta insieme a tutte le altre. Fra un mese ci saranno anche `media_docenti()`, `media_aule()`, e nessuno saprà più quale funzione lavora su quale dizionario. Chi legge il codice deve tenere a mente un'associazione che il codice non esprime.

Chiunque può scrivere qualsiasi cosa. Niente impedisce di fare così:

```
studente1["voti"].append(45)      # un voto da 45? in trentesimi?
studente1["email"] = None         # email cancellata, nessun controllo
studente1["nome"] = 12345         # il nome ora è un numero
```

Non esiste un punto unico in cui mettere il controllo «il voto deve stare fra 0 e 30». Dovresti ripeterlo in ogni punto del programma che tocca i voti — e prima o poi qualcuno se lo dimenticherà.

PROVA TU · 2 Che cosa succede se i dati si rovinano

Adesso maltrattiamo di proposito quel dizionario, per vedere come reagisce il programma.

```
studente = {"nome": "Anna Rossi", "voti": [28, 30, 25]}

def media(s):
    return sum(s["voti"]) / len(s["voti"])

studente["voti"].append(45)
print("media dopo aver aggiunto un 45:", media(studente))

studente["voti"] = "trenta"
print("media dopo aver rovinato i voti:", media(studente))
```

Che cosa compare quando lo esegui (e' voluto!)

```
media dopo aver aggiunto un 45: 32.0
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Perché: guarda bene le due cose diverse che sono successe. Un voto 45 (impossibile in trentesimi) è stato **accettato in silenzio**, e la media è semplicemente sbagliata: nessun errore, nessun avviso. Poi, sostituendo la lista con una stringa, il programma si è fermato — ma si è fermato dentro `media()`, non sulla riga che ha causato il guaio.

Prova a cambiare: immagina che le due righe che rovinano i dati siano in un altro file, scritte da un compagno di corso la settimana scorsa. Quanto ci metteresti a trovarle? Nel capitolo 4 costruirai una classe in cui queste due righe non sono proprio possibili.

Aggiungere un comportamento significa toccare tutto. Se domani serve la lode, o la media pesata, o lo storico delle modifiche, devi cercare in tutto il progetto ogni funzione che manipola gli studenti e aggiornarla.

NOTA

Il problema di fondo non è la sintassi: è che **i dati e le operazioni che hanno senso su quei dati sono scollegati**. Il linguaggio non sa che `media()` "appartiene" agli studenti. Lo sai solo tu, e finché lo sai solo tu, il programma dipende dalla tua memoria.

1.3 La stessa cosa, con gli oggetti

Ecco dove arriveremo alla fine del capitolo 2. Non devi capire ancora ogni riga: guarda solo la forma.

```
class Studente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self.voti = []

    def aggiungi_voto(self, voto):
        self.voti.append(voto)

    def media(self):
        if len(self.voti) == 0:
            return 0
        return sum(self.voti) / len(self.voti)

    def promosso(self):
        return self.media() >= 24

anna = Studente("Anna Rossi", "anna@example.com")
anna.aggiungi_voto(28)
anna.aggiungi_voto(30)
print(anna.media())          # 29.0
print(anna.promosso())       # True
```

Che cosa è cambiato davvero?

- **La forma è garantita.** Ogni `Studente` creato con questa classe ha un nome, un'email e una lista di voti. Non esiste uno studente "fatto male", perché c'è un solo posto in cui gli studenti nascono.
- **Il comportamento sta con i dati.** `media()` è dentro `Studente`: chiunque legga la classe vede subito tutto quello che uno studente sa fare.
- **Il codice si legge come la realtà che describe.** `anna.promosso()` si legge ad alta voce e significa esattamente quello che dice. Confrontalo con `promosso(studente1)`.
- **C'è un solo punto dove mettere i controlli.** La validazione del voto andrà dentro `aggiungi_voto()`, e varrà automaticamente per tutto il programma. Lo faremo nel capitolo 4.

Questo è il patto della programmazione a oggetti: **tieni insieme i dati e le operazioni che su quei dati hanno senso**, e chiama il risultato "oggetto".

PROVA TU · 3 La stessa cosa, scritta con una classe

Ecco lo stesso programma di prima, ma con una classe. Non devi capire ancora ogni riga: guarda com'è organizzato.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome
```

```

        self.voti = []

    def aggiungi_voto(self, voto):
        self.voti.append(voto)

    def media(self):
        if len(self.voti) == 0:
            return 0
        return sum(self.voti) / len(self.voti)

anna = Studente("Anna Rossi")
anna.aggiungi_voto(28)
anna.aggiungi_voto(30)

luca = Studente("Luca Bianchi")
luca.aggiungi_voto(18)

print(anna.nome, "ha media", anna.media())
print(luca.nome, "ha media", luca.media())

```

Che cosa compare quando lo esegui

```

Anna Rossi ha media 29.0
Luca Bianchi ha media 18.0

```

Perché: hai scritto `aggiungi_voto` e `media` **una volta sola**, e funzionano per Anna, per Luca e per qualunque altro studente creerai. E la scrittura `anna.media()` si legge come una frase: «la media di Anna». Confrontala con `media(studente1)` della prova 1.

Prova a cambiare: crea un terzo studente di nome Sara **senza** dargli nessun voto, e stampa la sua media. Ottieni `0` invece di un errore, perché il controllo su lista vuota è scritto una volta dentro la classe e vale per tutti.

1.4 Il vocabolario minimo: classe, istanza, campo, metodo

Quattro parole, e le useremo fino all'ultima pagina. La metafora più onesta è quella dello *stampo*.

Termine	Che cos'è	Nell'esempio
Classe	Il progetto, lo stampo. Descrive com'è fatto e cosa sa fare un certo tipo di cosa. Non contiene dati di nessuno in particolare.	<code>Studente</code>
Oggetto (o istanza)	Una cosa concreta costruita a partire dallo stampo, con i suoi dati. Da una classe puoi crearne quante ne vuoi.	<code>anna</code> , <code>luca</code>
Campo (o attributo)	Un dato che l'oggetto porta con sé. È lo <i>stato</i> dell'oggetto.	<code>nome</code> , <code>email</code> , <code>voti</code>
Metodo	Una funzione che appartiene alla classe e lavora sui campi dell'oggetto. È il <i>comportamento</i> .	<code>media()</code> , <code>promosso()</code>

Detto in una frase: **una classe è uno stampo per biscotti, un oggetto è il biscotto**. Lo stampo non si mangia e non ha sapore; ogni biscotto uscito da quello stampo ha la stessa forma ma può avere la sua glassa. Se ti serve un'immagine più tecnica: la classe è la planimetria, l'oggetto è la casa costruita. Puoi costruire cento case dalla stessa planimetria, e ognuna avrà i suoi inquilini.

ATTENZIONE

Stai già usando oggetti da quando hai scritto la tua prima riga di Python: `"ciao".upper()` è un metodo dell'oggetto stringa, `[3,1,2].sort()` è un metodo dell'oggetto lista. In Python *tutto* è un oggetto. La novità del capitolo 2 non è usarli: è imparare a costruirne di tuoi.

PROVA TU · 4 Stai già usando oggetti da sempre

Un'ultima cosa prima di iniziare davvero: gli oggetti non sono una novità, li usi da quando hai scritto la tua prima riga di Python.

```
print(type("ciao"))
print(type([1, 2, 3]))
print(type(42))

parola = "ciao"
print(parola.upper())

numeri = [3, 1, 2]
numeri.sort()
print(numeri)
```

Che cosa compare quando lo esegui

```
<class 'str'>
<class 'list'>
<class 'int'>
CIAO
[1, 2, 3]
```

Perché: `type()` risponde sempre `<class ...>`: stringhe, liste e numeri sono *tutti* oggetti creati a partire da una classe. E `parola.upper()` è un metodo chiamato con il punto, esattamente come `anna.media()` della prova precedente. La novità del prossimo capitolo non è usare gli oggetti: è imparare a costruirne di tuoi.

Prova a cambiare: scrivi `print(dir("ciao"))`: compare l'elenco di tutto quello che una stringa sa fare. Per ora ignora i nomi con i doppi trattini bassi all'inizio, guarda gli altri: sono i metodi che già conosci.

1.5 La mappa del viaggio

La programmazione a oggetti si regge su quattro idee. Le incontrerai una per capitolo, sempre partendo da un problema concreto e mai dalla definizione.

Idea	La domanda a cui risponde	Capitolo
Incapsulamento	Come impedisco al resto del programma di rompere i miei dati?	4
Ereditarietà	Come riuso una classe che è quasi giusta, senza copiarla?	5
Polimorfismo	Come tratto cose diverse allo stesso modo?	6
Astrazione	Come descrivo cosa serve, senza dire come è fatto?	6 e 7

Alla fine del capitolo 7 avrai costruito un componente vero — una gerarchia di sorgenti dati riutilizzabile — che ritroverai nelle lezioni su Git, sui test, sulle API REST e su MongoDB. Non è un esercizio da buttare: è il primo mattone del tuo codice da data engineer.

Esercizio 1 *Il codice che scricchiola*

●○○○

Ecco un frammento di codice procedurale per gestire i sensori di uno stabilimento.

```
s1 = {"id": "T-01", "tipo": "temperatura", "letture": [21.5, 22.0]}
s2 = {"id": "P-07", "tipo": "pressione", "letture": []}

def ultima_lettura(s):
    return s["letture"][-1]

def registra(s, valore):
    s["letture"].append(valore)

def in_allarme(s):
    return ultima_lettura(s) > 30
```

Rispondi in italiano, senza scrivere codice:

1. Individua **almeno tre** situazioni in cui questo codice si rompe o si comporta in modo scorretto. Per ognuna, scrivi la riga di codice che causerebbe il problema.
2. La funzione `in_allarme()` usa la soglia 30 per tutti i sensori. Perché è un problema, considerando che `s2` misura una pressione?
3. Elenca quali dovrebbero essere i **campi** e quali i **metodi** di una futura classe `Sensore`.

Soluzione in Appendice A.

RIEPILOGO DEL CAPITOLO

- Senza oggetti, **dati e comportamento restano scollegati**: nessuno garantisce la forma dei dati, i controlli si ripetono ovunque e ogni modifica tocca mezzo programma.
- Una **classe** è lo stampo, un **oggetto** (o istanza) è la cosa concreta costruita da quello stampo.
- I **campi** sono lo stato dell'oggetto, i **metodi** sono il suo comportamento.
- In Python usi già oggetti ovunque: stringhe, liste e dizionari sono oggetti con i loro metodi.

Classi, oggetti, campi e metodi

IN QUESTO CAPITOLO

- Scriverai la tua prima classe e creerai oggetti a partire da essa.
- Capirai a cosa serve `__init__` e cosa significa davvero `self`.
- Imparerai a rendere i tuoi oggetti stampabili con `__str__`.
- Distinguerai gli attributi di istanza da quelli di classe — una delle confusioni più frequenti.

2.1 La prima classe

La classe più semplice che Python accetta è questa:

```
class Studente:
    pass
```

Tre cose da notare subito. La parola chiave è `class`. Il nome si scrive in **MaiuscoloCammello** (`Studente`, `ContoBancario`, `SorgenteDati`): è una convenzione universale in Python, e serve a distinguere a colpo d'occhio le classi dalle funzioni e dalle variabili, che si scrivono invece in `minuscolo_con_underscore`. Il corpo della classe è indentato, come per le funzioni; `pass` serve solo perché Python non accetta un blocco vuoto.

Da questa classe posso già creare oggetti, chiamandola come se fosse una funzione:

```
a = Studente()
b = Studente()

print(type(a))      # <class '__main__.Studente'>
print(a is b)       # False: sono due oggetti diversi
```

`Studente()` con le parentesi significa «costruisci un nuovo studente». È un'operazione diversa da `Studente` senza parentesi, che è la classe stessa. Confondere le due cose è un errore classico: `Studente` è lo stampo, `Studente()` è il biscotto appena sfornato.

Il problema è che questi oggetti sono vuoti: non hanno né dati né comportamento. Rimediamo.

PROVA TU · 5 La classe piu' semplice possibile

Partiamo dalla classe più piccola che Python accetta, per vedere che cosa succede quando creiamo due oggetti.

```
class Studente:
    pass

a = Studente()
b = Studente()

print(type(a))
print(a is b)
print(a.__dict__)
```

Che cosa compare quando lo esegui

```
<class '__main__.Studente'>
False
{}
```

Perché: la riga `a is b` chiede a Python: «*a* e *b* sono lo stesso identico oggetto?». La risposta è `False`: anche se sono uguali e vuoti, sono due oggetti distinti, come due biscotti usciti dallo stesso stampo. L'ultima riga stampa `{}`, cioè un dizionario vuoto: è il posto dove Python tiene i dati dell'oggetto, e per ora non ce n'è nessuno.

Prova a cambiare: aggiungi in fondo `print(Studente)`. Vedrai che `Studente` (senza parentesi) è la *classe*, cioè lo stampo, mentre `Studente()` (con le parentesi) crea un oggetto nuovo. Sono due cose diverse ed è importante non confonderle.

2.2 Il costruttore `__init__`

Vogliamo che ogni studente nasca già con un nome e un'email. Il posto dove dirlo è un metodo speciale chiamato `__init__` (due underscore prima e due dopo), che Python esegue **automaticamente ogni volta che crei un oggetto**.

```
class Studente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self.voti = []
```

```
anna = Studente("Anna Rossi", "anna@example.com")
print(anna.nome)      # Anna Rossi
print(anna.voti)      # []
```

Segui il flusso di quello che succede quando scrivi `Studente("Anna Rossi", "anna@example.com")`:

1. Python crea in memoria un oggetto nuovo e vuoto di tipo `Studente`.
2. Chiama `__init__` passando **quell'oggetto appena creato come primo argomento** (`self`), e poi i tuoi argomenti (`"Anna Rossi"` e `"anna@example.com"`).

3. Il corpo di `__init__` riempie l'oggetto: crea i campi `nome`, `email` e `voti`.
4. L'oggetto riempito viene restituito e finisce nella variabile `anna`.

Ecco perché `__init__` non ha bisogno di `return`: non deve restituire l'oggetto, deve solo *riempirlo*. Il suo nome, del resto, viene da *initialize*, non da *create*.

ERRORE COMUNE

Il metodo si chiama `__init__` con **due** underscore per lato, non uno. Se scrivi `_init_` o `__init_` Python non protesta e non lo chiama mai: otterrai oggetti vuoti e un `AttributeError` alla prima riga che usa un campo. Se un oggetto sembra misteriosamente privo dei suoi campi, la prima cosa da controllare sono gli underscore.

I parametri di `__init__` funzionano come quelli di qualsiasi funzione: puoi dare valori di default, e puoi passare gli argomenti per nome.

```
class Studente:
    def __init__(self, nome, email, matricola=None):
        self.nome = nome
        self.email = email
        self.matricola = matricola
        self.voti = []

anna = Studente("Anna Rossi", "anna@example.com", matricola="S2024-018")
luca = Studente("Luca Bianchi", "luca@example.com") # matricola vale None
```

PROVA TU · 6 Guardiamo il costruttore mentre lavora

Mettiamo un `print` dentro `__init__`, così vediamo con i nostri occhi *quando* viene eseguito.

```
class Studente:
    def __init__(self, nome, email):
        print("sto costruendo lo studente", nome)
        self.nome = nome
        self.email = email
        self.voti = []

print("prima")
anna = Studente("Anna", "anna@example.com")
luca = Studente("Luca", "luca@example.com")
print("dopo")
print(anna.__dict__)
```

Che cosa compare quando lo esegui

```
prima
sto costruendo lo studente Anna
sto costruendo lo studente Luca
dopo
{'nome': 'Anna', 'email': 'anna@example.com', 'voti': []}
```

Perché: non hai mai scritto `anna.__init__(...)` da nessuna parte, eppure il messaggio compare due volte: **Python esegue `__init__` da solo**, una volta per ogni oggetto che crei. E l'ultima riga mostra che ora l'oggetto non è più vuoto: contiene i tre campi che il costruttore gli ha messo dentro.

Prova a cambiare: aggiungi `return self.nome` come ultima riga di `__init__` ed esegui: ottieni `TypeError: __init__() should return None`. Il costruttore non deve *restituire* l'oggetto, deve solo riempirlo.

PROVA TU · 7 Sbagliamo i trattini bassi di proposito

Il costruttore si chiama `__init__` con **due** trattini bassi prima e due dopo. Vediamo che succede scrivendone uno solo.

```
class Studente:
    def _init_(self, nome):          # sbagliato: un solo trattino per lato
        self.nome = nome

anna = Studente("Anna")
print(anna.nome)
```

Che cosa compare quando lo esegui (e' voluto!)

```
TypeError: Studente() takes no arguments
```

Perché: per Python `_init_` è un metodo qualunque, non il costruttore. Quindi la classe è rimasta *senza* costruttore, e una classe senza costruttore non accetta argomenti: ecco il messaggio *takes no arguments*.

Prova a cambiare: toglì l'argomento e scrivi `anna = Studente()`, poi `print(anna.__dict__)`. Adesso l'oggetto viene creato, ma è **vuoto**: `{}`. Ricordati questi due messaggi, perché capiterà a tutti di sbagliare un trattino.

PROVA TU · 8 Dare un valore predefinito a un dato

A volte un dato non si conosce al momento della creazione. Si può dargli un valore di partenza.

```
class Studente:
    def __init__(self, nome, email, matricola=None):
        self.nome = nome
        self.email = email
        self.matricola = matricola
```

```
a = Studente("Anna", "anna@example.com", "S-018")
b = Studente("Luca", "luca@example.com")
c = Studente(email="sara@example.com", nome="Sara")

print(a.matricola)
print(b.matricola)
print(c.nome, c.email, c.matricola)
```

Che cosa compare quando lo esegui

```
S-018
None
Sara sara@example.com None
```

Perché: lo studente `b` è stato creato senza matricola e non dà errore: prende il valore predefinito `None`, che in Python significa «nessun valore». Lo studente `c` mostra un'altra comodità: puoi passare i dati **indicando il nome** (`email=...`, `nome=...`), anche in ordine diverso.

Prova a cambiare: prova a scrivere il costruttore mettendo il parametro con il valore predefinito per primo, così: `def __init__(self, matricola=None, nome, email):`. Python si rifiuta di partire con un `SyntaxError`: i parametri con valore predefinito devono stare **dopo** quelli obbligatori.

2.3 Che cos'è davvero `self`

`self` è la sola cosa che confonde tutti all'inizio, quindi diciamola nel modo più diretto possibile:

NOTA

`self` è l'**oggetto su cui il metodo è stato chiamato**. Quando scrivi `anna.media()`, dentro `media()` la variabile `self` vale `anna`. Quando scrivi `luca.media()`, lo stesso identico codice vede `self` che vale `luca`.

È il meccanismo che permette a un solo blocco di codice — scritto una volta nella classe — di lavorare su migliaia di oggetti diversi. Il metodo non sa in anticipo su quale studente sta lavorando: glielo dice `self`, ogni volta.

Python passa `self` automaticamente. Queste due righe sono equivalenti, e la seconda mostra cosa sta succedendo dietro le quinte:

```
anna.media()           # il modo normale di scriverlo
Studente.media(anna)   # quello che Python fa davvero: passa anna come self
```

Ecco perché **ogni metodo deve dichiarare `self` come primo parametro**, anche se non lo scrivi mai quando lo chiami. Se lo dimentichi:

```
class Studente:
    def saluta():
        print("ciao")

anna = Studente()
anna.saluta()
# TypeError: saluta() takes 0 positional arguments but 1 was given
```

Quel messaggio d'errore ora ha senso: tu pensavi di passare zero argomenti, ma Python ne ha passato uno (l'oggetto), e il metodo non aveva un posto dove metterlo.

ATTENZIONE

Il nome `self` non è una parola chiave del linguaggio: tecnicamente potresti chiamarlo `io` o `questo`. **Non farlo mai.** Ogni programmatore Python al mondo si aspetta `self`, e cambiarlo rende il tuo codice illeggibile per chiunque altro.

PROVA TU · 9 Che cos'è self, dimostrato

Questa è la prova che chiarisce `self` una volta per tutte. Facciamo restituire al metodo proprio `self`, e poi controlliamo di che oggetto si tratta.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome

    def chi_sono(self):
        print("mi hanno chiamato su:", self.nome)
        return self

anna = Studente("Anna")
luca = Studente("Luca")

print(anna.chi_sono() is anna)
print(anna.chi_sono() is luca)
print(luca.chi_sono() is luca)
```

Che cosa compare quando lo esegui

```
mi hanno chiamato su: Anna
True
mi hanno chiamato su: Anna
False
mi hanno chiamato su: Luca
True
```

Perché: `self` è semplicemente **l'oggetto scritto a sinistra del punto**. Quando scrivi `anna.chi_sono()`, dentro il metodo `self` vale `anna`; quando scrivi `luca.chi_sono()`, lo stesso

identico codice vede `self` che vale `luca`. È questo che permette a un metodo scritto una volta sola di funzionare con migliaia di oggetti diversi.

Prova a cambiare: aggiungi un terzo studente e chiamaci sopra `chi_sono()`. Il metodo nella classe non cambia di una virgola, ma stampa il nome giusto.

PROVA TU · 10 Le due scritture equivalenti

Vediamo che cosa fa davvero Python quando chiami un metodo con il punto.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome

    def saluta(self):
        print("ciao, sono", self.nome)

anna = Studente("Anna")

anna.saluta()
Studente.saluta(anna)
```

Che cosa compare quando lo esegui

```
ciao, sono Anna
ciao, sono Anna
```

Perché: le due righe finali fanno la stessa cosa. La seconda rende visibile il meccanismo nascosto: scrivere `anna.saluta()` significa chiedere alla classe di eseguire il metodo **passandogli `anna` come primo argomento**. Ecco perché `self` va sempre scritto nella definizione del metodo, ma non si passa mai quando lo chiami.

Prova a cambiare: nella vita reale userai sempre la prima forma. La seconda serve solo a capire: adesso sai da dove salta fuori quell'argomento in più di cui parla l'errore della prova successiva.

PROVA TU · 11 L'errore che faranno tutti almeno una volta

Proviamo a scrivere un metodo dimenticando `self`, così impari a riconoscere il messaggio.

```
class Studente:
    def saluta():
        print("ciao")

anna = Studente()
anna.saluta()
```

Che cosa compare quando lo esegui (e' voluto!)

```
TypeError: Studente.saluta() takes 0 positional arguments but 1 was given
```

Perché: il messaggio sembra assurdo: dice che hai passato un argomento, ma tu hai scritto solo `anna.saluta()`, senza niente fra le parentesi. L'argomento in più è proprio `anna`, che Python aggiunge sempre in automatico (l'hai visto nella prova 10). Il metodo però non aveva un posto dove metterlo, perché le sue parentesi erano vuote.

Prova a cambiare: correggi scrivendo `def saluta(self):` ed esegui di nuovo: adesso stampa `ciao`. Da oggi, quando leggerai *takes 0 positional arguments but 1 was given*, saprai già che cosa cercare.

2.4 I campi: lo stato dell'oggetto

Ogni `self.qualcosa = valore` dentro un metodo crea (o aggiorna) un campo di quell'oggetto. I campi sono la **memoria** dell'oggetto: restano lì fra una chiamata e l'altra, ed è ciò che distingue un oggetto da una semplice funzione.

```
anna = Studente("Anna Rossi", "anna@example.com")

print(anna.nome)           # Leggo un campo
anna.nome = "Anna Rossi-Neri" # scrivo un campo
anna.corso = "Data Engineer" # ne aggiungo uno al volo (si può, ma...)
```

Sì, in Python puoi aggiungere campi a un oggetto dall'esterno, in qualsiasi momento. È una libertà pericolosa: quel campo `corso` esisterà solo su `anna` e non sugli altri studenti, e chi legge la classe non ne saprà nulla. **Regola pratica: tutti i campi vanno creati in `__init__`**, anche quelli che all'inizio valgono `None` o lista vuota. Così la classe documenta da sola tutto lo stato che un oggetto può avere.

PROVA TU · 12 Ogni oggetto ha i suoi dati

Verifichiamo che i campi appartengano davvero al singolo oggetto e non alla classe.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome

anna = Studente("Anna")
luca = Studente("Luca")

anna.corso = "Data Engineer"           # aggiunto SOLO ad anna

print(anna.__dict__)
print(luca.__dict__)
print(luca.corso)
```

Che cosa compare quando lo esegui (e' voluto!)

```
{'nome': 'Anna', 'corso': 'Data Engineer'}
{'nome': 'Luca'}
AttributeError: 'Studiante' object has no attribute 'corso'
```

Perché: i due oggetti hanno contenuti diversi. Python ti *permette* di attaccare un campo nuovo a un oggetto già creato, ma quel campo esiste solo su `anna`: qualsiasi codice che lo usi si romperà su tutti gli altri studenti, come dimostra l'ultima riga.

Prova a cambiare: sposta il campo dentro il costruttore scrivendo `self.corso = None` in `__init__`, e togli la riga `anna.corso = ...`. Adesso *tutti* gli studenti hanno il campo `corso` e nessuno si rompe. **La regola:** tutti i campi si dichiarano dentro `__init__`, anche quelli che all'inizio non hanno un valore.

2.5 I metodi: il comportamento dell'oggetto

Un metodo è una funzione definita dentro la classe. La differenza rispetto a una funzione normale è che riceve `self` e quindi può leggere e modificare i campi dell'oggetto.

```
class Studiante:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self.voti = []

    def aggiungi_voto(self, voto):
        """Registra un nuovo voto per questo studente."""
        self.voti.append(voto)

    def media(self):
        """Restituisce la media dei voti, 0 se non ce ne sono."""
        if len(self.voti) == 0:
            return 0
        return sum(self.voti) / len(self.voti)

    def promosso(self):
        """True se la media raggiunge la sufficienza (24)."""
        return self.media() >= 24
```

```
anna = Studiante("Anna Rossi", "anna@example.com")
anna.aggiungi_voto(28)
anna.aggiungi_voto(30)
print(anna.media())      # 29.0
print(anna.promosso())   # True
```

Due dettagli che valgono più di quanto sembri.

Un metodo può chiamare un altro metodo dello stesso oggetto, ma deve passare da `self`: dentro `promosso()` scrivo `self.media()`, non `media()`. Senza `self.`, Python cercherebbe una funzione globale chiamata `media` e non la troverebbe.

Le stringhe fra triplice virgolette sotto la firma del metodo sono *docstring*: la documentazione del metodo. Non sono commenti decorativi, sono accessibili a runtime con `help(Studente)` e vengono lette dagli editor per mostrarti i suggerimenti. Prendi l'abitudine di scriverle: una riga che dica cosa fa il metodo, non come.

PROVA TU · 13 Dentro un metodo serve sempre self

Un metodo che ne chiama un altro deve passare da `self`. Vediamo che succede se lo dimentichiamo.

```
class Studente:
    def __init__(self, voti):
        self.voti = voti

    def media(self):
        return sum(self.voti) / len(self.voti)

    def promosso(self):
        return media() >= 24      # sbagliato: manca self.

anna = Studente([28, 30])
print(anna.media())
print(anna.promosso())
```

Che cosa compare quando lo esegui (e' voluto!)

```
29.0
NameError: name 'media' is not defined. Did you mean: 'self.media'?
```

Perché: la prima riga funziona, la seconda no. Dentro un metodo, gli altri metodi della classe **non sono nomi visibili da soli**: Python cerca una funzione chiamata `media` fuori dalla classe, non la trova, e si ferma. Per raggiungerla devi partire dall'oggetto: `self.media()`. Nota che Python ti suggerisce da solo la correzione: *Did you mean: 'self.media'?*, cioè «intendevi `self.media` ?». Conviene sempre leggere fino in fondo i messaggi d'errore.

Prova a cambiare: correggi scrivendo `return self.media() >= 24` ed esegui: stampa `True`. La stessa regola vale per i dati: dentro i metodi si scrive sempre `self.voti`, mai `voti` da solo.

PROVA TU · 14 Le frasi di documentazione servono davvero

Le stringhe fra tre virgolette sotto la definizione non sono commenti: Python le conserva e te le può rileggere.

```
class Studente:
    """Rappresenta uno studente iscritto al corso."""

    def __init__(self, nome):
        self.nome = nome

    def media(self):
```

```
        """Restituisce la media dei voti dello studente."""
        return 0

print(Studente.__doc__)
print(Studente.media.__doc__)
```

Che cosa compare quando lo esegui

```
Rappresenta uno studente iscritto al corso.
Restituisce la media dei voti dello studente.
```

Perché: queste frasi si chiamano *docstring* e sono documentazione vera, leggibile mentre il programma gira. Sono anche quelle che il tuo editor ti mostra nel suggerimento quando scrivi `anna.media()`, ed è il motivo per cui conviene scriverne una per ogni metodo.

Prova a cambiare: prova anche `help(Studente)`: Python ti costruisce una piccola pagina di manuale della tua classe, usando le frasi che hai scritto tu.

2.6 Tante istanze, tante vite separate

Questo è il punto in cui gli oggetti smettono di sembrare dizionari complicati. Da una classe puoi creare quanti oggetti vuoi, e **ognuno ha la sua copia dei campi**.

```
anna = Studente("Anna Rossi", "anna@example.com")
luca = Studente("Luca Bianchi", "luca@example.com")

anna.aggiungi_voto(30)
anna.aggiungi_voto(28)
luca.aggiungi_voto(18)

print(anna.voti)      # [30, 28]
print(luca.voti)      # [18]
print(anna.media())   # 29.0
print(luca.media())   # 18.0
```

Il codice di `aggiungi_voto()` esiste *una volta sola*, nella classe. I dati esistono invece in tante copie quante sono le istanze. Questa separazione fra "comportamento condiviso" e "stato individuale" è il cuore dell'intera faccenda.

Il modo naturale di gestirne molti è una lista:

```

classe = [
    Studente("Anna Rossi", "anna@example.com"),
    Studente("Luca Bianchi", "luca@example.com"),
    Studente("Sara Verdi", "sara@example.com"),
]

for s in classe:
    s.aggiungi_voto(25)

promossi = [s.nome for s in classe if s.promosso()]
print(promossi)          # ['Anna Rossi', 'Luca Bianchi', 'Sara Verdi']

```

PROVA TU · 15 Un solo codice, tanti dati diversi

Creiamo tre studenti e diamo dei voti solo ad alcuni, per vedere che ognuno ha la sua vita.

```

class Studente:
    def __init__(self, nome):
        self.nome = nome
        self.voti = []

    def aggiungi_voto(self, voto):
        self.voti.append(voto)

    def media(self):
        if len(self.voti) == 0:
            return 0
        return sum(self.voti) / len(self.voti)

classe = [Studente("Anna"), Studente("Luca"), Studente("Sara")]

classe[0].aggiungi_voto(30)
classe[0].aggiungi_voto(28)
classe[1].aggiungi_voto(18)

for s in classe:
    print(s.nome, s.voti, s.media())

print("Anna e Luca hanno la stessa lista di voti?", classe[0].voti is classe[1].voti)

```

Che cosa compare quando lo esegui

```

Anna [30, 28] 29.0
Luca [18] 18.0
Sara [] 0
Anna e Luca hanno la stessa lista di voti? False

```

Perché: il codice di `aggiungi_voto` esiste in un punto solo, dentro la classe. I dati invece esistono in tre copie separate, una per studente: Sara ha ancora la lista vuota. L'ultima riga lo conferma. Questa divisione fra *comportamento condiviso* e *dati personali* è il cuore di tutta la programmazione a oggetti.

Prova a cambiare: aggiungi un quarto studente alla lista e rilancia: il ciclo `for` funziona senza modifiche. Sta già trattando allo stesso modo oggetti diversi, ed è un'anticipazione del capitolo 6.

2.7 Stampare un oggetto: `__str__` e `__repr__`

Prova a stampare uno studente così com'è:

```
print(anna)
# <__main__.Studente object at 0x000001C3F8A2D9D0>
```

Illeggibile. Quel numero è l'indirizzo di memoria dell'oggetto, e non serve a niente. Python però ti lascia decidere come deve apparire un tuo oggetto, definendo il metodo speciale `__str__`:

```
class Studente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self.voti = []

    def media(self):
        if len(self.voti) == 0:
            return 0
        return sum(self.voti) / len(self.voti)

    def __str__(self):
        return f"{self.nome} ({self.email}) - media {self.media():.1f}"
```

```
anna = Studente("Anna Rossi", "anna@example.com")
anna.voti = [28, 30]
print(anna)          # Anna Rossi (anna@example.com) - media 29.0
```

`__str__` deve **restituire** una stringa, non stamparla: è `print()` a stampare. Se dentro `__str__` scrivi `print(...)` invece di `return`, otterrai un errore poco chiaro.

Esiste anche `__repr__`, che è la rappresentazione "tecnica", pensata per chi sviluppa: è quella che vedi nella console interattiva e, cosa importante, **è quella usata quando stampi una lista di oggetti**.

```
print([anna, luca])
# [<__main__.Studente object at 0x...>, <...>] se manca __repr__
```

```
def __repr__(self):
    return f"Studente({self.nome!r}, {self.email!r})"
```

```
print([anna, luca])
# [Studente('Anna Rossi', 'anna@example.com'), Studente('Luca Bianchi', 'Luca@example.com')]
```

La convenzione è che `__str__` sia per l'utente finale e `__repr__` per lo sviluppatore, e che `__repr__` assomigli il più possibile al codice necessario per ricreare l'oggetto. Se ne definisci uno solo, definisci `__repr__`: quando `__str__` manca, Python ripiega su `__repr__`, mentre il contrario non vale.

PROVA TU · 16 Rendere leggibile un oggetto quando lo stampi

Confrontiamo tre classi identiche nei dati ma diverse nel modo in cui si fanno stampare.

```
class Base:
    def __init__(self, nome):
        self.nome = nome

class ConStr:
    def __init__(self, nome):
        self.nome = nome

    def __str__(self):
        return "studente " + self.nome

class ConStrERepr:
    def __init__(self, nome):
        self.nome = nome

    def __str__(self):
        return "studente " + self.nome

    def __repr__(self):
        return "ConStrERepr('" + self.nome + "')"

a = Base("Anna")
b = ConStr("Anna")
c = ConStrERepr("Anna")

print("1)", a)
print("2)", b)
print("3)", [b])
print("4)", c)
print("5)", [c])
```

Che cosa compare quando lo esegui

```
1) <__main__.Base object at 0x000001C3F8A2D9D0>
2) studente Anna
3) [<__main__.ConStr object at 0x000001C3F8A2DA10>]
4) studente Anna
5) [ConStrERepr('Anna')]
```

Perché: la riga 1 è quello che Python stampa quando non gli dici niente: un indirizzo di memoria illeggibile (sul tuo computer i numeri dopo `0x` saranno diversi, è normale). Con `__str__` decidi tu

come appare l'oggetto, e infatti le righe 2 e 4 sono leggibili. Ma guarda la riga 3: dentro una **lista** l'oggetto torna illeggibile! Per quel caso serve `__repr__`, come mostra la riga 5.

Prova a cambiare: la regola pratica: se ne scrivi uno solo, scrivi `__repr__`, perché quando manca `__str__` Python usa `__repr__` al suo posto. Provalo: cancella `__str__` dalla terza classe ed esegui di nuovo — la riga 4 continuerà a funzionare.

2.8 Attributi di classe e attributi di istanza

Tutti i campi visti finora nascono dentro `__init__` con `self.`, e appartengono al singolo oggetto: sono **attributi di istanza**. Puoi però definire una variabile direttamente nel corpo della classe: quello è un **attributo di classe**, ed è *condiviso da tutte le istanze*.

```
class Studente:
    SOGLIA_PROMOZIONE = 24          # attributo di CLASSE: uno solo, condiviso
    numero_studenti = 0             # attributo di CLASSE: un contatore comune

    def __init__(self, nome):
        self.nome = nome            # attributo di ISTANZA: uno per oggetto
        self.voti = []
        Studente.numero_studenti += 1

    def promosso(self):
        return self.media() >= Studente.SOGLIA_PROMOZIONE
```

```
a = Studente("Anna")
b = Studente("Luca")
print(Studente.numero_studenti)    # 2
print(a.SOGLIA_PROMOZIONE)         # 24 -- leggibile anche dall'istanza
```

L'uso più frequente e più utile degli attributi di classe sono le **costanti**, scritte in MAIUSCOLO: una soglia, un valore di default, un formato. Metterle lì significa avere un solo punto da modificare quando la regola cambia, invece di un `24` sparso in cinque metodi.

ERRORE COMUNE

Non mettere **mai** una lista o un dizionario come attributo di classe pensando che sia un valore iniziale per ogni oggetto:

```
class Studente:
    voti = []          # SBAGLIATO: una sola lista per TUTTI gli studenti

a = Studente(); b = Studente()
a.voti.append(30)
print(b.voti)         # [30] -- il voto di a è finito anche su b!
```

I tipi mutabili vanno sempre creati dentro `__init__` con `self.voti = []`, così ogni oggetto riceve la sua lista. Capirai perfettamente il perché nel capitolo 3.

PROVA TU · 17 Un dato condiviso da tutti gli oggetti

Un valore scritto nel corpo della classe, fuori dai metodi, appartiene alla classe e non al singolo oggetto.

```
class Studente:
    SOGLIA_PROMOZIONE = 24          # scritto nella classe: uno per tutti
    quanti = 0

    def __init__(self, nome):
        self.nome = nome            # scritto nel costruttore: uno per oggetto
        Studente.quanti += 1

a = Studente("Anna")
b = Studente("Luca")
c = Studente("Sara")

print("studenti creati:", Studente.quanti)
print("soglia vista da a:", a.SOGLIA_PROMOZIONE)
print("dati propri di a:", a.__dict__)

a.SOGLIA_PROMOZIONE = 18
print("dopo aver cambiato la soglia solo su a:")
print("  a:", a.SOGLIA_PROMOZIONE)
print("  b:", b.SOGLIA_PROMOZIONE)
print("  classe:", Studente.SOGLIA_PROMOZIONE)
```

Che cosa compare quando lo esegui

```
studenti creati: 3
soglia vista da a: 24
dati propri di a: {'nome': 'Anna'}
dopo aver cambiato la soglia solo su a:
  a: 18
  b: 24
  classe: 24
```

Perché: il contatore `quanti` è uno solo e cresce a ogni studente creato. La soglia è leggibile anche dal singolo oggetto (`a.SOGLIA_PROMOZIONE`), ma nota che nei *dati propri* di `a` c'è soltanto il nome: la soglia non è sua, è della classe.

Prova a cambiare: l'ultima parte mostra una cosa delicata: scrivendo `a.SOGLIA_PROMOZIONE = 18` non hai cambiato la soglia per tutti, hai creato un dato *personale* di `a` che copre quello della classe. Gli altri sono rimasti a 24. Per questo le costanti condivise si scrivono in MAIUSCOLO: ricordano che non vanno modificate sui singoli oggetti.

PROVA TU · 18 La lista condivisa: sbagliato e giusto a confronto

Questa è la prova più importante del capitolo. Mettiamo la versione sbagliata e quella giusta una accanto all'altra.

```
class CarrelloSbagliato:
    articoli = []                # attributo di CLASSE: uno solo per tutti

    def aggiungi(self, nome):
        self.articoli.append(nome)

class CarrelloGiusto:
    def __init__(self):
        self.articoli = []      # attributo di ISTANZA: uno per oggetto

    def aggiungi(self, nome):
        self.articoli.append(nome)

x1 = CarrelloSbagliato()
x2 = CarrelloSbagliato()
x1.aggiungi("Mouse")
print("sbagliato -> x1:", x1.articoli, "x2:", x2.articoli)
print("sbagliato -> e' la stessa lista?", x1.articoli is x2.articoli)

y1 = CarrelloGiusto()
y2 = CarrelloGiusto()
y1.aggiungi("Mouse")
print("giusto -> y1:", y1.articoli, "y2:", y2.articoli)
print("giusto -> e' la stessa lista?", y1.articoli is y2.articoli)
```

Che cosa compare quando lo esegui

```
sbagliato -> x1: ['Mouse'] x2: ['Mouse']
sbagliato -> e' la stessa lista? True
giusto -> y1: ['Mouse'] y2: []
giusto -> e' la stessa lista? False
```

Perché: nella versione sbagliata il mouse finisce anche nel carrello di `x2`, perché la lista scritta nel corpo della classe è **una sola**, condivisa da tutti i carrelli che creerai. Nella versione giusta la lista nasce dentro `__init__`, quindi ogni carrello riceve la sua.

Prova a cambiare: immagina questo errore in un negozio online: il prodotto messo nel carrello da un cliente comparirebbe nel carrello di tutti gli altri. **La regola:** liste, dizionari e insiemi si creano *sempre* dentro `__init__`. Numeri, testi e costanti nel corpo della classe vanno invece benissimo, come vedi nella prova 17.

Esercizio 2 Completare la classe *Studente*

●○○

Partendo dalla classe `Studente` del paragrafo 2.5, aggiungi:

1. un campo `matricola`, impostato nel costruttore;
2. un metodo `numero_esami()` che restituisce quanti voti sono stati registrati;
3. un metodo `voto_massimo()` che restituisce il voto più alto, oppure `None` se non ci sono voti;
4. un metodo `__str__` che stampi `"S2024-018 - Anna Rossi - 3 esami - media 27.7"`.

Crea poi due studenti, registra qualche voto e stampali.

Esercizio 3 La classe *Misurazione*

●●○

Scrivi da zero una classe `Misurazione` che rappresenti una singola lettura di un sensore.

- **Campi:** `sensore_id` (stringa), `valore` (float), `unita` (stringa, default `"C"`), `timestamp` (stringa).
- **Metodi:**
 - `e_valida()`: restituisce `True` se il valore è compreso fra -50 e 150;
 - `in_fahrenheit()`: restituisce il valore convertito in Fahrenheit (`valore * 9/5 + 32`), ma solo se `unita` è `"C"`, altrimenti restituisce `None`;
 - `__str__`: produce ad esempio `"[2026-03-01 10:00] T-01: 21.5 C"`.

Verifica con almeno tre misurazioni, di cui una non valida.

Esercizio 4 Il carrello

●●○

Scrivi una classe `Carrello` con:

- un campo `articoli`, inizializzato a lista vuota nel costruttore (attenzione all'errore comune del paragrafo 2.8);
- un attributo di classe `IVA = 0.22`;
- `aggiungi(nome, prezzo, quantita=1)` che memorizza l'articolo come dizionario;
- `totale_netto()` e `totale_lordo()` (netto + IVA);
- `svuota()` e `__str__` che riassume numero di articoli e totale lordo.

Crea *due* carrelli diversi e verifica che aggiungere un articolo al primo non modifichi il secondo.

Esercizio 5 *Da procedurale a oggetti*



Riprendi il codice dei sensori dell'Esercizio 1 e riscrivilo interamente come classe `Sensore`. Requisiti:

1. Il costruttore riceve `id`, `tipo` e una `soglia_allarme` propria di quel sensore (risolvendo il problema individuato al punto 2 dell'Esercizio 1).
2. Metodi `registra(valore)`, `ultima_lettura()`, `media_letture()`, `in_allarme()`.
3. `ultima_lettura()` deve restituire `None` se non ci sono letture, invece di sollevare `IndexError`.
4. Un attributo di classe `TIPI_AMMESSI = ("temperatura", "pressione", "umidita")`.

Crea infine una lista di tre sensori e scrivi un ciclo che stampi solo quelli in allarme.

RIEPILOGO DEL CAPITOLO

- `class NomeClasse:` definisce lo stampo; `NomeClasse()` crea un oggetto.
- `__init__` viene eseguito automaticamente alla creazione e serve a **riempire** l'oggetto: tutti i campi vanno dichiarati lì.
- `self` è l'oggetto su cui il metodo è stato chiamato; va scritto come primo parametro di ogni metodo, ma non si passa mai nella chiamata.
- Ogni istanza ha **la sua copia** dei campi, mentre il codice dei metodi è scritto una volta sola.
- `__str__` rende leggibile l'oggetto quando lo stampi; `__repr__` è la versione per lo sviluppatore ed è quella usata nelle liste.
- Gli attributi di classe sono condivisi da tutte le istanze: perfetti per le costanti, **pericolosi** per liste e dizionari.

Variabili che contengono oggetti

IN QUESTO CAPITOLO

- Capirai che una variabile non contiene l'oggetto, ma un *riferimento* ad esso.
- Imparerai a riconoscere e a evitare il bug più subdolo dei principianti: l'alias involontario.
- Distinguerai `is` da `==` e saprai quando usare quale.
- Costruirai oggetti che contengono altri oggetti (composizione), il mattone di ogni programma reale.

Questo è il capitolo che i corsi saltano e che poi costa ore di debug. Il programma del corso lo cita esplicitamente — «*variabili contenenti oggetti*» — e c'è un motivo: la maggior parte dei bug misteriosi nei programmi a oggetti nasce dal non aver capito questo meccanismo.

3.1 La variabile non contiene l'oggetto

Con i numeri l'intuizione funziona bene: `x = 5` mette il cinque dentro `x`. Con gli oggetti l'intuizione tradisce. Quando scrivi:

```
anna = Studente("Anna Rossi", "anna@example.com")
```

l'oggetto `Studente` viene creato in una zona di memoria, e la variabile `anna` contiene soltanto **l'indirizzo dove quell'oggetto si trova**. In gergo: `anna` è un *riferimento* all'oggetto, non l'oggetto.

L'immagine giusta è questa: la variabile non è una scatola che contiene lo studente, è **un'etichetta appiccicata sopra lo studente**. E allo stesso oggetto si possono appiccicare più etichette.

```
anna = Studente("Anna Rossi", "anna@example.com")
print(id(anna))      # 2181755134416 -- l'indirizzo dell'oggetto
```

La funzione `id()` restituisce l'identità dell'oggetto, in pratica il suo indirizzo. Ci servirà nelle prossime pagine per vedere in modo inequivocabile quanti oggetti esistono davvero.

PROVA TU · 19 Dati uguali non vuol dire stesso oggetto

Creiamo due oggetti con gli stessi identici dati e chiediamo a Python se sono la stessa cosa.

```
class Punto:
    def __init__(self, x):
        self.x = x

p = Punto(1)
q = Punto(1)

print("hanno gli stessi dati?", p.x == q.x)
print("sono lo stesso oggetto?", p is q)
print("hanno lo stesso indirizzo?", id(p) == id(q))
```

Che cosa compare quando lo esegui

```
hanno gli stessi dati? True
sono lo stesso oggetto? False
hanno lo stesso indirizzo? False
```

Perché: i due punti si assomigliano come due gocce d'acqua, ma sono due oggetti separati, in due posti diversi della memoria. La funzione `id()` restituisce l'indirizzo di un oggetto: da qui in avanti la useremo come strumento di verifica, per non dover indovinare se due nomi indicano la stessa cosa.

Prova a cambiare: aggiungi `print(id(p))` : vedrai un numero lungo, diverso sul tuo computer e diverso a ogni esecuzione. Non farci caso: quello che conta non è il numero, ma se due indirizzi sono *uguali fra loro* oppure no.

3.2 Alias: due nomi, un solo oggetto

Guarda con attenzione, perché qui si nasconde il bug:

```
anna = Studente("Anna Rossi", "anna@example.com")
b = anna                                     # NON crea un nuovo studente

b.nome = "Anna Neri"
print(anna.nome)                            # Anna Neri  <-- è cambiato anche anna!
print(id(anna) == id(b))                    # True: sono lo stesso identico oggetto
```

L'assegnazione `b = anna` **non copia niente**: copia solo l'indirizzo. Ora ci sono due etichette (`anna` e `b`) appiccate sullo stesso identico oggetto. Modificarlo attraverso una delle due etichette lo modifica per entrambe, perché l'oggetto è uno solo. Questa situazione si chiama **alias**.

Confronta con quello che succede con i numeri, dove tutto si comporta come ti aspetti:

```
x = 5
y = x
y = 10
print(x)           # 5 -- x non è cambiato
```

Perché la differenza? Perché `y = 10` non *modifica* l'oggetto 5: sposta l'etichetta `y` su un altro oggetto, il 10. Mentre `b.nome = "Anna Neri"` non sposta nessuna etichetta: **entra dentro l'oggetto e lo modifica**. È questa la distinzione che conta.

LA REGOLA IN UNA RIGA

`b = qualcosa` sposta l'etichetta `b`. `b.campo = qualcosa` modifica l'oggetto che sta sotto l'etichetta, e la modifica è visibile a **tutte** le altre etichette appiccate su di esso.

PROVA TU · 20 Due nomi per lo stesso oggetto

Adesso il punto delicato: che cosa succede davvero quando scrivi `b = anna`.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome

anna = Studente("Anna Rossi")
b = anna                                # non crea un secondo studente!

print("sono lo stesso oggetto?", anna is b)

b.nome = "Anna Neri"
print("nome letto da anna:", anna.nome)
print("nome letto da b   :", b.nome)

b = Studente("Terzo")                   # qui invece spostiamo l'etichetta b
print("dopo aver riassegnato b:")
print("  anna:", anna.nome)
print("  b   :", b.nome)
print("  sono lo stesso oggetto?", anna is b)
```

Che cosa compare quando lo esegui

```
sono lo stesso oggetto? True
nome letto da anna: Anna Neri
nome letto da b   : Anna Neri
dopo aver riassegnato b:
  anna: Anna Neri
  b   : Terzo
sono lo stesso oggetto? False
```

Perché: la riga `b = anna` **non copia nulla**: mette una seconda etichetta sullo stesso studente. Cambiando il nome tramite `b`, cambia anche per `anna`, perché lo studente è uno solo. Nella seconda parte invece `b = Studente("Terzo")` stacca l'etichetta e la attacca a un oggetto nuovo: `anna` resta dov'era.

Prova a cambiare: tieni a mente questa differenza, perché è l'origine di moltissimi errori: `b.nome = ...` entra nell'oggetto e lo modifica, `b = ...` sposta soltanto l'etichetta.

PROVA TU · 21 Perché con i numeri sembra funzionare diversamente

Con i numeri l'intuizione sembra tornare. Vediamo perché in realtà la regola è sempre la stessa.

```
x = 5
y = x
print("sono lo stesso oggetto?", x is y)

y = 10
print("x vale", x, "e y vale", y)
print("sono lo stesso oggetto adesso?", x is y)
```

Che cosa compare quando lo esegui

```
sono lo stesso oggetto? True
x vale 5 e y vale 10
sono lo stesso oggetto adesso? False
```

Perché: anche qui `y = x` ha creato due etichette sullo stesso oggetto (il numero 5). La differenza è che `y = 10` **sposta l'etichetta** su un altro numero, esattamente come `b = Studente("Terzo")` nella prova precedente. Non esiste una regola per i numeri e una per gli oggetti: cambia solo quello che stai facendo.

Prova a cambiare: non puoi «entrare dentro» il numero 5 per modificarlo, mentre puoi entrare dentro uno studente e cambiargli il nome. È per questo che con i numeri il problema non si presenta mai.

3.3 `is` contro `==`

Due domande diverse, che i principianti confondono di continuo:

- `a is b` chiede: «sono lo stesso identico oggetto?» (stesso indirizzo in memoria)
- `a == b` chiede: «hanno lo stesso valore?»

```
a1 = Studente("Anna Rossi", "anna@example.com")
a2 = Studente("Anna Rossi", "anna@example.com")  # stessi dati, oggetto diverso
a3 = a1                                           # alias di a1

print(a1 is a2)    # False: due oggetti distinti in memoria
print(a1 is a3)    # True: stesso oggetto
print(a1 == a2)    # False (!) -- vedi sotto
```

L'ultima riga sorprende: due studenti con gli stessi identici dati risultano diversi. Il motivo è che, se non dici tu il contrario, **Python confronta gli oggetti per identità**: `==` di default si comporta come `is`. Per decidere tu quando due oggetti sono "uguali", devi definire il metodo speciale `__eq__`:

```
class Studente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email

    def __eq__(self, altro):
        """Due studenti sono uguali se hanno la stessa email."""
        if not isinstance(altero, Studente):
            return NotImplemented
        return self.email == altro.email
```

```
print(a1 == a2)      # True: stessa email
print(a1 is a2)     # False: restano due oggetti diversi
```

Nota che `is` non cambia mai comportamento: l'identità non si può ridefinire, ed è proprio questo che la rende affidabile. Usa `is` solo per confronti con `None` (`if x is None:`) e per verificare che due variabili puntino allo stesso oggetto; per tutto il resto usa `==` .

PROVA TU · 22 Insegnare a Python quando due oggetti sono uguali

Mettiamo a confronto due classi identiche, tranne che la seconda spiega a Python che cosa significa «uguale».

```
class SenzaEq:
    def __init__(self, email):
        self.email = email

class ConEq:
    def __init__(self, email):
        self.email = email

    def __eq__(self, altro):
        return self.email == altro.email

a1 = SenzaEq("anna@example.com")
a2 = SenzaEq("anna@example.com")
print("senza __eq__ -> a1 == a2 :", a1 == a2)
print("senza __eq__ -> a1 is a2 :", a1 is a2)

b1 = ConEq("anna@example.com")
b2 = ConEq("anna@example.com")
print("con __eq__ -> b1 == b2 :", b1 == b2)
print("con __eq__ -> b1 is b2 :", b1 is b2)
print("con __eq__ -> b1 in [b2] :", b1 in [b2])
```

Che cosa compare quando lo esegui

```
senza __eq__ -> a1 == a2 : False
senza __eq__ -> a1 is a2 : False
con __eq__   -> b1 == b2 : True
con __eq__   -> b1 is b2 : False
con __eq__   -> b1 in [b2] : True
```

Perché: se non dici niente, per Python `==` si comporta come `is` : due oggetti sono uguali solo se sono lo stesso oggetto. Scrivendo `__eq__` decidi tu la regola — qui: stessa email, stesso studente. Nota che `is` resta `False` : l'identità non si può ridefinire, ed è proprio questo che la rende affidabile.

Prova a cambiare: guarda l'ultima riga: anche l'operatore `in` usa `__eq__` . Un solo metodo ha cambiato il comportamento delle ricerche in tutto il programma. È comodo, ma va saputo.

3.4 Oggetti dentro le liste

Una lista di oggetti è una lista di *riferimenti*. Il che significa che modificare l'oggetto attraverso la lista, o attraverso la variabile originale, è esattamente la stessa cosa.

```
anna = Studente("Anna Rossi", "anna@example.com")
classe = [anna]

classe[0].aggiungi_voto(30)
print(anna.voti)           # [30] -- anna e classe[0] sono lo stesso oggetto
```

Questo è comodissimo: puoi tenere lo stesso studente in più liste (i promossi, gli iscritti, i borsisti) senza duplicare i dati, e un aggiornamento si riflette ovunque. Ma è anche la fonte di sorprese, soprattutto quando copi una lista:

```
classe_a = [Studente("Anna", "a@x.it"), Studente("Luca", "l@x.it")]
classe_b = classe_a[:]           # copia della LISTA...

classe_b[0].nome = "Anna Neri"
print(classe_a[0].nome)          # Anna Neri -- gli oggetti sono condivisi!

classe_b.append(Studente("Sara", "s@x.it"))
print(len(classe_a))             # 2 -- ma le liste sì, sono separate
```

La copia con `[:]` (detta *copia superficiale*, o *shallow copy*) crea una nuova lista che contiene **gli stessi riferimenti**. Due contenitori distinti, gli stessi oggetti dentro. Al paragrafo 3.7 vedremo come copiare davvero.

PROVA TU · 23 Copiare una lista non copia quello che c'è dentro

Questo è il tranello che coglie di sorpresa quasi tutti. Copiamo una lista di studenti e vediamo che cosa è stato copiato davvero.

```

class Studente:
    def __init__(self, nome):
        self.nome = nome

classe_a = [Studente("Anna"), Studente("Luca")]
classe_b = classe_a[:]          # copia della lista

print("le due liste sono lo stesso oggetto?", classe_a is classe_b)
print("il primo studente e' lo stesso?", classe_a[0] is classe_b[0])

classe_b[0].nome = "Anna Neri"
print("nome letto dalla lista A:", classe_a[0].nome)

classe_b.append(Studente("Sara"))
print("quanti in A:", len(classe_a), "- quanti in B:", len(classe_b))

```

Che cosa compare quando lo esegui

```

le due liste sono lo stesso oggetto? False
il primo studente e' lo stesso? True
nome letto dalla lista A: Anna Neri
quanti in A: 2 - quanti in B: 3

```

Perché: succedono due cose opposte nella stessa prova, ed è qui che ci si confonde. Le *liste* sono davvero separate: aggiungendo Sara alla seconda, la prima resta di due. Ma gli *studenti dentro* sono gli stessi: cambiando il nome dalla lista B, cambia anche nella lista A. La copia con `[:]` duplica il contenitore, non il contenuto.

Prova a cambiare: immagina la lista B come una seconda scatola in cui hai messo *gli stessi* studenti, non delle fotocopie. Nel paragrafo 3.7 vedrai come fare le fotocopie vere.

3.5 Oggetti dentro altri oggetti: la composizione

Un campo di un oggetto può contenere un altro oggetto. Questa è la **composizione**, ed è il modo in cui si costruiscono i programmi veri: un `Corso` contiene una lista di `Studente`, un `Ordine` contiene un `Cliente` e una lista di `RigaOrdine`, un `Sensore` contiene una lista di `Misurazione`.

```

class Corso:
    def __init__(self, titolo, ore):
        self.titolo = titolo
        self.ore = ore
        self.iscritti = []          # conterrà oggetti Studente

    def iscrivi(self, studente):
        if studente not in self.iscritti:
            self.iscritti.append(studente)

    def numero_iscritti(self):
        return len(self.iscritti)

```

```

def media_classe(self):
    if not self.iscritti:
        return 0
    return sum(s.media() for s in self.iscritti) / len(self.iscritti)

def promossi(self):
    return [s for s in self.iscritti if s.promosso()]

def __str__(self):
    return f"{self.titolo} ({self.ore}h) - {self.numero_iscritti()} iscritti"

```

```

corso = Corso("Junior Data Engineer", 200)
anna = Studente("Anna Rossi", "anna@example.com")
luca = Studente("Luca Bianchi", "luca@example.com")

corso.iscrivi(anna)
corso.iscrivi(luca)
anna.aggiungi_voto(30)
luca.aggiungi_voto(20)

print(corso)                # Junior Data Engineer (200h) - 2 iscritti
print(corso.media_classe()) # 25.0
print([s.nome for s in corso.promossi()]) # ['Anna Rossi']

```

Osserva la catena: `corso.iscritti` è una lista, ogni elemento è uno `Studente`, ogni studente ha i suoi `voti`. Puoi scendere lungo la catena con i punti: `corso.iscritti[0].media()`. E soprattutto: siccome `anna` e `corso.iscritti[0]` sono lo stesso oggetto, aggiungere un voto ad `anna` cambia *immediatamente* la media del corso. Non c'è nessuna sincronizzazione da fare, perché non c'è nessuna duplicazione.

NOTA

Il metodo `iscrivi()` usa `studente not in self.iscritti`. L'operatore `in` su una lista di oggetti usa `==`, quindi il suo comportamento dipende dal fatto che tu abbia definito o meno `__eq__`. Senza `__eq__` impedisce solo di iscrivere *lo stesso oggetto* due volte; con `__eq__` basato sull'email, impedisce anche di iscrivere due oggetti diversi con la stessa email. Un dettaglio piccolo con conseguenze grosse.

PROVA TU · 24 Un oggetto dentro un altro oggetto

Costruiamo un corso che contiene studenti, e vediamo un effetto molto comodo.

```

class Studente:
    def __init__(self, nome):
        self.nome = nome
        self.voti = []

    def aggiungi_voto(self, voto):
        self.voti.append(voto)

```

```

def media(self):
    if len(self.voti) == 0:
        return 0
    return sum(self.voti) / len(self.voti)

class Corso:
    def __init__(self, titolo):
        self.titolo = titolo
        self.iscritti = []

    def iscrivi(self, studente):
        self.iscritti.append(studente)

    def media_classe(self):
        if len(self.iscritti) == 0:
            return 0
        return sum(s.media() for s in self.iscritti) / len(self.iscritti)

corso = Corso("Junior Data Engineer")
anna = Studente("Anna")
corso.iscrivi(anna)

print("media del corso all'inizio:", corso.media_classe())

anna.aggiungi_voto(30)          # modifichiamo ANNA, non il corso
print("media del corso adesso:", corso.media_classe())
print("lo studente nel corso e' anna?", corso.iscritti[0] is anna)

```

Che cosa compare quando lo esegui

```

media del corso all'inizio: 0.0
media del corso adesso: 30.0
lo studente nel corso e' anna? True

```

Perché: non hai toccato il corso, eppure la sua media è cambiata. Il motivo è nell'ultima riga: dentro `corso.iscritti` non c'è una copia di Anna, c'è **proprio Anna**. Non esistono due versioni da tenere allineate, quindi non possono contraddirsi. Mettere oggetti dentro altri oggetti si chiama *composizione* ed è il modo normale di costruire i programmi veri.

Prova a cambiare: aggiungi un secondo studente con un voto diverso e ristampa la media del corso: il calcolo continua a funzionare senza modifiche.

3.6 Passare oggetti alle funzioni

Quando passi un oggetto a una funzione, non ne passi una copia: passi il riferimento. La funzione lavora **sull'oggetto originale**.

```
def bonus(studente):
    studente.aggiungi_voto(30)      # modifica l'oggetto vero

anna = Studente("Anna", "a@x.it")
bonus(anna)
print(anna.voti)                  # [30] -- modificato dall'esterno
```

Attenzione però alla differenza fra *modificare* l'oggetto e *riassegnare* il parametro:

```
def sostituisci(studente):
    studente = Studente("Nuovo", "n@x.it")  # sposta solo l'etichetta locale
    studente.aggiungi_voto(18)

anna = Studente("Anna", "a@x.it")
sostituisci(anna)
print(anna.nome)      # Anna -- fuori non è cambiato nulla
```

Dentro la funzione, `studente` è una nuova etichetta che all'inizio punta allo stesso oggetto di `anna`. Riassegnarla stacca l'etichetta locale e la appiccica altrove, ma `anna` resta dov'era. È la stessa regola del riquadro del paragrafo 3.2, applicata ai parametri.

ATTENZIONE

Una funzione che modifica gli oggetti che riceve va documentata con chiarezza. Chi legge `bonus(anna)` non ha modo di sapere che l'oggetto verrà cambiato, a meno che il nome del metodo o la docstring non lo dicano. È uno dei motivi per cui, nel capitolo 4, preferiremo far modificare gli oggetti *dai loro stessi metodi*.

PROVA TU · 25 Modificare un oggetto oppure sostituirlo

Due funzioni che si assomigliano molto ma hanno effetti opposti su chi le chiama.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome
        self.voti = []

def aggiungi_bonus(studente):
    studente.voti.append(30)      # entra nell'oggetto e lo modifica

def sostituisci(studente):
    studente = Studente("Nuovo")  # sposta solo l'etichetta locale
    studente.voti.append(18)

anna = Studente("Anna")

aggiungi_bonus(anna)
```

```
print("dopo aggiungi_bonus:", anna.nome, anna.voti)

sostituisci(anna)
print("dopo sostituisci :", anna.nome, anna.voti)
```

Che cosa compare quando lo esegui

```
dopo aggiungi_bonus: Anna [30]
dopo sostituisci   : Anna [30]
```

Perché: quando passi un oggetto a una funzione, la funzione riceve l'oggetto vero, non una copia: `aggiungi_bonus` lo modifica per davvero. Ma `sostituisci` non lascia traccia, perché riassegnare il parametro sposta soltanto l'etichetta che esiste dentro la funzione. È la stessa regola della prova 20.

Prova a cambiare: una funzione che modifica gli oggetti che riceve dovrebbe dirlo nel nome o in una frase di documentazione: chi legge `aggiungi_bonus(anna)` non ha modo di indovinare che Anna verrà cambiata.

3.7 Copiare davvero un oggetto

Quando serve una copia indipendente, il modulo `copy` offre due livelli.

```
import copy

anna = Studente("Anna", "a@x.it")
anna.aggiungi_voto(28)

superficiale = copy.copy(anna)           # shallow copy
profonda     = copy.deepcopy(anna)      # deep copy
```

La copia superficiale crea un nuovo oggetto `Studente`, ma i suoi campi puntano *agli stessi* oggetti dell'originale. Siccome `voti` è una lista (quindi un oggetto mutabile), la lista resta condivisa:

```
superficiale.nome = "Copia"
print(anna.nome)           # Anna -- il campo nome è indipendente

superficiale.aggiungi_voto(18)
print(anna.voti)           # [28, 18] -- ma la LISTA è condivisa!
```

La copia profonda ricrea ricorsivamente anche gli oggetti contenuti:

```
profonda.aggiungi_voto(21)
print(anna.voti)           # [28, 18] -- invariato: liste separate
```

Operazione	L'oggetto esterno	Gli oggetti contenuti
<code>b = a</code>	Condiviso (alias)	Condivisi
<code>copy.copy(a)</code>	Nuovo	Condivisi
<code>copy.deepcopy(a)</code>	Nuovo	Nuovi

Nella pratica quotidiana userai `deepcopy` raramente: è lenta su strutture grandi e spesso il vero problema è di progettazione, non di copia. La cosa importante è **sapere che il problema esiste**, così quando un oggetto cambia "da solo" saprai dove guardare.

PROVA TU · 26 Le tre profondità di copia, verificate

Mettiamo alla prova la tabella qui sopra invece di fidarci: alias, copia superficiale e copia profonda a confronto.

```
import copy

class Studente:
    def __init__(self, nome):
        self.nome = nome
        self.voti = []

anna = Studente("Anna")
anna.voti.append(28)

alias = anna
superficiale = copy.copy(anna)
profonda = copy.deepcopy(anna)

print("alias e' lo stesso studente?", alias is anna)
print("la copia superficiale lo e'?", superficiale is anna)
print("la sua lista voti e' la stessa?", superficiale.voti is anna.voti)
print("quella della copia profonda?", profonda.voti is anna.voti)

superficiale.voti.append(18)
print("voti di anna dopo aver toccato la copia superficiale:", anna.voti)

profonda.voti.append(21)
print("voti di anna dopo aver toccato la copia profonda:", anna.voti)
```

Che cosa compare quando lo esegui

```
alias e' lo stesso studente? True
la copia superficiale lo e'? False
la sua lista voti e' la stessa? True
quella della copia profonda? False
voti di anna dopo aver toccato la copia superficiale: [28, 18]
voti di anna dopo aver toccato la copia profonda: [28, 18]
```

Perché: la terza riga è il punto della prova. `copy.copy()` ha creato un nuovo studente, ma la sua lista di voti è ancora *quella di Anna*: aggiungendo un voto alla copia superficiale, il voto compare anche in Anna. Solo `copy.deepcopy()` duplica anche le liste contenute, e infatti l'ultimo 21 non tocca Anna.

Prova a cambiare: nella pratica userai `deepcopy` di rado. L'importante è sapere che questo comportamento esiste: quando un oggetto sembra cambiare da solo, adesso sai dove guardare.

Esercizio 6 *Prevedi l'output*

●○○

Senza eseguire il codice, scrivi che cosa stampa ciascuna riga. Poi esegui e confronta.

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Punto(1, 2)
p2 = Punto(1, 2)
p3 = p1

p3.x = 99

print(p1.x)          # (a)
print(p2.x)          # (b)
print(p1 is p2)       # (c)
print(p1 is p3)       # (d)
print(p1 == p2)       # (e)

lista = [p1, p2]
lista[0].y = 77
print(p1.y)           # (f)
```

Per ogni risposta, spiega in una frase *perché*.

Esercizio 7 *La trappola dell'alias*

●●○

Il codice seguente dovrebbe creare tre squadre *indipendenti*, ma non funziona: aggiungendo un giocatore a una squadra, compare in tutte.

```
class Squadra:
    def __init__(self, nome, giocatori):
        self.nome = nome
        self.giocatori = giocatori

comune = []
rossi = Squadra("Rossi", comune)
blu = Squadra("Blu", comune)
verdi = Squadra("Verdi", comune)

rossi.giocatori.append("Marco")
print(blu.giocatori)      # ['Marco'] -- errore!
```

1. Spiega esattamente perché succede, usando la parola "riferimento".
2. Correggi la classe in **due modi diversi**: (a) rendendo il parametro opzionale con default `None` e creando la lista dentro `__init__`; (b) mantenendo il parametro obbligatorio ma copiando la lista ricevuta.
3. Perché `def __init__(self, nome, giocatori=[])` **non** è una correzione valida? Provalo e osserva.

Il punto 3 è uno dei tranelli più famosi di Python: la risposta è in Appendice A.

Esercizio 8 *Composizione: Biblioteca e Libro*

●●○

Scrivi due classi che collaborano.

- **Libro**: campi `titolo`, `autore`, `isbn`, `disponibile` (default `True`); metodi `presta()` e `restituisce()` che cambiano `disponibile` e restituiscono `True / False` a seconda che l'operazione sia riuscita; `__str__`.
- **Biblioteca**: campi `nome` e `libri` (lista di oggetti `Libro`); metodi `aggiungi(libro)`, `cerca_per_autore-autore)` che restituisce la lista dei libri di quell'autore, `disponibili()`, e `presta_isbn(isbn)` che trova il libro e lo presta.

Verifica infine che, dopo `biblioteca.prestare_isbn("123")`, anche la variabile originale del libro risulti non disponibile. Spiega perché.

Esercizio 9 *Copia superficiale e profonda*

●●●

Usando le classi `Corso` e `Studente` del paragrafo 3.5:

1. Crea un corso con due studenti iscritti.
2. Fanne una copia con `copy.copy()` e una con `copy.deepcopy()`.
3. Aggiungi un voto al primo studente *della copia superficiale*. Controlla la media dell'originale: è cambiata? Perché?
4. Ripeti con la copia profonda. Che differenza osservi?

5. Iscriviti un nuovo studente alla copia superficiale: il corso originale ha ora due o tre iscritti?
Attenzione, la risposta non è quella dei punti precedenti — spiega la differenza.

Usa `id()` per dimostrare ogni tua affermazione.

RIEPILOGO DEL CAPITOLO

- Una variabile è **un'etichetta appiccicata a un oggetto**, non una scatola che lo contiene.
- `b = a` crea un **alias**: due nomi, un solo oggetto. Modificarlo da uno lo modifica per entrambi.
- `is` confronta l'identità, `==` confronta il valore; per decidere quando due tuoi oggetti sono uguali devi definire `__eq__`.
- Liste e funzioni ricevono **riferimenti**: gli oggetti che contengono o ricevono sono gli stessi originali.
- La **composizione** (oggetti dentro oggetti) è il modo normale di costruire programmi reali.
- `copy.copy()` duplica solo il contenitore, `copy.deepcopy()` duplica tutta la struttura.

Incapsulamento

IN QUESTO CAPITOLO

- Capirai perché lasciare i campi liberamente accessibili è un problema, e come si mette ordine.
- Userai le *property* per validare i dati senza cambiare la sintassi di chi usa la classe.
- Imparerai a progettare l'**interfaccia pubblica** di un componente *prima* di scriverlo.
- Scriverai i primi test automatici, con `assert` e poi con `pytest`.

4.1 Il problema: nessuno controlla i dati

La classe `Studente` del capitolo 2 ha ancora un difetto grosso: chiunque, da qualsiasi punto del programma, può scrivere qualunque cosa nei suoi campi.

```
anna = Studente("Anna Rossi", "anna@example.com")

anna.voti.append(45)           # voto impossibile in trentesimi
anna.voti = "trenta"          # La lista è diventata una stringa
anna.email = None              # email cancellata

print(anna.media())            # TypeError, a chilometri dal vero colpevole
```

Il danno vero non è l'errore: è *dove* esplode. L'errore compare dentro `media()`, ma la riga colpevole è `anna.voti = "trenta"`, magari scritta in un altro file da un altro sviluppatore tre settimane prima. Quando lo stato di un oggetto può essere corrotto da chiunque, ogni bug diventa una caccia al tesoro.

L'**incapsulamento** è la risposta a questo problema, e si riassume in due mosse:

1. **Nascondi** lo stato interno, in modo che non sia manipolabile direttamente dall'esterno.
2. **Esponi** un insieme ristretto e controllato di operazioni — l'interfaccia pubblica — che sono l'unico modo di interagire con l'oggetto.

Il risultato è che tutti i controlli stanno in un punto solo, e l'oggetto diventa responsabile della propria correttezza. Si dice che l'oggetto mantiene i propri *invarianti*: proprietà che devono essere sempre vere, come «i voti stanno fra 0 e 30» o «il saldo non è mai negativo».

PROVA TU · 27 Una classe che non si difende

Prendiamo una classe scritta come nel capitolo 2, senza protezioni, e maltrattiamola.

```
class Studente:
    def __init__(self, nome):
```

```

        self.nome = nome
        self.voti = []

    def media(self):
        return sum(self.voti) / len(self.voti)

anna = Studente("Anna")
anna.voti.append(28)
anna.voti.append(45)                # voto impossibile: nessuno lo impedisce

print("media con il voto 45:", anna.media())

anna.voti = "trenta"                # la lista diventa testo
print("media adesso:", anna.media())

```

Che cosa compare quando lo esegui (e' voluto!)

```

media con il voto 45: 36.5
TypeError: unsupported operand type(s) for +: 'int' and 'str'

```

Perché: chiunque, da qualunque punto del programma, può mettere questo studente in uno stato impossibile. Il voto 45 viene accettato in silenzio e falsa la media; la lista sostituita da un testo fa esplodere il programma *dentro* `media()`, lontano dalla riga che ha causato il guaio.

Prova a cambiare: tieni da parte questo esempio: alla fine del capitolo riscriverai la stessa classe in modo che entrambe le righe sbagliate vengano rifiutate **subito**, con un messaggio chiaro.

4.2 Convenzioni di visibilità: `_` e `__`

Molti linguaggi hanno parole chiave come `private` e `public`. Python no: usa delle **convenzioni sui nomi**, e questa è una scelta di filosofia, non una mancanza.

Scrittura	Nome	Significato
<code>voti</code>	Pubblico	Fa parte dell'interfaccia. Chiunque può usarlo, e tu ti impegni a non cambiarlo senza preavviso.
<code>_voti</code>	Protetto (<i>per convenzione</i>)	«Dettaglio interno, non toccare.» Python non lo impedisce, ma ogni programmatore capisce il messaggio.
<code>__voti</code>	Privato (<i>name mangling</i>)	Python rinomina l'attributo internamente, rendendolo scomodo da raggiungere per sbaglio dall'esterno.

```

class ContoBancario:
    def __init__(self, intestatario, saldo_iniziale=0):
        self.intestatario = intestatario    # pubblico
        self._saldo = saldo_iniziale        # interno: non toccare
        self.__pin = "0000"                 # privato: name mangling

```

```
c = ContoBancario("Anna", 100)
print(c._saldo)      # 100 -- funziona, ma stai violando il contratto
print(c.__pin)       # AttributeError: 'ContoBancario' object has no attribute '__pin'
print(c._ContoBancario__pin) # "0000" -- il vero nome dopo il mangling
```

NOTA

Il doppio underscore non è una vera protezione: è una *ridenominazione* automatica (`__pin` diventa `_ContoBancario__pin`). Serve soprattutto a evitare collisioni accidentali di nomi nelle gerarchie di ereditarietà. In pratica, nel codice Python professionale si usa quasi sempre **un solo underscore**: la filosofia del linguaggio è «*siamo tutti adulti consenzienti*» — segnale cosa è interno, e mi fido che tu non lo tocchi.

PROVA TU · 28 Un trattino basso, due trattini bassi

Vediamo la differenza pratica fra le due convenzioni che Python usa per dire «questo è roba mia, non toccare».

```
class Conto:
    def __init__(self):
        self.intestatario = "Anna"      # pubblico
        self._saldo = 100               # interno per convenzione
        self.__pin = "0000"            # con due trattini bassi

c = Conto()
print(c.intestatario)
print(c._saldo)
print([n for n in dir(c) if "pin" in n])
print(c._Conto__pin)
print(c.__pin)
```

Che cosa compare quando lo esegui (e' voluto!)

```
Anna
100
['_Conto__pin']
0000
AttributeError: 'Conto' object has no attribute '__pin'
```

Perché: il campo con un solo trattino è perfettamente leggibile: Python non te lo impedisce, è un accordo fra programmatori. Quello con due trattini invece viene **rinominato** automaticamente in `_Conto__pin`, come mostra la terza riga: per questo `c.__pin` non esiste. Non è una cassaforte, è solo un nome cambiato.

Prova a cambiare: nella pratica, nel codice Python professionale si usa quasi sempre **un solo trattino basso**. Significa: «questo è un dettaglio interno, se lo tocchi e domani lo cambio, il problema è tuo».

4.3 Le property: controllare senza cambiare la sintassi

Nascondere un campo dietro un underscore risolve metà del problema: sappiamo che non va toccato, ma ora come lo leggiamo e come lo modifichiamo in modo controllato? La soluzione ingenua sono i metodi `get_saldo()` e `set_saldo()`, come si fa in Java. In Python esiste qualcosa di più elegante: le **property**.

```
class ContoBancario:
    def __init__(self, intestatario, saldo_iniziale=0):
        self.intestatario = intestatario
        self._saldo = 0
        if saldo_iniziale > 0:
            self.deposita(saldo_iniziale)

    @property
    def saldo(self):
        """Il saldo corrente (sola lettura dall'esterno)."""
        return self._saldo

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da depositare deve essere positivo")
        self._saldo += importo
        return self._saldo

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere positivo")
        if importo > self._saldo:
            raise ValueError(f"Saldo insufficiente: disponibili {self._saldo}")
        self._saldo -= importo
        return self._saldo
```

```
c = ContoBancario("Anna Rossi", 100)
print(c.saldo)           # 100 -- si legge come un campo, ma è un metodo
c.deposita(50)
print(c.saldo)           # 150
c.saldo = 1_000_000      # AttributeError: property 'saldo' of 'ContoBancario' object has no
                        # setter
```

Il decoratore `@property` trasforma un metodo in qualcosa che si *legge* come un campo: scrivi `c.saldo`, senza parentesi. Il vantaggio è enorme: puoi partire da un campo pubblico normale e trasformarlo in property più tardi, **senza che nessuno debba cambiare una riga del codice che usa la tua classe**.

Se serve anche poter scrivere il valore, ma con controlli, si aggiunge un *setter*:

```
class Misurazione:
    def __init__(self, sensore_id, valore):
        self.sensore_id = sensore_id
        self.valore = valore          # passa già dal setter qui sotto!
```

```

@property
def valore(self):
    return self._valore

@valore.setter
def valore(self, nuovo):
    if not isinstance(nuovo, (int, float)):
        raise TypeError("Il valore deve essere numerico")
    if not -50 <= nuovo <= 150:
        raise ValueError(f"Valore fuori scala: {nuovo}")
    self._valore = float(nuovo)

```

```

m = Misurazione("T-01", 21.5)
m.valore = 22.0          # ok, passa dal setter
m.valore = 999          # ValueError: Valore fuori scala: 999
m.valore = "caldo"      # TypeError: IL valore deve essere numerico

```

Nota il dettaglio elegante: nel costruttore scriviamo `self.valore = valore` (senza underscore), quindi **anche la creazione dell'oggetto passa per il setter** e viene validata. Una sola regola scritta in un solo posto, valida ovunque.

ATTENZIONE

Non trasformare ogni campo in property per abitudine. Se un campo non ha regole da far rispettare, lascialo pubblico e semplice: in Python aggiungere getter e setter vuoti è considerato rumore. La property serve quando c'è **una regola da difendere** o un valore da calcolare.

PROVA TU · 29 Un conto che si difende da solo

Ecco la stessa idea del conto bancario, ma scritta in modo che sia impossibile rovinarlo dall'esterno. (Il blocco `try / except` serve solo a mostrare il messaggio di errore senza fermare il programma.)

```

class ContoBancario:
    def __init__(self, intestatario):
        self.intestatario = intestatario
        self._saldo = 0

    @property
    def saldo(self):
        return self._saldo

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("l'importo deve essere positivo")
        self._saldo += importo

    def preleva(self, importo):
        if importo > self._saldo:
            raise ValueError("saldo insufficiente")
        self._saldo -= importo

```

```

c = ContoBancario("Anna")
c.deposita(100)
print("saldo:", c.saldo)

c.preleva(30)
print("saldo dopo il prelievo:", c.saldo)

try:
    c.deposita(-50)
except ValueError as errore:
    print("rifiutato:", errore)

try:
    c.preleva(1000)
except ValueError as errore:
    print("rifiutato:", errore)

print("saldo alla fine:", c.saldo)

c.saldo = 999999

```

Che cosa compare quando lo esegui (e' voluto!)

```

saldo: 100
saldo dopo il prelievo: 70
rifiutato: l'importo deve essere positivo
rifiutato: saldo insufficiente
saldo alla fine: 70
AttributeError: property 'saldo' of 'ContoBancario' object has no setter

```

Perché: il saldo si legge come se fosse un dato normale (`c.saldo` , senza parentesi) grazie a `@property` , ma **non si può scrivere:** l'ultima riga fallisce. Le operazioni sbagliate vengono rifiutate con un messaggio che spiega il motivo, e nota la penultima riga: dopo i due tentativi falliti il saldo è ancora quello giusto.

Prova a cambiare: la parola `raise` significa «solleva un errore e fermati qui». È il modo con cui un oggetto dice a chi lo usa: «questa cosa non te la lascio fare».

PROVA TU · 30 Un controllo che vale anche alla creazione

Con `@property` si può anche permettere la scrittura, ma controllando il valore prima di accettarlo.

```

class Misurazione:
    def __init__(self, sensore, valore):
        self.sensore = sensore
        self.valore = valore          # attenzione: passa dal controllo qui sotto

    @property
    def valore(self):
        return self._valore

```

```

@valore.setter
def valore(self, nuovo):
    if not -50 <= nuovo <= 150:
        raise ValueError("valore fuori scala: " + str(nuovo))
    self._valore = float(nuovo)

m = Misurazione("T-01", 21.5)
print("valore iniziale:", m.valore)

m.valore = 22.0
print("valore aggiornato:", m.valore)

try:
    m.valore = 999
except ValueError as errore:
    print("rifiutato:", errore)

Misurazione("T-02", 999)

```

Che cosa compare quando lo esegui (e' voluto!)

```

valore iniziale: 21.5
valore aggiornato: 22.0
rifiutato: valore fuori scala: 999
ValueError: valore fuori scala: 999

```

Perché: l'ultima riga è la più importante: il controllo scatta **anche quando crei l'oggetto**. Il motivo è che nel costruttore è scritto `self.valore = valore` (senza trattino basso), e quella riga passa dal controllo che hai scritto sotto.

Prova a cambiare: prova a cambiare il costruttore in `self._valore = valore` (con il trattino) e riesegui: l'ultima riga viene accettata e crea una misurazione da 999. Hai scavalcato il tuo stesso controllo. È un errore facile da fare e difficile da notare.

Property calcolate

Il secondo uso frequente è esporre un valore *derivato*, che non è memorizzato ma calcolato al volo:

```

class Studente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self._voti = []

    @property
    def media(self):
        if not self._voti:
            return 0.0
        return sum(self._voti) / len(self._voti)

    @property

```

```
def promosso(self):
    return self.media >= 24
```

```
print(anna.media)      # 29.0 -- niente parentesi: sembra un dato
print(anna.promosso)   # True
```

Il vantaggio rispetto a un campo `self.media` aggiornato a mano è che **non può mai essere disallineato**: viene ricalcolato ogni volta che lo leggi. La regola pratica: usa una property quando il calcolo è veloce e senza effetti collaterali; usa un metodo normale con le parentesi quando l'operazione è costosa, o quando *fa* qualcosa invece di limitarsi a rispondere.

PROVA TU · 31 Un dato calcolato non può essere sbagliato

Confrontiamo due modi di offrire la media: memorizzarla in un campo, oppure calcolarla ogni volta.

```
class StudenteFragile:
    def __init__(self):
        self.voti = []
        self.media = 0                # media memorizzata in un campo

    def aggiungi_voto(self, voto):
        self.voti.append(voto)        # ci si dimentica di aggiornare media

class StudenteSolido:
    def __init__(self):
        self._voti = []

    def aggiungi_voto(self, voto):
        self._voti.append(voto)

    @property
    def media(self):                  # media calcolata ogni volta
        if len(self._voti) == 0:
            return 0.0
        return sum(self._voti) / len(self._voti)

a = StudenteFragile()
a.aggiungi_voto(30)
print("fragile -> voti:", a.voti, "media:", a.media)

b = StudenteSolido()
b.aggiungi_voto(30)
print("solido -> media:", b.media)
```

Che cosa compare quando lo esegui

```
fragile -> voti: [30] media: 0
solido -> media: 30.0
```

Perché: nella prima classe la media è un dato che qualcuno deve ricordarsi di aggiornare a ogni voto, e prima o poi nessuno lo farà: infatti mostra o pur avendo un 30. Nella seconda non può essere sbagliata, perché viene ricalcolata nel momento in cui la leggi. Da fuori la scrittura è identica in entrambi i casi: `a.media`.

Prova a cambiare: usa una `@property` quando il valore è veloce da calcolare e si limita a rispondere. Usa un metodo normale, con le parentesi, quando l'operazione è lenta o quando *fa* qualcosa invece di limitarsi a dire qualcosa.

4.4 L'interfaccia pubblica come contratto

L'**interfaccia pubblica** di una classe è l'insieme di tutto ciò che gli altri possono usare: i metodi e gli attributi senza underscore. È un *contratto* in due direzioni.

- **Tu prometti** che quei nomi continueranno a esistere e a comportarsi allo stesso modo, anche quando riscriverai la classe da capo.
- **Chi la usa promette** di limitarsi a quei nomi e di non andare a frugare in `_saldo`.

Il valore di questo patto si vede quando cambi implementazione. Immagina di dover riscrivere `ContoBancario` per tenere lo storico dei movimenti invece di un semplice numero:

```
class ContoBancario:
    def __init__(self, intestatario, saldo_iniziale=0):
        self.intestatario = intestatario
        self._movimenti = []                # tutto cambiato dentro
        if saldo_iniziale > 0:
            self.deposita(saldo_iniziale)

    @property
    def saldo(self):
        return sum(self._movimenti)        # ora è calcolato

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da depositare deve essere positivo")
        self._movimenti.append(importo)
        return self.saldo

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere positivo")
        if importo > self.saldo:
            raise ValueError(f"Saldo insufficiente: disponibili {self.saldo}")
        self._movimenti.append(-importo)
        return self.saldo

    def estratto_conto(self):
        return list(self._movimenti)       # nuova funzionalità
```

L'interno è completamente diverso: non esiste più il campo `_saldo`. Eppure **tutto il codice scritto da altri continua a funzionare**, perché `c.saldo`, `c.deposita()` e `c.preleva()` si comportano esattamente come prima. Questo è il ritorno dell'investimento fatto nascondendo lo stato: la libertà di cambiare idea.

NOTA

Osserva anche `estratto_conto()`: restituisce `list(self._movimenti)`, cioè una *copia*, non la lista interna. Se restituisse `self._movimenti`, chiunque potrebbe fare `c.estratto_conto().append(1000000)` e alterare il saldo dall'esterno, aggirando tutti i controlli. È il capitolo 3 che torna: restituire un riferimento interno significa regalare le chiavi di casa.

PROVA TU · 32 Cambiare tutto dentro senza rompere niente fuori

Ecco perché nascondere i dati conviene. Due classi completamente diverse al loro interno, usate dallo stesso identico codice.

```
class ContoVersione1:
    def __init__(self, intestatario):
        self.intestatario = intestatario
        self._saldo = 0                # tiene un numero

    @property
    def saldo(self):
        return self._saldo

    def deposita(self, importo):
        self._saldo += importo

    def preleva(self, importo):
        self._saldo -= importo

class ContoVersione2:
    def __init__(self, intestatario):
        self.intestatario = intestatario
        self._movimenti = []          # tiene la lista dei movimenti

    @property
    def saldo(self):
        return sum(self._movimenti)

    def deposita(self, importo):
        self._movimenti.append(importo)

    def preleva(self, importo):
        self._movimenti.append(-importo)

    def estratto_conto(self):
        return list(self._movimenti)
```

```
def usa_il_conto(c):                                # questo codice non cambia mai
    c.deposita(100)
    c.preleva(30)
    print(c.intestatario, "ha un saldo di", c.saldo)

usa_il_conto(ContoVersione1("Anna"))
usa_il_conto(ContoVersione2("Luca"))
```

Che cosa compare quando lo esegui

```
Anna ha un saldo di 70
Luca ha un saldo di 70
```

Perché: le due classi non hanno nulla in comune al loro interno: nella prima il saldo è un numero, nella seconda non esiste proprio e viene calcolato sommando i movimenti. Eppure la funzione `usa_il_conto` funziona con entrambe, perché entrambe offrono gli stessi nomi verso l'esterno: `saldo`, `deposita`, `preleva`.

Prova a cambiare: questo è il guadagno vero dell'incapsulamento: hai la **libertà di cambiare idea** su come funziona una classe, senza costringere nessun altro a riscrivere il proprio codice. Prova ad aggiungere `print(ContoVersione2("Sara").estratto_conto())`: aggiungere cose nuove non rompe niente.

4.5 Progettare il componente prima di scriverlo

Il programma del corso parla di «*progettazione e testing del componente*», e l'ordine delle parole non è casuale. Il modo professionale di costruire una classe è partire dall'esterno: prima decidi **cosa deve saper fare**, poi come lo fa.

Il metodo in quattro domande, da farsi *prima* di scrivere codice:

1. **Di cosa è responsabile questa classe?** Rispondi in una frase sola. Se ti servono due frasi con una "e" in mezzo, probabilmente sono due classi.
2. **Quali operazioni deve offrire?** Elencale con nomi di verbi, dal punto di vista di chi la userà.
3. **Quali dati deve custodire?** Questi diventeranno campi interni, con l'underscore.
4. **Quali regole devono valere sempre?** Questi sono gli invarianti, e determinano dove metterai i controlli.

Applichiamolo a un componente che ci servirà davvero: un registro di misurazioni.

```
# 1. RESPONSABILITÀ
#   Custodire le misurazioni di un sensore e rispondere a domande su di esse.
#
# 2. OPERAZIONI (interfaccia pubblica)
#   registra(valore)      -> aggiunge una misurazione validata
#   ultima                -> l'ultima misurazione, None se vuoto [property]
#   media                 -> media delle misurazioni, 0.0 se vuoto [property]
```

```

#    massimo / minimo          -> estremi, None se vuoto          [property]
#    in_allarme                -> True se l'ultima supera la soglia [property]
#    storico()                  -> copia della lista delle misurazioni
#
# 3. DATI INTERNI
#    _valori (lista di float), _soglia (float), sensore_id (str, pubblico)
#
# 4. INVARIANTI
#    - _valori contiene solo numeri
#    - ogni valore sta fra -50 e 150
#    - _valori non è mai manipolabile dall'esterno

```

Solo adesso si scrive il codice, e diventa quasi una trascrizione:

```

class RegistroMisurazioni:
    VALORE_MIN = -50
    VALORE_MAX = 150

    def __init__(self, sensore_id, soglia_allarme=30.0):
        self.sensore_id = sensore_id
        self._soglia = soglia_allarme
        self._valori = []

    def registra(self, valore):
        if not isinstance(valore, (int, float)):
            raise TypeError(f"Valore non numerico: {valore!r}")
        if not self.VALORE_MIN <= valore <= self.VALORE_MAX:
            raise ValueError(f"Valore fuori scala: {valore}")
        self._valori.append(float(valore))

    @property
    def ultima(self):
        return self._valori[-1] if self._valori else None

    @property
    def media(self):
        return sum(self._valori) / len(self._valori) if self._valori else 0.0

    @property
    def massimo(self):
        return max(self._valori) if self._valori else None

    @property
    def minimo(self):
        return min(self._valori) if self._valori else None

    @property
    def in_allarme(self):
        return self.ultima is not None and self.ultima > self._soglia

    def storico(self):
        return list(self._valori)

    def __len__(self):
        return len(self._valori)

```

```
def __str__(self):
    stato = "ALLARME" if self.in_allarme else "ok"
    return f"{self.sensore_id}: {len(self)} letture, media {self.media:.1f} [{stato}]"
```

Nota `__len__`: è un altro metodo speciale, e ti permette di scrivere `len(registro)` come se fosse una lista. Definire i metodi speciali giusti fa sì che i tuoi oggetti si comportino come quelli nativi di Python.

PROVA TU · 33 La falla che non dà nessun errore

Un errore molto sottile: restituire la lista interna invece di una copia. Vediamo le due versioni a confronto.

```
class ContoBucato:
    def __init__(self):
        self._movimenti = [100]

    @property
    def saldo(self):
        return sum(self._movimenti)

    def estratto_conto(self):
        return self._movimenti          # restituisce la lista VERA

class ContoSicuro:
    def __init__(self):
        self._movimenti = [100]

    @property
    def saldo(self):
        return sum(self._movimenti)

    def estratto_conto(self):
        return list(self._movimenti)    # restituisce una COPIA

a = ContoBucato()
a.estratto_conto().append(1000000)
print("conto bucato -> saldo:", a.saldo)

b = ContoSicuro()
b.estratto_conto().append(1000000)
print("conto sicuro -> saldo:", b.saldo)
```

Che cosa compare quando lo esegui

```
conto bucato -> saldo: 1000100
conto sicuro -> saldo: 100
```

Perché: nel primo conto siamo riusciti a modificare il saldo dall'esterno senza toccare nessun campo privato e senza nessun errore: bastava chiedere l'estratto conto e aggiungerci dentro un milione.

Restituire una struttura interna equivale a consegnarne le chiavi.

Prova a cambiare: la differenza fra le due classi è una sola parola: `list(...)`, che crea una copia. Sono cinque caratteri che separano un componente sicuro da uno bucato.

4.6 Testare il componente dalla sua interfaccia

Adesso arriva la parte che chiude il cerchio. Se hai progettato bene l'interfaccia, **testare il componente significa verificare le promesse che l'interfaccia fa** — e nient'altro. I test non devono mai andare a guardare `_valori`: se lo facessero, si romperebbero al primo cambio di implementazione, esattamente come il codice che viola l'incapsulamento.

Il modo più semplice per iniziare è `assert`, che solleva un errore se la condizione è falsa:

```
r = RegistroMisurazioni("T-01", soglia_allarme=30)

# stato iniziale
assert len(r) == 0
assert r.ultima is None
assert r.media == 0.0
assert r.in_allarme is False

# dopo una registrazione
r.registra(21.5)
assert len(r) == 1
assert r.ultima == 21.5
assert r.media == 21.5

# media su più valori
r.registra(22.5)
assert r.media == 22.0
assert r.massimo == 22.5
assert r.minimo == 21.5

# allarme
r.registra(35.0)
assert r.in_allarme is True

# i valori non validi vengono rifiutati
try:
    r.registra(999)
    assert False, "avrebbe dovuto sollevare ValueError"
except ValueError:
    pass

# Lo storico restituito è una copia
copia = r.storico()
copia.append(123)
assert len(r) == 3, "storico() ha esposto la lista interna!"

print("Tutti i test superati")
```

Guarda quali casi abbiamo verificato, perché è uno schema che si ripete sempre: lo **stato iniziale** (l'oggetto appena creato è sensato?), il **caso normale**, i **casi limite** (lista vuota, un solo elemento, valore esattamente sulla soglia), e i **casi d'errore** (l'oggetto si difende dagli input sbagliati?). L'ultimo test è particolarmente istruttivo: verifica un invariante di incapsulamento, non un calcolo.

PROVA TU · 34 I primi controlli automatici con assert

`assert` significa: «questa cosa deve essere vera, altrimenti fermati». È il modo più semplice di controllare che una classe funzioni.

```
class Registro:
    def __init__(self):
        self._valori = []

    def registra(self, valore):
        if not -50 <= valore <= 150:
            raise ValueError("valore fuori scala")
        self._valori.append(float(valore))

    @property
    def media(self):
        if len(self._valori) == 0:
            return 0.0
        return sum(self._valori) / len(self._valori)

    def __len__(self):
        return len(self._valori)

r = Registro()

assert len(r) == 0
assert r.media == 0.0

r.registra(20)
r.registra(30)
assert r.media == 25.0

print("tutti i controlli sono passati")

assert r.media == 99, "questo controllo e' sbagliato di proposito"
```

Che cosa compare quando lo esegui (e' voluto!)

```
tutti i controlli sono passati
AssertionError: questo controllo e' sbagliato di proposito
```

Perché: finché le condizioni sono vere, `assert` non stampa niente: nessuna notizia, buona notizia. L'ultima riga fallisce apposta, così vedi com'è fatto il messaggio. Nota anche il metodo `__len__`: definendolo, puoi scrivere `len(r)` sul tuo oggetto come se fosse una lista.

Prova a cambiare: c'è però un limite: al primo controllo fallito il programma si ferma e non sai quanti degli altri sarebbero passati. Ed è esattamente il problema che risolve pytest, nella pagina successiva.

PROVA TU · 35 Controllare che l'oggetto si rifiuti

I controlli più utili non verificano che il codice funzioni: verificano che *si rifiuti* di fare le cose sbagliate.

```
class Registro:
    def __init__(self):
        self._valori = []

    def registra(self, valore):
        if not -50 <= valore <= 150:
            raise ValueError("valore fuori scala")
        self._valori.append(float(valore))

    def storico(self):
        return list(self._valori)

    def __len__(self):
        return len(self._valori)

r = Registro()
r.registra(20)

try:
    r.registra(999)
    print("PROBLEMA: il valore sbagliato e' stato accettato")
except ValueError:
    print("bene: il valore fuori scala e' stato rifiutato")

copia = r.storico()
copia.append(123)
print("valori nel registro:", len(r))
```

Che cosa compare quando lo esegui

```
bene: il valore fuori scala e' stato rifiutato
valori nel registro: 1
```

Perché: il primo controllo verifica che il registro respinga un valore impossibile. Il secondo verifica una cosa diversa e altrettanto importante: che `storico()` non consegni la lista interna, come nella prova 33. Se la consegnasse, l'ultima riga stamperebbe 2.

Prova a cambiare: prova a cambiare `storico()` in `return self._valori` e riesegui: l'ultima riga diventa 2, e hai appena scoperto una falla grazie a un controllo automatico. È esattamente il lavoro che fanno i test.

4.7 Da `assert` a `pytest`

Gli `assert` sparsi nel codice funzionano, ma hanno due limiti: al primo fallimento tutto si ferma, e non sai quanti controlli sarebbero passati. **pytest** risolve entrambi.

Si installa una volta sola con `pip install pytest` (in PyCharm dal terminale integrato, in un notebook con `!pip install pytest`) e chiede di rispettare due sole convenzioni: il file deve chiamarsi `test_qualscosa.py` e le funzioni devono iniziare per `test_`. A differenza delle prove che hai fatto finora, questi test stanno in un **file separato**, non in una cella del notebook.

```
# file: test_registro.py
import pytest
from registro import RegistroMisurazioni

def test_registro_nuovo_e_vuoto():
    r = RegistroMisurazioni("T-01")
    assert len(r) == 0
    assert r.ultima is None
    assert r.media == 0.0

def test_registra_una_misurazione():
    r = RegistroMisurazioni("T-01")
    r.registra(21.5)
    assert r.ultima == 21.5
    assert r.media == 21.5

def test_media_di_piu_valori():
    r = RegistroMisurazioni("T-01")
    r.registra(20)
    r.registra(30)
    assert r.media == 25.0

def test_allarme_sopra_soglia():
    r = RegistroMisurazioni("T-01", soglia_allarme=30)
    r.registra(35)
    assert r.in_allarme is True

def test_valore_fuori_scala_rifiutato():
    r = RegistroMisurazioni("T-01")
    with pytest.raises(ValueError):
        r.registra(999)

def test_valore_non_numerico_rifiutato():
    r = RegistroMisurazioni("T-01")
    with pytest.raises(TypeError):
        r.registra("caldo")
```

```
def test_storico_non_espone_la_lista_interna():
    r = RegistroMisurazioni("T-01")
    r.registra(20)
    r.storico().append(999)
    assert len(r) == 1
```

Per eseguirlo ti serve il **terminale**: in PyCharm lo apri con la linguetta *Terminal* in basso, in un notebook puoi scrivere lo stesso comando in una cella mettendogli davanti un punto esclamativo (`!pytest -v`). Posizionati nella cartella del progetto e lancia:

```
$ pytest -v

test_registro.py::test_registro_nuovo_e_vuoto PASSED [ 14%]
test_registro.py::test_registra_una_misurazione PASSED [ 28%]
test_registro.py::test_media_di_piu_valori PASSED [ 42%]
test_registro.py::test_allarme_sopra_soglia PASSED [ 57%]
test_registro.py::test_valore_fuori_scala_rifiutato PASSED [ 71%]
test_registro.py::test_valore_non_numerico_rifiutato PASSED [ 85%]
test_registro.py::test_storico_non_espone_la_lista_interna PASSED [100%]

===== 7 passed in 0.03s =====
```

Tre cose da portare a casa. Ogni test è **indipendente**: crea il suo oggetto da zero, così l'ordine di esecuzione non conta. Il **nome del test descrive il comportamento verificato**, tanto che leggendo l'elenco dei nomi si capisce cosa fa la classe. E `pytest.raises` serve a verificare che l'oggetto *si rifiuti* di fare la cosa sbagliata: sono i test più preziosi, perché l'incapsulamento serve esattamente a questo.

NOTA

Riprenderai pytest e il Test Driven Development in una lezione dedicata del corso. Quello che serve fissare adesso è il legame concettuale: **l'incapsulamento rende il codice testabile**. Una classe con un'interfaccia pubblica piccola e chiara si testa in dieci minuti; una classe i cui campi vengono modificati da mezzo programma non si testa affatto.

PROVA TU · 36 Il tuo primo pytest, passo per passo

Questa prova è l'unica che **non** si fa in una cella: pytest lavora sui file. Crea una cartella nuova e dentro due file, poi apri il terminale in quella cartella.

```
# --- file 1: registro.py -----

class Registro:
    def __init__(self):
        self._valori = []

    def registra(self, valore):
        if not -50 <= valore <= 150:
            raise ValueError("valore fuori scala")
        self._valori.append(float(valore))
```

```

@property
def media(self):
    if len(self._valori) == 0:
        return 0.0
    return sum(self._valori) / len(self._valori)

def __len__(self):
    return len(self._valori)

# --- file 2: test_registro.py -----

import pytest
from registro import Registro

def test_registro_nuovo_e_vuoto():
    r = Registro()
    assert len(r) == 0
    assert r.media == 0.0

def test_media_di_due_valori():
    r = Registro()
    r.registra(20)
    r.registra(30)
    assert r.media == 25.0

def test_valore_fuori_scala_rifiutato():
    r = Registro()
    with pytest.raises(ValueError):
        r.registra(999)

# --- poi, nel terminale, dai il comando:  pytest -v -----

```

Che cosa compare quando lo esegui

```

test_registro.py::test_registro_nuovo_e_vuoto PASSED          [ 33%]
test_registro.py::test_media_di_due_valori PASSED            [ 66%]
test_registro.py::test_valore_fuori_scala_rifiutato PASSED    [100%]

===== 3 passed in 0.02s =====

```

Perché: pytest trova da solo i file che iniziano per `test_` e le funzioni che iniziano per `test_`, le esegue tutte e ti fa il riepilogo. A differenza di `assert` da solo, **un test che fallisce non ferma gli altri**. La scrittura `with pytest.raises(ValueError):` significa: «qui dentro *deve* succedere questo errore, altrimenti il test fallisce».

Prova a cambiare: ora rompi il codice di proposito: in `registro.py` cambia la media in `sum(self._valori) / (len(self._valori) + 1)` e rilancia `pytest -v`. Ti dirà quale test è fallito, la riga esatta, il valore atteso e quello ottenuto. Poi rimetti a posto e verifica che tornino tutti verdi.

Esercizio 10 *Il conto bancario che si difende*



Scrivi una classe `ContoBancario` con questi requisiti:

- campi interni `_saldo` e `_movimenti`; campo pubblico `intestatario`;
- property `saldo` in **sola lettura**;
- `deposita(importo)`: rifiuta importi ≤ 0 con `ValueError`;
- `preleva(importo)`: rifiuta importi ≤ 0 e prelievi superiori al saldo, con messaggi d'errore diversi;
- `trasferisci_a(altro_conto, importo)`: preleva da questo conto e deposita sull'altro, **senza toccare direttamente i campi dell'altro oggetto**;
- `estratto_conto()` che restituisce una copia dei movimenti.

Verifica poi che `c.saldo = 1000` sollevi un errore.

Esercizio 11 *Property con validazione*



Riprendi `Studiante` e rendilo a prova di errore:

1. `email` diventa una property: il setter accetta solo stringhe contenenti `"@"`, altrimenti `ValueError`.
2. `nome` diventa una property: rifiuta stringhe vuote o di soli spazi, e memorizza il valore ripulito con `.strip()`.
3. `aggiungi_voto(voto)` accetta solo interi fra 0 e 30.
4. `media` e `promosso` diventano property calcolate.
5. I voti non devono essere modificabili dall'esterno: aggiungi `voti()` che ne restituisce una copia.

Verifica che `Studiante(" ", "x")` fallisca *già nel costruttore*. Se non fallisce, il costruttore non sta usando i setter: correggilo.

Esercizio 12 *Progettare prima di scrivere*



Devi realizzare una classe `Playlist` per un'applicazione musicale. **Prima di scrivere una sola riga di codice**, compila per iscritto lo schema delle quattro domande del paragrafo 4.5: responsabilità in una frase, operazioni pubbliche, dati interni, invarianti.

Solo dopo, implementa la classe. Deve gestire: aggiunta e rimozione di brani, durata totale, brano corrente, `avanti()` e `indietro()`, e un `mescola()`. Un invariante da rispettare: l'indice del brano corrente deve restare valido anche dopo una rimozione.

Consegna sia lo schema di progettazione sia il codice: qui il ragionamento vale quanto l'implementazione.

Esercizio 13 *Scrivi i test*



Scrivi un file `test_conto.py` con almeno **otto** test pytest per la classe `ContoBancario` dell'Esercizio 10. Devono coprire, come minimo:

- il saldo iniziale di un conto nuovo;
- un deposito e un prelievo validi;
- il rifiuto di un importo negativo (`pytest.raises`);
- il rifiuto di un prelievo superiore al saldo, e la verifica che *il saldo non sia cambiato* dopo il tentativo fallito;
- un trasferimento fra due conti, verificando **entrambi** i saldi;
- il fatto che `estratto_conto()` non esponga la lista interna;
- il caso limite del prelievo esattamente pari al saldo.

Regola ferrea: **nessun test può accedere a `_saldo` o `_movimenti`**. Se non riesci a testare qualcosa passando solo dall'interfaccia pubblica, è l'interfaccia a essere incompleta — sistemala.

RIEPILOGO DEL CAPITOLO

- L'**incapsulamento** nasconde lo stato interno ed espone solo operazioni controllate: i controlli stanno in un punto solo.
- Python usa convenzioni, non parole chiave: `_nome` significa "interno", `__nome` attiva il name mangling.
- Le **property** permettono di validare o calcolare un valore mantenendo la sintassi di un campo.
- L'**interfaccia pubblica** è un contratto: se lo rispetti, puoi riscrivere l'interno della classe senza rompere il codice altrui.
- Non restituire mai una struttura interna mutabile: restituiscine una copia.
- Si progetta partendo da **responsabilità, operazioni, dati, invarianti**; solo dopo si scrive il codice.
- I test verificano l'interfaccia pubblica, mai i campi interni: così sopravvivono alle riscritture.

Ereditarietà e overriding

IN QUESTO CAPITOLO

- Riuserai una classe esistente estendendola, invece di copiarla.
- Userai `super()` per non riscrivere il costruttore del padre.
- Farai *overriding*: sostituire o estendere il comportamento ereditato.
- Imparerai a riconoscere i casi in cui l'ereditarietà è la scelta sbagliata — sono più di quanti pensi.

5.1 Il codice duplicato che chiede aiuto

La nostra azienda ha dipendenti di tipo diverso. Cominciamo con gli impiegati:

```
class Dipendente:
    def __init__(self, nome, matricola, stipendio_base):
        self.nome = nome
        self.matricola = matricola
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

    def scheda(self):
        return f"{self.matricola} - {self.nome}: {self.stipendio_mensile():.2f} EUR/mese"
```

Poi arrivano i manager. Un manager è un dipendente in tutto e per tutto, ma ha in più un budget e una squadra, e prende un bonus. La tentazione è il copia-incolla:

```
class Manager:
    # DA NON FARE
    def __init__(self, nome, matricola, stipendio_base, budget):
        self.nome = nome
        self.matricola = matricola
        self.stipendio_base = stipendio_base
        self.budget = budget
        self.riporti = []

    def stipendio_mensile(self):
        return self.stipendio_base / 12 * 1.20 # quasi duplicato

    def scheda(self):
        # duplicato identico
        return f"{self.matricola} - {self.nome}: {self.stipendio_mensile():.2f} EUR/mese"
```

Il codice duplicato non è brutto per motivi estetici: è brutto perché **ogni correzione futura va fatta in due punti**, e prima o poi qualcuno ne dimenticherà uno. Se domani la scheda deve includere l'email,

hai due classi da modificare; con cinque tipi di dipendente, cinque.

C'è anche un problema più profondo. Il codice non dice da nessuna parte che *un manager è un dipendente*. Per Python, `Dipendente` e `Manager` sono due tipi scollegati, e una funzione che elabora dipendenti non può presumere niente sui manager.

PROVA TU · 37 Che cosa succede al codice copiato

Ecco due classi quasi identiche, nate da un copia-incolla. Qualcuno ha poi migliorato la scheda... ma in una sola delle due.

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

    def scheda(self):
        return "DIPENDENTE " + self.nome + ": " + str(self.stipendio_mensile())

class Manager:
    # copia-incolla della classe qui sopra
    def __init__(self, nome, stipendio_base, budget):
        self.nome = nome
        self.stipendio_base = stipendio_base
        self.budget = budget

    def stipendio_mensile(self):
        return self.stipendio_base / 12 * 1.20

    def scheda(self):
        return self.nome + ": " + str(self.stipendio_mensile())

print(Dipendente("Luca", 30000).scheda())
print(Manager("Sara", 60000, 250000).scheda())
```

Che cosa compare quando lo esegui

```
DIPENDENTE Luca: 2500.0
Sara: 6000.0
```

Perché: le due schede sono diverse: la prima scrive «DIPENDENTE» davanti al nome, la seconda no. È successo perché chi ha aggiunto quella parola ha modificato una sola delle due classi e si è dimenticato dell'altra. Nota la cosa peggiore: **non c'è nessun errore**. Il programma funziona e mente.

Prova a cambiare: il codice copiato non si rompe, *diverge in silenzio*. Con cinque tipi di dipendente le classi da tenere allineate a mano sarebbero cinque. Nel prossimo paragrafo elimineremo del tutto questa duplicazione.

5.2 Estendere una classe

L'**ereditarietà** permette di dichiarare che una classe è una versione specializzata di un'altra:

```
class Manager(Dipendente):          # Manager EREDITA da Dipendente
    pass
```

```
m = Manager("Sara Verdi", "M-01", 60000)
print(m.nome)                       # Sara Verdi -- ereditato
print(m.stipendio_mensile())        # 5000.0 -- ereditato
print(m.scheda())                   # M-01 - Sara Verdi: 5000.00 EUR/mese
```

Senza scrivere una riga, `Manager` ha già tutti i campi e tutti i metodi di `Dipendente`. La terminologia: `Dipendente` è la **classe base** (o superclasse, o classe padre), `Manager` è la **classe derivata** (o sottoclasse, o classe figlia).

Il criterio per decidere se usare l'ereditarietà è il **test "è un"**: la frase «un manager è un dipendente» ha senso, quindi l'ereditarietà è appropriata. Se invece la frase naturale è «ha un» — «un'auto ha un motore» — allora la relazione giusta è la composizione del capitolo 3, non l'ereditarietà.

PROVA TU · 38 Ereditare tutto scrivendo due righe

Adesso cancelliamo la copia e diciamo semplicemente che un manager è un dipendente.

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

    def scheda(self):
        return self.nome + " prende " + str(self.stipendio_mensile()) + " al mese"

class Manager(Dipendente):          # eredita tutto da Dipendente
    pass

m = Manager("Sara", 60000)
print(m.nome)
print(m.stipendio_mensile())
print(m.scheda())
print(Manager.__mro__)
```

Che cosa compare quando lo esegui

```
Sara
5000.0
Sara prende 5000.0 al mese
(<class '__main__.Manager'>, <class '__main__.Dipendente'>, <class 'object'>)
```

Perché: non hai scritto nemmeno un campo né un metodo dentro `Manager`, eppure funziona tutto. L'ultima riga mostra l'elenco dei posti in cui Python va a cercare, in ordine, quando gli chiedi qualcosa: prima `Manager`, poi `Dipendente`, poi `object` (la classe da cui deriva tutto). Si ferma appena trova.

Prova a cambiare: aggiungi `print(m.reparto)`: ottieni un `AttributeError` dopo che Python ha cercato invano in tutte e tre le classi dell'elenco.

5.3 `super()`: riusare il costruttore del padre

Un manager però ha dati in più. Serve un suo `__init__`, senza riscrivere quello del padre:

```
class Manager(Dipendente):
    def __init__(self, nome, matricola, stipendio_base, budget):
        super().__init__(nome, matricola, stipendio_base) # il padre fa la sua parte
        self.budget = budget                             # poi aggiungo il mio
        self.riporti = []

    def aggiungi_rapporto(self, dipendente):
        self.riporti.append(dipendente)
```

```
m = Manager("Sara Verdi", "M-01", 60000, budget=250000)
print(m.nome)      # Sara Verdi    -- impostato dal padre
print(m.budget)    # 250000       -- impostato dal figlio
```

`super()` significa «la classe da cui eredito». La riga `super().__init__(...)` chiama il costruttore del padre sullo stesso oggetto che stiamo costruendo: il padre inizializza la sua parte, poi il figlio aggiunge la propria.

ERRORE COMUNE

Dimenticare `super().__init__()` quando ridefinisci il costruttore. In quel caso i campi del padre **non vengono mai creati**, e otterrai un `AttributeError` alla prima riga che li usa:

```
class Manager(Dipendente):
    def __init__(self, nome, budget):
        self.budget = budget          # manca super().__init__(...)

m = Manager("Sara", 250000)
m.stipendio_mensile()
# AttributeError: 'Manager' object has no attribute 'stipendio_base'
```

Regola pratica: se scrivi `__init__` in una sottoclasse, la **prima riga** è quasi sempre `super().__init__(...)`.

PROVA TU · 39 Quando ti dimentichi di chiamare il padre

Se la sottoclasse ha bisogno di dati suoi, deve scrivere il proprio `__init__`. Vediamo che succede se ci si ferma lì.

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

class Manager(Dipendente):
    def __init__(self, budget):
        self.budget = budget          # manca la chiamata al costruttore del padre

m = Manager(250000)
print("budget:", m.budget)
print("nome:", m.nome)
```

Che cosa compare quando lo esegui (e' voluto!)

```
budget: 250000
AttributeError: 'Manager' object has no attribute 'nome'
```

Perché: scrivendo `__init__` nella sottoclasse hai **sostituito** quello del padre, non aggiunto qualcosa al suo. Risultato: l'oggetto nasce a metà, con il budget ma senza nome né stipendio. L'errore arriva solo quando qualcuno prova a usare un campo mancante.

Prova a cambiare: correggi scrivendo il costruttore così:

```
def __init__(self, nome, stipendio_base, budget):
    super().__init__(nome, stipendio_base)
```

```
self.budget = budget
```

e crea il manager con `Manager("Sara", 60000, 250000)`. **Regola pratica:** se scrivi `__init__` in una sottoclasse, la prima riga è quasi sempre `super().__init__(...)`.

5.4 Overriding: sovrascrivere un metodo

Il manager prende il 20% in più. Basta ridefinire il metodo nella sottoclasse: si chiama **overriding** (sovrascrittura).

```
class Manager(Dipendente):
    BONUS = 1.20

    def __init__(self, nome, matricola, stipendio_base, budget):
        super().__init__(nome, matricola, stipendio_base)
        self.budget = budget
        self.riporti = []

    def stipendio_mensile(self):
        return super().stipendio_mensile() * self.BONUS
```

```
d = Dipendente("Luca Bianchi", "D-07", 30000)
m = Manager("Sara Verdi", "M-01", 60000, 250000)

print(d.stipendio_mensile())    # 2500.0
print(m.stipendio_mensile())    # 6000.0 (5000 * 1.20)
print(m.scheda())              # M-01 - Sara Verdi: 6000.00 EUR/mese
```

Fermati un attimo sull'ultima riga, perché è il punto più importante del capitolo. Il metodo `scheda()` è definito **solo** in `Dipendente`, e al suo interno chiama `self.stipendio_mensile()`. Quando lo eseguiamo su un manager, quale versione viene chiamata?

Quella del *manager*. Perché `self` è l'oggetto reale, e Python cerca sempre il metodo **partendo dalla classe effettiva dell'oggetto**, non da quella in cui il codice è scritto. Il codice scritto in `Dipendente`, senza saperlo e senza essere modificato, ha usato il comportamento specializzato del figlio.

NOTA

Questo meccanismo si chiama **dispatch dinamico**, ed è il motore del polimorfismo che vedremo nel capitolo 6. Ti permette di scrivere codice generico nella classe base e di iniettare comportamenti specifici dalle sottoclassi, senza toccare una riga della base.

Quando Python cerca un attributo o un metodo su un oggetto, segue quest'ordine: prima nell'istanza, poi nella sua classe, poi nella classe padre, poi nel padre del padre, fino a `object`. La prima definizione trovata vince. Se non trova nulla, `AttributeError`.

Adesso il manager prende il 20% in più. Basta riscrivere un metodo: si chiama *overriding*.

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

class Manager(Dipendente):
    def __init__(self, nome, stipendio_base, budget):
        super().__init__(nome, stipendio_base)      # il padre fa la sua parte
        self.budget = budget                       # poi aggiungo la mia

    def stipendio_mensile(self):
        return super().stipendio_mensile() * 1.20  # riuso il calcolo e aggiungo il
bonus

d = Dipendente("Luca", 60000)
m = Manager("Sara", 60000, 250000)

print(d.nome, d.stipendio_mensile())
print(m.nome, m.stipendio_mensile(), m.budget)
```

Che cosa compare quando lo esegui

```
Luca 5000.0
Sara 6000.0 250000
```

Perché: stesso stipendio di partenza, stesso nome di metodo, risultati diversi. Nota che `Manager.stipendio_mensile` non riscrive la divisione per 12: chiama `super().stipendio_mensile()`, cioè la versione del padre, e poi ci aggiunge il bonus. Se un giorno cambia il calcolo di base, il bonus continuerà ad applicarsi correttamente.

Prova a cambiare: la parola `super()` significa «la classe da cui eredito». La usi sia nel costruttore sia dentro i metodi riscritti, ogni volta che vuoi *aggiungere* qualcosa al comportamento del padre invece di buttarlo via.

PROVA TU · 41 Il meccanismo piu' importante del capitolo

Fermati su questa prova più a lungo delle altre. Il metodo `scheda()` esiste **solo** nella classe padre. Quale calcolo userà quando lo chiamiamo su un manager?

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
```

```

        print("    (sto usando il calcolo del DIPENDENTE)")
        return self.stipendio_base / 12

    def scheda(self):
        print("scheda di", self.nome, "- adesso chiamo self.stipendio_mensile()")
        return self.stipendio_mensile()

class Manager(Dipendente):
    def stipendio_mensile(self):
        print("    (sto usando il calcolo del MANAGER)")
        return self.stipendio_base / 12 * 1.20

print(Dipendente("Luca", 60000).scheda())
print()
print(Manager("Sara", 60000).scheda())

```

Che cosa compare quando lo esegui

```

scheda di Luca - adesso chiamo self.stipendio_mensile()
    (sto usando il calcolo del DIPENDENTE)
5000.0

scheda di Sara - adesso chiamo self.stipendio_mensile()
    (sto usando il calcolo del MANAGER)
6000.0

```

Perché: la risposta è nell'output: quando `scheda()` viene chiamato su un manager, usa il calcolo del **manager**, anche se `scheda()` è scritto dentro `Dipendente` e non sa nemmeno che i manager esistano. Il motivo è che `self` è sempre l'oggetto vero, e Python cerca i metodi partendo dalla sua classe reale.

Prova a cambiare: questo è il motore di tutto il capitolo 6: puoi scrivere codice generico nella classe padre e far sì che ogni figlio ci infili dentro il proprio comportamento, **senza modificare una riga del padre**.

5.5 Estendere invece di sostituire

Nell'overriding hai due possibilità: **sostituire** completamente il comportamento del padre, oppure **estenderlo** chiamando `super()` e aggiungendo qualcosa.

```

class Manager(Dipendente):

    def stipendio_mensile(self):
        return super().stipendio_mensile() * 1.20      # ESTENDE

    def scheda(self):
        base = super().scheda()                        # ESTENDE
        return f"{base} | budget {self.budget} EUR | {len(self.riporti)} riporti"

```

```
class Consulente(Dipendente):

    def __init__(self, nome, matricola, tariffa_oraria):
        super().__init__(nome, matricola, stipendio_base=0)
        self.tariffa_oraria = tariffa_oraria
        self.ore_mese = 0

    def stipendio_mensile(self):
        return self.tariffa_oraria * self.ore_mese      # SOSTITUISCE del tutto
```

```
m = Manager("Sara Verdi", "M-01", 60000, 250000)
m.aggiungi_riporto(Dipendente("Luca", "D-07", 30000))
print(m.scheda())
# M-01 - Sara Verdi: 6000.00 EUR/mese | budget 250000 EUR | 1 riporti

c = Consulente("Marco Neri", "C-03", tariffa_oraria=75)
c.ore_mese = 120
print(c.scheda())
# C-03 - Marco Neri: 9000.00 EUR/mese
```

Il `Consulente` ignora completamente la formula del padre, perché per lui non ha senso. Va benissimo: l'importante è che `stipendio_mensile()` continui a *significare* la stessa cosa (quanto costa al mese questa persona) e a restituire lo stesso tipo di dato. È la promessa dell'interfaccia del capitolo 4, applicata alle gerarchie.

ATTENZIONE

Una sottoclasse non deve mai *tradire* il contratto del padre. Se `stipendio_mensile()` del padre restituisce sempre un numero positivo, il figlio non può restituire `None` o sollevare un'eccezione a sorpresa: tutto il codice che lavora con dipendenti generici si romperebbe. Questa regola ha un nome altisonante, *principio di sostituzione di Liskov*, e dice una cosa semplice: **dove funziona il padre, deve funzionare anche il figlio.**

PROVA TU · 42 Aggiungere al padre oppure sostituirlo

Due sottoclassi che trattano i metodi del padre in modo opposto: una li estende, l'altra li rimpiazza.

```
class Notifica:
    def __init__(self, destinatario, testo):
        self.destinatario = destinatario
        self.testo = testo

    def valida(self):
        if self.testo == "":
            raise ValueError("il testo non puo' essere vuoto")

    def invia(self):
        self.valida()
        return "a " + self.destinatario + ": " + self.testo
```

```

class NotificaEmail(Notifica):
    def valida(self):
        super().valida()           # ESTENDE: tiene il controllo del padre
        if "@" not in self.destinatario:
            raise ValueError("email non valida")

    def invia(self):
        self.valida()
        return "[EMAIL] a " + self.destinatario + ": " + self.testo

class NotificaSMS(Notifica):
    def invia(self):               # SOSTITUISCE del tutto
        self.valida()
        return "[SMS] " + self.testo.upper()

notifiche = [
    NotificaEmail("anna@example.com", "riunione alle 15"),
    NotificaSMS("+39333", "riunione alle 15"),
]

for n in notifiche:
    print(n.invia())

try:
    NotificaEmail("anna-senza-chiocciola", "ciao").invia()
except ValueError as errore:
    print("rifiutata:", errore)

```

Che cosa compare quando lo esegui

```

[EMAIL] a anna@example.com: riunione alle 15
[SMS] RIUNIONE ALLE 15
rifiutata: email non valida

```

Perché: `NotificaEmail.valida` chiama `super().valida()` e poi aggiunge il suo controllo: il controllo del padre (testo non vuoto) vale ancora. `NotificaSMS.invia` invece riscrive tutto da capo, perché un SMS ha un formato completamente diverso. Entrambe le scelte sono corrette: dipende se il comportamento del padre ti serve ancora oppure no.

Prova a cambiare: prova a togliere `super().valida()` dalla email e a creare una notifica con testo vuoto: verrà accettata, perché hai buttato via il controllo del padre senza accorgertene.

5.6 `isinstance` e la catena di ereditarietà

Un oggetto appartiene alla sua classe e a tutte le classi da cui essa deriva:

```

m = Manager("Sara Verdi", "M-01", 60000, 250000)

print(isinstance(m, Manager))      # True
print(isinstance(m, Dipendente))   # True  <-- un manager È un dipendente
print(isinstance(m, object))       # True  <-- tutto eredita da object
print(type(m) is Dipendente)       # False <-- type() è esatto, non gerarchico
print(issubclass(Manager, Dipendente)) # True

```

La differenza fra `isinstance()` e `type()` conta: `isinstance()` tiene conto della gerarchia, `type()` no. Nel 99% dei casi vuoi `isinstance()`.

Il vantaggio pratico è che puoi trattare l'intera gerarchia in modo uniforme:

```

azienda = [
    Dipendente("Luca Bianchi", "D-07", 30000),
    Manager("Sara Verdi", "M-01", 60000, 250000),
    Consulente("Marco Neri", "C-03", 75),
]

costo_totale = 0
for persona in azienda:
    print(persona.scheda())          # ognuno usa la SUA versione
    costo_totale += persona.stipendio_mensile()

print(f"Costo mensile: {costo_totale:.2f} EUR")

```

Il ciclo non sa e non deve sapere che tipo di dipendente sta trattando. Chiede a ciascuno il suo stipendio, e ognuno risponde secondo le proprie regole. Questo è il polimorfismo, e il capitolo 6 lo porterà alle sue conseguenze.

5.7 Quando l'ereditarietà è la scelta sbagliata

L'ereditarietà è lo strumento di cui i principianti abusano di più. È potente ma crea un legame fortissimo: la sottoclasse dipende dai dettagli interni del padre, e una modifica al padre può rompere silenziosamente tutti i figli.

Primo caso da evitare: la relazione non è "è un".

```

class Motore:
    def avvia(self): ...

class Auto(Motore):          # SBAGLIATO: un'auto non È un motore
    pass

class Auto:                  # GIUSTO: un'auto HA un motore
    def __init__(self, modello, motore):
        self.modello = modello
        self.motore = motore    # composizione

```

```
def avvia(self):
    return self.motore.avvia()    # delega
```

PROVA TU · 43 «E' un» oppure «ha un»

L'errore più comune con l'ereditarietà è usarla quando la relazione giusta è un'altra. Confrontiamo le due versioni.

```
class Motore:
    def __init__(self, cavalli):
        self.cavalli = cavalli

    def avvia(self):
        return "motore da " + str(self.cavalli) + " CV avviato"

class AutoSbagliata(Motore):          # un'auto NON e' un motore
    pass

class AutoGiusta:
    def __init__(self, modello, motore):
        self.modello = modello
        self.motore = motore        # L'auto HA un motore

    def avvia(self):
        return self.motore.avvia()   # gira la richiesta al motore

a = AutoSbagliata(120)
print("sbagliata:", a.avvia())

b = AutoGiusta("Panda", Motore(70))
print("giusta:", b.avvia())

b.motore = Motore(120)              # il motore si puo' sostituire
print("giusta, dopo il cambio motore:", b.avvia())
```

Che cosa compare quando lo esegui

```
sbagliata: motore da 120 CV avviato
giusta: motore da 70 CV avviato
giusta, dopo il cambio motore: motore da 120 CV avviato
```

Perché: la versione sbagliata funziona, e questo è proprio il problema: nasconde un modello falso. Guarda l'ultima riga per capire la differenza pratica: nella versione giusta puoi **sostituire il motore** a auto già costruita. Con l'ereditarietà non potresti: la classe di un oggetto è decisa per sempre nel momento in cui lo crei.

Prova a cambiare: la prova del nove è leggere la frase ad alta voce. «Un manager è *un* dipendente»: suona bene, usa l'ereditarietà. «Un'auto è *un* motore»: suona male, l'auto *ha un* motore, quindi mettilo dentro come dato.

Secondo caso: ereditare solo per riusare due metodi comodi. Se `GestoreFile` eredita da `Logger` solo per avere `log()`, hai legato due concetti che non c'entrano nulla fra loro. Meglio contenere un logger come campo, o importare una funzione.

Terzo caso: gerarchie troppo profonde. Quando arrivi a `A → B → C → D`, per capire cosa fa un metodo devi risalire quattro file. Nel codice professionale si tende a restare su uno, al massimo due livelli, e a preferire la composizione quando la gerarchia inizia a gonfiarsi.

LA REGOLA D'ORO

Preferisci la composizione all'ereditarietà. Usa l'ereditarietà quando le sottoclassi sono *varianti dello stesso concetto* che devono essere usate in modo intercambiabile — è il caso del capitolo 7. Per tutto il resto, contieni un oggetto e delega.

PROVA TU · 44 Un ciclo che non sa con chi ha a che fare

Mettiamo insieme tutta la famiglia di classi e facciamo un unico ciclo.

```
class Dipendente:
    def __init__(self, nome, stipendio_base):
        self.nome = nome
        self.stipendio_base = stipendio_base

    def stipendio_mensile(self):
        return self.stipendio_base / 12

class Manager(Dipendente):
    def stipendio_mensile(self):
        return super().stipendio_mensile() * 1.20

class Apprendista(Dipendente):
    def stipendio_mensile(self):
        return super().stipendio_mensile() * 0.50

azienda = [
    Dipendente("Luca", 30000),
    Manager("Sara", 60000),
    Apprendista("Marco", 24000),
]

costo = 0
for persona in azienda:
    print(type(persona).__name__, persona.nome, persona.stipendio_mensile())
    costo = costo + persona.stipendio_mensile()
```

```
print("costo mensile totale:", costo)
```

Che cosa compare quando lo esegui

```
Dipendente Luca 2500.0
Manager Sara 6000.0
Apprendista Marco 1000.0
costo mensile totale: 9500.0
```

Perché: il ciclo non contiene nessun `if` sul tipo di persona: chiede a ciascuno il proprio stipendio e ognuno risponde secondo le sue regole. `type(persona).__name__` serve solo a farti vedere chi è chi.

Prova a cambiare: aggiungi una quarta classe, per esempio `Consulente`, e mettila nell'elenco. Quante righe del ciclo devi modificare? **Nessuna.** Tieni a mente questa sensazione: si chiama polimorfismo ed è l'argomento del capitolo 6.

PROVA TU · 45 `isinstance` oppure `type`

Due modi di chiedere «di che tipo è questo oggetto?», con risposte diverse.

```
class Dipendente:
    def __init__(self, nome):
        self.nome = nome

class Manager(Dipendente):
    pass

m = Manager("Sara")

print("m e' un Manager?", isinstance(m, Manager))
print("m e' un Dipendente?", isinstance(m, Dipendente))
print("il tipo esatto di m e' Dipendente?", type(m) is Dipendente)
print("Manager deriva da Dipendente?", issubclass(Manager, Dipendente))
```

Che cosa compare quando lo esegui

```
m e' un Manager? True
m e' un Dipendente? True
il tipo esatto di m e' Dipendente? False
Manager deriva da Dipendente? True
```

Perché: `isinstance()` tiene conto dell'ereditarietà: un manager è un manager *e anche* un dipendente. `type()` invece guarda solo il tipo esatto e risponde di no.

Prova a cambiare: quasi sempre vuoi `isinstance()`. Se scrivessi una funzione che accetta dipendenti controllando con `type(x) is Dipendente`, rifiuteresti proprio i manager, che sono dipendenti a tutti gli effetti.

Esercizio 14 *La gerarchia dei dipendenti*

●○○

Completa la gerarchia del capitolo:

1. Scrivi `Dipendente`, `Manager` e `Consulente` come mostrato.
2. Aggiungi `Apprendista(Dipendente)`, che prende il 60% dello stipendio base e la cui `scheda()` aggiunge il testo `"(in formazione)"`.
3. Crea una lista con almeno un'istanza per tipo e stampa il costo mensile totale.
4. Scrivi una funzione `più_costoso(persone)` che restituisce il dipendente con lo stipendio mensile più alto. Non deve contenere nessun `isinstance`.

Domanda finale: perché il punto 4 può essere risolto senza sapere quali tipi ci sono nella lista?

Esercizio 15 *Overriding: estendere o sostituire*

●●○

Parti da questa classe base:

```
class Notifica:
    def __init__(self, destinatario, testo):
        self.destinatario = destinatario
        self.testo = testo

    def valida(self):
        if not self.testo:
            raise ValueError("Il testo non può essere vuoto")

    def invia(self):
        self.valida()
        return f"A {self.destinatario}: {self.testo}"
```

1. `NotificaEmail`: aggiunge un campo `oggetto`; `valida()` **estende** quella del padre verificando anche che il destinatario contenga `"@"`; `invia()` restituisce una stringa che include l'oggetto.
2. `NotificaSMS`: `valida()` estende il padre verificando che il testo non superi i 160 caratteri; `invia()` **sostituisce** completamente il comportamento del padre.
3. Crea una lista mista di notifiche e inviale tutte con un solo ciclo.

Per ciascun metodo che scrivi, annota in un commento se hai *esteso* (con `super()`) o *sostituito*, e perché.

Esercizio 16 "È un" oppure "ha un"?



Per ciascuna coppia, decidi se la relazione corretta è **ereditarietà** o **composizione**, e giustifica in una frase:

1. Quadrato e Rettangolo
2. Ordine e Cliente
3. ContoCorrente e ContoDeposito
4. Playlist e Brano
5. FileCSV e SorgenteDati
6. Computer e ProcessorePentium

Il caso 1 è celebre e più insidioso di quanto sembri: prova a implementare `Quadrato(Rettangolo)` con un metodo `imposta_larghezza()` e osserva cosa succede all'invariante «tutti i lati sono uguali».

Discussione in Appendice A.

Esercizio 17 Rifattorizzare il duplicato



Ti viene consegnato questo codice, pieno di duplicazione:

```
class FormaRettangolo:
    def __init__(self, nome, base, altezza):
        self.nome = nome; self.base = base; self.altezza = altezza
    def area(self): return self.base * self.altezza
    def descrivi(self): return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"
    def perimetro(self): return 2 * (self.base + self.altezza)

class FormaCerchio:
    def __init__(self, nome, raggio):
        self.nome = nome; self.raggio = raggio
    def area(self): return 3.14159 * self.raggio ** 2
    def descrivi(self): return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"
    def perimetro(self): return 2 * 3.14159 * self.raggio
```

1. Individua tutto il codice duplicato.
2. Rifattorizza introducendo una classe base `Forma` che contenga `nome` e `descrivi()`, e due sottoclassi `Rettangolo` e `Cerchio`.
3. Aggiungi `Triangolo` senza toccare la classe base. Quante righe ti servono?
4. Che cosa dovrebbe fare `Forma.area()`, dato che una forma generica non ha una formula propria? Prova a rispondere prima di leggere il capitolo 6 — lì troverai la soluzione corretta.

Suggerimento: usa `math.pi` invece di `3.14159`.

RIEPILOGO DEL CAPITOLO

- `class Figlia(Padre):` eredita tutti i campi e i metodi della classe base.
- `super().__init__(...)` va chiamato come prima riga del costruttore della sottoclasse.
- L'**overriding** ridefinisce un metodo: puoi *estenderlo* (chiamando `super()`) o *sostituirlo*.
- Python cerca i metodi partendo dalla classe **reale** dell'oggetto: il codice della classe base usa automaticamente le versioni sovrascritte.
- `isinstance()` rispetta la gerarchia, `type()` no.
- Usa l'ereditarietà solo se regge il test "**è un**"; altrimenti usa la composizione.

Generalizzazione e astrazione

IN QUESTO CAPITOLO

- Imparerai a *generalizzare*: riconoscere il concetto comune dietro classi diverse.
- Scriverai codice polimorfico, che tratta oggetti diversi allo stesso modo.
- Definirai classi astratte con `abc`, che obbligano le sottoclassi a rispettare un contratto.
- Capirai quando astrarre è utile e quando è solo complicazione.

6.1 Generalizzare: salire di un livello

Nel capitolo 5 siamo andati dall'alto verso il basso: c'era `Dipendente`, e ne abbiamo derivato dei casi speciali. La **generalizzazione** è il percorso opposto, e nella pratica è quello che farai più spesso: hai già scritto diverse classi e ti accorgi che condividono un concetto.

Riprendiamo le forme dell'Esercizio 17. Rettangolo, cerchio e triangolo sono cose diverse, ma *tutte* hanno un nome, un'area e un perimetro. Quel «tutte hanno» è il segnale della generalizzazione: esiste un concetto più generale, `Forma`, di cui le tre classi sono casi particolari.

```
import math

class Forma:
    def __init__(self, nome):
        self.nome = nome

    def area(self):
        # ...e qui cosa scrivo? Una forma generica non ha una formula.
        return 0

    def descrivi(self):
        return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

    def perimetro(self):
        return 2 * (self.base + self.altezza)
```

```

class Cerchio(Forma):
    def __init__(self, raggio):
        super().__init__("Cerchio")
        self.raggio = raggio

    def area(self):
        return math.pi * self.raggio ** 2

    def perimetro(self):
        return 2 * math.pi * self.raggio

```

La duplicazione è sparita: `descrivi()` è scritto una volta sola e funziona per tutte le forme presenti e future. Ma resta il problema segnalato nel commento, ed è il cuore di questo capitolo: **che cosa deve fare `Forma.area()` ?**

Restituire `0` è una pessima idea: se domani scrivo `Triangolo` e mi dimentico di implementare `area()`, il programma non protesta e mi dà tranquillamente area zero per tutti i triangoli. Un bug silenzioso, il tipo peggiore. Ci arriviamo fra due paragrafi.

PROVA TU · 46 Il difetto nascosto del return 0

Mettiamo nella classe generica un'area che vale 0, e poi dimentichiamoci di scriverla in una sottoclasse. Succede spesso.

```

class Forma:
    def __init__(self, nome):
        self.nome = nome

    def area(self):
        return 0 # una forma generica non ha una formula...

    def descrivi(self):
        return self.nome + " ha area " + str(self.area())

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

class Triangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Triangolo")
        self.base = base
        self.altezza = altezza
    # qui ci siamo dimenticati di scrivere area()

```

```
print(Rettangolo(3, 4).descrivi())
print(Triangolo(3, 4).descrivi())
```

Che cosa compare quando lo esegui

```
Rettangolo ha area 12
Triangolo ha area 0
```

Perché: il triangolo dichiara tranquillamente di avere area zero. Nessun errore, nessun avviso: ha semplicemente ereditato l'area finta della classe generica. Se quel numero finisce in un totale, in un preventivo o in una fattura, l'errore viaggia indisturbato fino al cliente.

Prova a cambiare: tieni presente questo esempio: fra due pagine renderemo l'area *obbligatoria*, e questa stessa classe si rifiuterà di essere creata, dicendo esattamente che cosa manca.

6.2 Il polimorfismo in azione

Prima godiamoci il risultato. *Polimorfismo* significa letteralmente "molte forme", e in pratica vuol dire questo: **lo stesso codice funziona con oggetti di tipi diversi, purché espongano le stesse operazioni.**

```
forme = [Rettangolo(3, 4), Cerchio(5), Rettangolo(2, 2)]

for f in forme:
    print(f.descrivi())

area_totale = sum(f.area() for f in forme)
piu_grande = max(forme, key=lambda f: f.area())

print(f"Area totale: {area_totale:.2f}")
print(f"La più grande: {piu_grande.nome}")
```

```
Rettangolo: area 12.00, perimetro 14.00
Cerchio: area 78.54, perimetro 31.42
Rettangolo: area 4.00, perimetro 8.00
Area totale: 94.54
La più grande: Cerchio
```

Nessun `if`, nessun `isinstance`, nessun controllo di tipo. E soprattutto: **se domani aggiungo `Triangolo`, questo codice non va toccato.** Funziona già.

Confronta con la versione senza polimorfismo, che è quella che scriverebbe chi non conosce gli oggetti:

```
def area(forma):
    # FRAGILE
    if forma["tipo"] == "rettangolo":
        return forma["base"] * forma["altezza"]
    elif forma["tipo"] == "cerchio":
        return math.pi * forma["raggio"] ** 2
    elif forma["tipo"] == "triangolo":
        ...
    else:
        # e ogni nuova forma tocca QUESTA funzione
        raise ValueError("Tipo sconosciuto")
```

Ogni nuovo tipo obbliga a modificare la funzione, e insieme a lei ogni altra funzione costruita sullo stesso schema (`perimetro()` , `disegna()` , `esporta()` ...). Con il polimorfismo, un nuovo tipo si aggiunge **senza modificare nulla di esistente**: aggiungi una classe, e tutto il codice che già c'era la accetta.

NOTA

Questo è uno dei principi più citati della progettazione software: «*aperto all'estensione, chiuso alla modifica*». Una lunga catena di `if` sul *tipo* di qualcosa è quasi sempre il sintomo di un polimorfismo mancante. Quando ne vedi una, chiediti: «e se questi fossero oggetti diversi con lo stesso metodo?»

PROVA TU · 47 Lo stesso codice per forme diverse

Adesso il vantaggio: un unico ciclo che lavora su forme di tipo diverso, senza sapere quali sono.

```
import math

class Forma:
    def __init__(self, nome):
        self.nome = nome

    def descrivi(self):
        return self.nome + ": area " + str(round(self.area(), 2))

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

class Cerchio(Forma):
    def __init__(self, raggio):
        super().__init__("Cerchio")
        self.raggio = raggio

    def area(self):
        return math.pi * self.raggio ** 2
```

```

forme = [Rettangolo(3, 4), Cerchio(5), Rettangolo(2, 2)]

for f in forme:
    print(f.descrivi())

print("area totale:", round(sum(f.area() for f in forme), 2))
print("la piu' grande:", max(forme, key=lambda f: f.area()).nome)

```

Che cosa compare quando lo esegui

```

Rettangolo: area 12
Cerchio: area 78.54
Rettangolo: area 4
area totale: 94.54
la piu' grande: Cerchio

```

Perché: non c'è nessun `if`, nessun controllo di tipo. Il ciclo chiede a ogni forma la sua area e ognuna risponde a modo suo. E `descrivi()` è scritto **una volta sola** nella classe generica, ma stampa i valori giusti per tutte. Questo si chiama *polimorfismo*: molte forme, un solo modo di trattarle.

Prova a cambiare: aggiungi una classe `Quadrato(Forma)` con la sua `area()` e mettila nella lista, **senza toccare nient'altro**. Funziona subito: il ciclo, il totale e il massimo la accettano senza modifiche.

6.3 Classi astratte con `abc`

Torniamo al problema di `Forma.area()`. Quello che vogliamo dire è: «ogni forma deve avere un'area, ma la formula la decide la sottoclasse; e una "forma generica" non esiste, non deve essere possibile crearne una».

Questa è l'**astrazione**: descrivere *che cosa serve* senza dire *come* si fa. Python lo supporta con il modulo `abc` (*Abstract Base Class*).

```

from abc import ABC, abstractmethod
import math

class Forma(ABC):
    # ABC = classe astratta

    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def area(self):
        """Restituisce l'area della forma. Ogni sottoclasse DEVE implementarlo."""

    @abstractmethod
    def perimetro(self):
        """Restituisce il perimetro della forma."""

```

```
def descrivi(self):                # metodo concreto: ereditato da tutti
    return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"

def piu_grande_di(self, altra):    # funziona con QUALSIASI forma
    return self.area() > altra.area()
```

Ora la classe base fa due promesse fortissime.

Non si può istanziare una forma generica:

```
f = Forma("qualcosa")
# TypeError: Can't instantiate abstract class Forma without an implementation
#           for abstract methods 'area', 'perimetro'
```

Chi eredita è obbligato a implementare tutto:

```
class Triangolo(Forma):
    def __init__(self, a, b, c):
        super().__init__("Triangolo")
        self.a, self.b, self.c = a, b, c

    def perimetro(self):
        return self.a + self.b + self.c
    # area() dimenticata!

t = Triangolo(3, 4, 5)
# TypeError: Can't instantiate abstract class Triangolo without an implementation
#           for abstract method 'area'
```

Questa è la differenza decisiva rispetto al `return 0` di prima: l'errore arriva **subito, al momento della creazione dell'oggetto**, con un messaggio che dice esattamente cosa manca. Non un risultato sbagliato scoperto in produzione tre mesi dopo.

La versione completa:

```
class Triangolo(Forma):
    def __init__(self, a, b, c):
        super().__init__("Triangolo")
        self.a, self.b, self.c = a, b, c

    def area(self):
        s = self.perimetro() / 2                # formula di Erone
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))

    def perimetro(self):
        return self.a + self.b + self.c
```

```
forme = [Rettangolo(3, 4), Cerchio(5), Triangolo(3, 4, 5)]
for f in forme:
    print(f.descrivi())
# Triangolo: area 6.00, perimetro 12.00
```

NOTA

Una classe astratta può contenere **sia metodi astratti sia metodi concreti**. È la combinazione che la rende utile: i metodi astratti definiscono il contratto che ogni sottoclasse deve firmare (`area` , `perimetro`), i metodi concreti forniscono comportamento già pronto costruito su quel contratto (`descrivi` , `piu_grande_di`). Scrivi una volta la logica generale, e ogni nuova sottoclasse la eredita gratis.

PROVA TU · 48 Rendere un metodo obbligatorio

Riprendiamo la dimenticanza della prova 46, ma stavolta dichiariamo l'area come metodo **astratto**, cioè obbligatorio.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def area(self):
        """Ogni forma DEVE dire come si calcola la sua area."""

    def descrivi(self):
        return self.nome + " ha area " + str(self.area())

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

class Triangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Triangolo")
        self.base = base
        self.altezza = altezza
    # anche qui manca area(), ma stavolta non passa liscia
```

```
print(Rettangolo(3, 4).descrivi())
Triangolo(3, 4)
```

Che cosa compare quando lo esegui (e' voluto!)

```
Rettangolo ha area 12
TypeError: Can't instantiate abstract class Triangolo without an implementation for
abstract method 'area'
```

Perché: leggi per intero il messaggio: ti dice *quale classe* è incompleta e *quale metodo* manca. E lo dice **nel momento in cui provi a creare l'oggetto**, non mesi dopo dentro una fattura sbagliata. Confronta con lo zero silenzioso della prova 46: stessa dimenticanza, due mondi diversi.

Prova a cambiare: aggiungi a `Triangolo` il metodo `def area(self): return self.base * self.altezza / 2` ed esegui di nuovo: adesso l'oggetto si crea e stampa `Triangolo ha area 6.0`.

PROVA TU · 49 Quello che la classe generica regala ai figli

Una classe astratta non contiene solo obblighi: contiene anche metodi già pronti che tutte le sottoclassi ricevono gratis.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def area(self):
        """Ogni forma deve dire come si calcola la sua area."""

    def descrivi(self):
        # regalo numero 1
        return self.nome + " ha area " + str(self.area())

    def piu_grande_di(self, altra):
        # regalo numero 2
        return self.area() > altra.area()

    def __lt__(self, altra):
        # regalo numero 3: ordinamento
        return self.area() < altra.area()

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza
```

```

class Quadrato(Rettangolo):
    def __init__(self, lato):
        super().__init__(lato, lato)
        self.nome = "Quadrato"

r = Rettangolo(3, 4)
q = Quadrato(5)

print(r.descrivi())
print(q.descrivi())
print("il quadrato e' piu' grande?", q.piu_grande_di(r))

for f in sorted([q, r]):
    print("in ordine di area:", f.nome, f.area())

```

Che cosa compare quando lo esegui

```

Rettangolo ha area 12
Quadrato ha area 25
il quadrato e' piu' grande? True
in ordine di area: Rettangolo 12
in ordine di area: Quadrato 25

```

Perché: hai scritto tre metodi in un punto solo e li hanno ottenuti tutte le forme, comprese quelle che scriverai domani. Il `Quadrato` non ha nemmeno l'area: la eredita dal `Rettangolo`, perché un quadrato è un rettangolo con i lati uguali. E definendo `__lt__` hai reso le tue forme ordinabili con `sorted()`, come se fossero numeri.

Prova a cambiare: il modello da ricordare è questo: i metodi **astratti** sono il contratto che ogni sottoclasse deve firmare, i metodi **normali** sono il vantaggio che riceve in cambio della firma.

6.4 Il duck typing pythonico

C'è una particolarità di Python che va conosciuta, perché altrimenti le classi astratte sembrano più obbligatorie di quanto siano. In Python, per poter usare un oggetto **non serve che appartenga a una certa classe: basta che abbia i metodi giusti**.

```

class Rombo:                                # NON eredita da Forma
    def __init__(self, d1, d2, lato):
        self.nome = "Rombo"
        self.d1, self.d2, self.lato = d1, d2, lato

    def area(self):
        return self.d1 * self.d2 / 2

    def perimetro(self):
        return 4 * self.lato

    def descrivi(self):
        return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"

```

```
forme = [Rettangolo(3, 4), Rombo(6, 8, 5)]
for f in forme:
    print(f.descrivi())          # funziona lo stesso!
```

Il principio si chiama **duck typing**, dal detto inglese: «*se cammina come un'anatra e starnazza come un'anatra, allora è un'anatra*». A Python non interessa il certificato di nascita dell'oggetto, interessa cosa sa fare.

Perché allora usare le classi astratte? Per tre motivi concreti:

- **Documentano il contratto.** Chi deve scrivere una nuova forma legge `Forma` e sa esattamente cosa implementare, senza indovinare.
- **Falliscono presto.** Con il duck typing, un metodo dimenticato si scopre solo quando qualcuno lo chiama, magari in un ramo raro del programma. Con `abc`, si scopre alla creazione dell'oggetto.
- **Condividono codice.** `descrivi()` e `piu_grande_di()` sono scritti una volta nella classe astratta; il `Rombo` qui sopra ha dovuto ricopiarsi `descrivi()`.

La linea di condotta comune nei progetti professionali: duck typing per le cose piccole e locali, classe astratta quando stai definendo un **punto di estensione** che altri (o il te stesso di fra sei mesi) dovranno implementare. Il componente del capitolo 7 è esattamente questo caso.

PROVA TU · 50 Se si comporta come una forma, va bene lo stesso

Una particolarità di Python: per usare un oggetto non serve che appartenga a una certa classe, basta che sappia fare le cose giuste.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def area(self):
        """Ogni forma deve dire come si calcola la sua area."""

    def descrivi(self):
        return self.nome + " ha area " + str(self.area())

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza
```

```

class Rombo:                                # NON eredita da Forma
    def __init__(self, d1, d2):
        self.nome = "Rombo"
        self.d1 = d1
        self.d2 = d2

    def area(self):
        return self.d1 * self.d2 / 2

for f in [Rettangolo(3, 4), Rombo(6, 8)]:
    print(f.nome, "ha area", f.area())

print("il rombo e' una Forma?", isinstance(Rombo(6, 8), Forma))
Rombo(6, 8).descrizi()

```

Che cosa compare quando lo esegui (e' voluto!)

```

Rettangolo ha area 12
Rombo ha area 24.0
il rombo e' una Forma? False
AttributeError: 'Rombo' object has no attribute 'descrizi'

```

Perché: il ciclo funziona anche con il rombo, che non è una `Forma`: a Python interessa che l'oggetto sappia fare `area()`, non da quale classe deriva. Questa idea si chiama *duck typing*. L'ultima riga però fallisce: il rombo non ha ereditato `descrizi()`, perché non è imparentato con nessuno.

Prova a cambiare: ecco perché, pur potendo farne a meno, le classi astratte convengono: documentano che cosa serve implementare, segnalano subito le dimenticanze (prova 48) e regalano i metodi già pronti (prova 49).

6.5 Quanto astrarre? Il costo dell'astrazione

L'astrazione non è gratis. Ogni livello che aggiungi è un file in più da aprire per capire cosa succede, e un'ipotesi in più su come evolverà il programma — ipotesi che spesso si rivela sbagliata.

Tre segnali che **è ora di astrarre**:

- Hai **almeno due o tre** classi che condividono lo stesso schema di metodi.
- Ti trovi a scrivere `if` sul tipo di un oggetto in più punti del programma.
- Sai per certo che ne arriveranno altre dello stesso tipo (altre sorgenti dati, altri formati di export, altri canali di notifica).

Tre segnali che **stai astraendo troppo**:

- La classe astratta ha **una sola** implementazione, e nessuna seconda è in vista.
- Per capire cosa fa una chiamata devi risalire tre livelli di gerarchia.

- Stai progettando per requisiti che nessuno ti ha dato, "nel caso servisse".

ATTENZIONE

La regola pratica più usata è «**la terza volta**»: la prima volta scrivi il codice; la seconda volta noti la somiglianza ma resisti; la terza volta astrai, perché ora hai tre esempi reali e sai davvero qual è la parte comune. Astrarre al primo colpo significa indovinare, e di solito si indovina male.

PROVA TU · 51 La lunga catena di if e i suoi guai

Ecco come si scriverebbe la stessa cosa senza classi, e perché conviene evitarlo.

```
def area(forma):
    if forma["tipo"] == "rettangolo":
        return forma["base"] * forma["altezza"]
    elif forma["tipo"] == "cerchio":
        return 3.14159 * forma["raggio"] ** 2

forme = [
    {"tipo": "rettangolo", "base": 3, "altezza": 4},
    {"tipo": "cerchio", "raggio": 5},
    {"tipo": "triangolo", "base": 3, "altezza": 4},    # tipo nuovo, non previsto
]

for f in forme:
    print(f["tipo"], "->", area(f))
```

Che cosa compare quando lo esegui

```
rettangolo -> 12
cerchio -> 78.53975
triangolo -> None
```

Perché: il triangolo restituisce `None`, cioè «niente», senza nessun errore: la funzione arriva in fondo senza trovare un ramo adatto. E se il programma avesse anche `perimetro()` e `disegna()` scritte allo stesso modo, per aggiungere il triangolo dovresti ricordarti di modificare *tutte* queste funzioni.

Prova a cambiare: confronta con la prova 47, dove per aggiungere una forma bastava scrivere una classe nuova senza toccare niente, e con la prova 48, dove dimenticare un metodo era proprio impossibile. **Da ricordare:** una lunga catena di `if` sul *tipo* di qualcosa è quasi sempre il segnale che servirebbero delle classi.

Esercizio 18 Forme astratte



Completa la gerarchia delle forme:

1. Scrivi `Forma` come classe astratta con `area()` e `perimetro()` astratti, e `descrivi()` concreto.

2. Implementa `Rettangolo`, `Cerchio`, `Triangolo` e `Quadrato`. Per il quadrato, riusa `Rettangolo` invece di riscrivere le formule.
3. Verifica che `Forma("x")` sollevi `TypeError`.
4. Scrivi una classe `Pentagono` dimenticando di implementare `perimetro()`, e osserva l'errore. Poi correggila.
5. Scrivi la funzione `riepilogo(forme)` che stampa tutte le forme ordinate per area decrescente e l'area totale. Non deve contenere nessun `isinstance`.

Suggerimento per il punto 5: `sorted(forme, key=lambda f: f.area(), reverse=True)`.

Esercizio 19 *Esportatori polimorfici*

...

Un caso che incontrerai davvero come data engineer: esportare gli stessi dati in formati diversi.

1. Definisci una classe astratta `Esportatore` con un metodo astratto `esporta(dati)`, dove `dati` è una lista di dizionari, e un metodo concreto `esporta_su_file(dati, percorso)` che scrive su file il risultato di `esporta()`.
2. Implementa `EsportatoreCSV` (intestazione dalle chiavi del primo dizionario, valori separati da virgola), `EsportatoreJSON` (usa `json.dumps` con `indent=2`) ed `EsportatoreTesto` (una riga `chiave: valore` per campo, righe vuote fra i record).
3. Scrivi una funzione `esporta_tutto(dati, esportatori)` che li usa tutti in un ciclo.
4. Aggiungi `EsportatoreMarkdown` che produce una tabella. **Quante righe del codice già scritto hai dovuto modificare?**

Il punto 4 è la vera lezione dell'esercizio: annota la risposta.

Esercizio 20 *Eliminare la catena di if*

...

Il codice seguente calcola il prezzo di una spedizione ed è destinato a peggiorare a ogni nuovo corriere.

```
def costo_spedizione(tipo, peso, distanza):
    if tipo == "standard":
        return 5 + peso * 0.5
    elif tipo == "espresso":
        return 12 + peso * 0.8 + distanza * 0.1
    elif tipo == "internazionale":
        base = 25 + peso * 1.2
        return base * 1.22 if distanza > 1000 else base
    else:
        raise ValueError("Tipo sconosciuto")
```

1. Rifattorizza con una classe astratta `Spedizione` e tre sottoclassi. Il metodo astratto sia `costo(peso, distanza)`.
2. Aggiungi alla classe astratta un metodo concreto `preventivo(peso, distanza)` che restituisce una stringa formattata, uguale per tutti i corrieri.

3. Aggiungi un corriere `SpedizioneDrone`, valido solo entro 50 km: oltre quella distanza `costo()` deve sollevare `ValueError`.
4. Scrivi `miglior_offerta(peso, distanza, spedizioni)` che restituisce la spedizione più economica fra quelle *disponibili*, ignorando quelle che sollevano `ValueError`.

Domanda: il punto 3 viola il principio di sostituzione visto nel capitolo 5? Argomenta la risposta — non è banale.

RIEPILOGO DEL CAPITOLO

- **Generalizzare** significa risalire dal particolare al concetto comune; **astrarre** significa descrivere cosa serve senza dire come si fa.
- Il **polimorfismo** permette di trattare oggetti diversi con lo stesso codice: niente `if` sul tipo.
- Con `ABC` e `@abstractmethod` definisci un contratto: la classe non è istanziabile e le sottoclassi sono obbligate a implementare i metodi.
- Una classe astratta unisce **metodi astratti** (il contratto) e **metodi concreti** (codice condiviso).
- Il **duck typing** rende le classi astratte facoltative in Python: servono per documentare, fallire presto e condividere codice.
- Astrai alla **terza** occorrenza, non alla prima: ogni astrazione ha un costo di leggibilità.

Progetto finale: il tuo primo componente da Data Engineer

IN QUESTO CAPITOLO

- Costruirai passo per passo un componente riutilizzabile, usando *tutto* quello che hai imparato.
- Vedrai come una buona astrazione rende il codice cliente immune ai cambiamenti.
- Scriverai i test del componente e capirai dove questo codice ti tornerà utile nel resto del corso.

7.1 Il requisito

Ecco il compito, formulato come te lo darebbero al lavoro:

REQUISITO

«Ci servono i dati da fonti diverse: alcuni arrivano in CSV, altri in JSON, presto ne arriveranno da un'API REST e da MongoDB. Gli script di analisi devono poterli leggere **senza sapere da dove vengono**, e non devono essere riscritti ogni volta che aggiungiamo una fonte.»

Leggi bene l'ultima frase, perché è quella che decide la progettazione. Il requisito non è "leggere un CSV": è «*il codice di analisi non deve cambiare quando cambia la sorgente*». Questa è, parola per parola, la definizione di astrazione del capitolo 6.

7.2 Passo 1: progettare l'interfaccia

Prima del codice, le quattro domande del paragrafo 4.5.

```
# 1. RESPONSABILITÀ
#   Fornire righe di dati in forma uniforme (lista di dizionari),
#   nascondendo da dove arrivano.
#
# 2. OPERAZIONI (interfaccia pubblica, uguale per OGNI sorgente)
#   leggi()           -> lista di dizionari
#   conta_righe()     -> int
#   colonne()         -> lista dei nomi di campo
#   anteprima(n=5)    -> le prime n righe
#   filtra(campo, val) -> le righe che soddisfano il criterio
#   ricarica()        -> rilegge la sorgente ignorando la cache
#
# 3. DATI INTERNI
#   _righe : cache dei dati già letti (None finché non si legge)
#   nome   : etichetta leggibile della sorgente (pubblico)
```

```
#
# 4. INVARIANTI
# - leggi() restituisce SEMPRE una lista di dizionari, anche se vuota
# - La sorgente viene letta una volta sola, poi si usa la cache
# - il chiamante non può alterare la cache interna
```

Nota una scelta importante: **il formato di uscita è deciso qui, non dalle singole sorgenti**. Tutte devono restituire una lista di dizionari, costi quel che costi. È proprio questa uniformità imposta che permetterà al codice cliente di ignorare l'origine dei dati.

7.3 Passo 2: la classe astratta

```
# file: sorgenti.py
from abc import ABC, abstractmethod
import csv
import json

class DataSource(ABC):
    """Sorgente di dati tabellari. Le sottoclassi implementano solo _carica()."""

    def __init__(self, nome):
        self.nome = nome
        self._righe = None          # cache: None = non ancora Letto

    # ----- il contratto che ogni sottoclasse deve firmare -----

    @abstractmethod
    def _carica(self):
        """Legge la sorgente e restituisce una lista di dizionari."""

    # ----- interfaccia pubblica, uguale per tutte le sorgenti -----

    def leggi(self):
        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)      # copia: la cache resta protetta

    def ricarica(self):
        self._righe = None
        return self.leggi()

    def conta_righe(self):
        return len(self.leggi())

    def colonne(self):
        righe = self.leggi()
        return list(righe[0].keys()) if righe else []

    def anteprima(self, n=5):
        return self.leggi()[:n]
```

```

def filtra(self, campo, valore):
    return [r for r in self.leggi() if r.get(campo) == valore]

def __len__(self):
    return self.conta_righe()

def __str__(self):
    return f"type(self).__name__('{self.nome}'): {self.conta_righe()} righe"

```

Guarda cosa è successo qui, perché è il punto in cui tutti i capitoli si incontrano. La classe astratta chiede alle sottoclassi **una sola cosa**: sapersi caricare. Tutto il resto — conteggio, colonne, anteprima, filtro, cache, stampa — è scritto una volta sola e vale per ogni sorgente presente e futura. Il metodo astratto ha l'underscore perché è un dettaglio interno: chi usa la classe chiama `leggi()`, non `_carica()`.

NOTA

Questa struttura — una classe base che definisce lo scheletro di un'operazione e delega alle sottoclassi solo i passi variabili — è un **design pattern** con un nome preciso: *Template Method*. Lo incontrerai nella lezione sui design pattern; per ora ti basti sapere che l'hai appena scritto da solo, partendo dai requisiti.

PROVA TU · 52 Lo scheletro, e il controllo che funzioni

Prima ancora di scrivere una sorgente vera, verifichiamo che la classe generica faccia il suo mestiere di guardiano.

```

from abc import ABC, abstractmethod

class DataSource(ABC):
    def __init__(self, nome):
        self.nome = nome
        self._righe = None

    @abstractmethod
    def _carica(self):
        """Ogni sorgente deve dire come si leggono i suoi dati."""

    def leggi(self):
        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)

    def conta_righe(self):
        return len(self.leggi())

class Incompleta(DataSource):
    pass # si e' dimenticata di scrivere _carica()

try:

```

```

        DataSource("prova")
except TypeError as errore:
    print("giusto:", errore)

try:
    Incompleta("prova")
except TypeError as errore:
    print("giusto:", errore)

```

Che cosa compare quando lo esegui

```

giusto: Can't instantiate abstract class DataSource without an implementation for
abstract method '_carica'
giusto: Can't instantiate abstract class Incompleta without an implementation for
abstract method '_carica'

```

Perché: non si può creare una «sorgente dati generica», perché non avrebbe senso: da dove leggerebbe? E non si può creare nemmeno una sorgente che si è dimenticata di dire come si legge. Il contratto è garantito prima ancora che esista una sola sorgente vera.

Prova a cambiare: nota il trattino basso davanti a `_carica`: significa che è un dettaglio interno. Chi userà le tue sorgenti chiamerà `leggi()`, mai `_carica()`.

7.4 Passo 3: le implementazioni concrete

Ogni sorgente ora costa pochissime righe.

```

class ListSource(DataSource):
    """Dati già in memoria. Utile soprattutto nei test."""

    def __init__(self, righe, nome="memoria"):
        super().__init__(nome)
        self._dati = list(righe)

    def _carica(self):
        return [dict(r) for r in self._dati]

class CsvSource(DataSource):
    """Dati da un file CSV con intestazione."""

    def __init__(self, percorso, delimitatore=","):
        super().__init__(nome=percorso)
        self.percorso = percorso
        self.delimitatore = delimitatore

    def _carica(self):
        with open(self.percorso, newline="", encoding="utf-8") as f:
            lettore = csv.DictReader(f, delimiter=self.delimitatore)
            return [dict(riga) for riga in lettore]

```

```

class JsonSource(DataSource):
    """Dati da un file JSON contenente una lista di oggetti."""

    def __init__(self, percorso):
        super().__init__(nome=percorso)
        self.percorso = percorso

    def _carica(self):
        with open(self.percorso, encoding="utf-8") as f:
            dati = json.load(f)
            if isinstance(dati, dict):           # un solo oggetto -> lista di uno
                dati = [dati]
        return dati

```

Tre sorgenti completamente diverse, quattro o cinque righe utili ciascuna. Nessuna di loro sa cosa siano `filtra()`, la cache o l'anteprima: le hanno ereditate.

PROVA TU · 53 Cinque righe che ne sbloccano sette

Adesso scriviamo la sorgente più semplice possibile, quella che tiene i dati in memoria, e proviamo **tutta** l'interfaccia che eredita.

```

from abc import ABC, abstractmethod

class DataSource(ABC):
    def __init__(self, nome):
        self.nome = nome
        self._righe = None

    @abstractmethod
    def _carica(self):
        """Ogni sorgente deve dire come si leggono i suoi dati."""

    def leggi(self):
        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)

    def conta_righe(self):
        return len(self.leggi())

    def colonne(self):
        righe = self.leggi()
        if len(righe) == 0:
            return []
        return list(righe[0].keys())

    def anteprima(self, n=5):
        return self.leggi()[:n]

    def filtra(self, campo, valore):
        return [r for r in self.leggi() if r.get(campo) == valore]

```

```

def __len__(self):
    return self.conta_righe()

def __str__(self):
    return type(self).__name__ + " '" + self.nome + "': " + str(self.conta_righe())
+ " righe"

class ListSource(DataSource):
    # ecco le uniche righe nuove
    def __init__(self, righe, nome="memoria"):
        super().__init__(nome)
        self._dati = list(righe)

    def _carica(self):
        return [dict(r) for r in self._dati]

righe = [
    {"id": "1", "citta": "Roma"},
    {"id": "2", "citta": "Milano"},
    {"id": "3", "citta": "Roma"},
]
s = ListSource(righe)

print(s)
print("quante righe:", len(s))
print("colonne:", s.colonne())
print("prime due:", s.anteprima(2))
print("solo Roma:", s.filtra("citta", "Roma"))
print("sorgente vuota, colonne:", ListSource([]).colonne())

s.leggi().append({"id": "999", "citta": "Napoli"})
print("dopo aver provato a modificare dall'esterno:", len(s))

```

Che cosa compare quando lo esegui

```

ListSource 'memoria': 3 righe
quante righe: 3
colonne: ['id', 'citta']
prime due: [{'id': '1', 'citta': 'Roma'}, {'id': '2', 'citta': 'Milano'}]
solo Roma: [{'id': '1', 'citta': 'Roma'}, {'id': '3', 'citta': 'Roma'}]
sorgente vuota, colonne: []
dopo aver provato a modificare dall'esterno: 3

```

Perché: la classe `ListSource` è lunga cinque righe e sa fare sette cose: contare, elencare le colonne, mostrare un'anteprima, filtrare, stamparsi e rispondere a `len()`. Tutto il resto arriva dalla classe generica, che non sa nulla di dove stiano i dati. E l'ultima riga conferma che `leggi()` restituisce una copia: modificarla dall'esterno non tocca la sorgente.

Prova a cambiare: prova ad aggiungere una riga a `righe` prima di creare la sorgente, e vedrai i conteggi cambiare di conseguenza.

PROVA TU · 54 Guardiamo la memoria di lavoro mentre lavora

Mettiamo una spia dentro il metodo che legge davvero i dati, per vedere quante volte viene eseguito.

```
from abc import ABC, abstractmethod

class DataSource(ABC):
    def __init__(self, nome):
        self.nome = nome
        self._righe = None

    @abstractmethod
    def _carica(self):
        """Ogni sorgente deve dire come si leggono i suoi dati."""

    def leggi(self):
        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)

    def ricarica(self):
        self._righe = None
        return self.leggi()

    def conta_righe(self):
        return len(self.leggi())

class ListSource(DataSource):
    def __init__(self, righe, nome="memoria"):
        super().__init__(nome)
        self._dati = list(righe)

    def _carica(self):
        print("    ---> sto leggendo la sorgente per davvero")
        return [dict(r) for r in self._dati]

s = ListSource([{"id": "1"}, {"id": "2"}])

print("prima chiamata a leggi():")
s.leggi()
print("seconda chiamata a leggi():")
s.leggi()
print("chiamata a conta_righe():")
s.conta_righe()
print("chiamata a ricarica():")
s.ricarica()
```

Che cosa compare quando lo esegui

```
prima chiamata a leggi():
    ---> sto leggendo la sorgente per davvero
seconda chiamata a leggi():
chiamata a conta_righe():
chiamata a ricarica():
    ---> sto leggendo la sorgente per davvero
```

Perché: la spia compare solo due volte: alla prima lettura e dopo `ricarica()`. Le chiamate in mezzo riusano i dati già letti, tenuti da parte nel campo `_righe`. Con dati in memoria non si nota, ma con un file da un milione di righe o con una richiesta via internet questa differenza vale secondi o minuti.

Prova a cambiare: nota *dove* è scritta questa furbizia: nella classe generica. Ogni sorgente che scriverai in futuro la otterrà gratis, senza doversene occupare.

7.5 Passo 4: il codice cliente che non cambia mai

Ecco il codice di analisi. Il punto da osservare è che **non nomina mai nessuna classe concreta**.

```
# file: analisi.py
from sorgenti import CsvSource, JsonSource, ListSource

def report(sorgente):
    """Funziona con QUALSIASI DataSource, presente o futuro."""
    print(sorgente)
    print(" colonne:", ", ".join(sorgente.colonne()))
    for riga in sorgente.anteprima(3):
        print(" ", riga)
    print()

def confronta(sorgenti):
    for s in sorgenti:
        print(f"{s.nome:<30} {s.conta_righe():>6} righe")

sorgenti = [
    CsvSource("vendite_2026.csv"),
    JsonSource("clienti.json"),
    ListSource([{"id": 1, "nome": "test"}], nome="dati di prova"),
]

for s in sorgenti:
    report(s)

confronta(sorgenti)
```

```
CsvSource('vendite_2026.csv'): 1240 righe
  colonne: data, prodotto, quantita, importo
  {'data': '2026-01-03', 'prodotto': 'A-100', 'quantita': '2', 'importo': '49.90'}
  ...

JsonSource('clienti.json'): 87 righe
  colonne: id, ragione_sociale, citta
  ...
```

Adesso il test finale della progettazione. Domani arriva il requisito «i dati vengono da un'API REST». Quanto codice devi modificare?

```
import requests

class ApiSource(DataSource):
    """Dati da un endpoint REST che restituisce JSON."""

    def __init__(self, url, timeout=10):
        super().__init__(nome=url)
        self.url = url
        self.timeout = timeout

    def _carica(self):
        risposta = requests.get(self.url, timeout=self.timeout)
        risposta.raise_for_status()
        dati = risposta.json()
        return dati if isinstance(dati, list) else [dati]
```

Zero righe modificate. Undici righe aggiunte, e `report()`, `confronta()` e ogni altro script già scritto funzionano con la nuova sorgente senza saperlo. Questo è il ritorno concreto di sei capitoli di teoria: non un codice più elegante, ma un codice che *non devi riscrivere* quando cambiano i requisiti.

PROVA TU · 55 Una sorgente che legge un file vero

Adesso una sorgente che legge un file CSV. Il codice crea prima il file, così puoi eseguirlo anche tu senza avere niente di pronto.

```
from abc import ABC, abstractmethod
import csv

class DataSource(ABC):
    def __init__(self, nome):
        self.nome = nome
        self._righe = None

    @abstractmethod
    def _carica(self):
        """Ogni sorgente deve dire come si leggono i suoi dati."""

    def leggi(self):
```

```

        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)

    def conta_righe(self):
        return len(self.leggi())

    def colonne(self):
        righe = self.leggi()
        if len(righe) == 0:
            return []
        return list(righe[0].keys())

    def filtra(self, campo, valore):
        return [r for r in self.leggi() if r.get(campo) == valore]

class CsvSource(DataSource):
    def __init__(self, percorso):
        super().__init__(percorso)
        self.percorso = percorso

    def _carica(self):
        with open(self.percorso, newline="", encoding="utf-8") as f:
            return [dict(riga) for riga in csv.DictReader(f)]

# creiamo un file di prova, così l'esempio funziona subito
with open("vendite.csv", "w", encoding="utf-8") as f:
    f.write("data,prodotto,importo\n")
    f.write("2026-01-03,A-100,49.90\n")
    f.write("2026-01-04,B-200,15.00\n")
    f.write("2026-01-05,A-100,49.90\n")

s = CsvSource("vendite.csv")

print("righe lette:", s.conta_righe())
print("colonne:", s.colonne())
print("solo A-100:", len(s.filtra("prodotto", "A-100")))
print("prima riga:", s.leggi()[0])

```

Che cosa compare quando lo esegui

```

righe lette: 3
colonne: ['data', 'prodotto', 'importo']
solo A-100: 2
prima riga: {'data': '2026-01-03', 'prodotto': 'A-100', 'importo': '49.90'}

```

Perché: i metodi che usi sono **gli stessi** della prova 53, anche se la sorgente è completamente diversa: prima i dati erano in memoria, adesso arrivano da un file su disco. La classe `CsvSource` ha dovuto scrivere solo come si legge un CSV.

Prova a cambiare: guarda l'ultima riga stampata: l'importo è `'49.90'` fra virgolette, cioè **testo**, non un numero. Un file CSV non conserva i tipi: tutto quello che leggi da lì è testo, e convertirlo è un lavoro

successivo. È una delle prime cose che scoprirai facendo davvero questo mestiere.

7.6 Passo 5: i test del componente

Come nel capitolo 4, i test passano solo dall'interfaccia pubblica. Nota il trucco di `ListSource`: avere una sorgente in memoria rende testabile tutta la logica ereditata senza toccare il disco.

```
# file: test_sorgenti.py
import pytest
from sorgenti import DataSource, ListSource, CsvSource

RIGHE = [
    {"id": "1", "citta": "Roma", "importo": "100"},
    {"id": "2", "citta": "Milano", "importo": "250"},
    {"id": "3", "citta": "Roma", "importo": "70"},
]

def test_datasource_non_istanziabile():
    with pytest.raises(TypeError):
        DataSource("x")

def test_sottoclasse_senza_carica_non_istanziabile():
    class Incompleta(DataSource):
        pass
    with pytest.raises(TypeError):
        Incompleta("x")

def test_conta_righe():
    s = ListSource(RIGHE)
    assert s.conta_righe() == 3
    assert len(s) == 3

def test_colonne():
    s = ListSource(RIGHE)
    assert s.colonne() == ["id", "citta", "importo"]

def test_sorgente_vuota():
    s = ListSource([])
    assert s.conta_righe() == 0
    assert s.colonne() == []
    assert s.anteprima() == []

def test_filtra():
    s = ListSource(RIGHE)
    assert len(s.filtra("citta", "Roma")) == 2
    assert s.filtra("citta", "Napoli") == []
```

```
def test_anteprima_non_supera_il_totale():
    s = ListSource(RIGHE)
    assert len(s.anteprima(10)) == 3

def test_leggi_non_espone_la_cache():
    s = ListSource(RIGHE)
    s.leggi().append({"id": "999"})
    assert s.conta_righe() == 3

def test_csv_source(tmp_path):
    f = tmp_path / "dati.csv"
    f.write_text("id,citta\n1,Roma\n2,Milano\n", encoding="utf-8")

    s = CsvSource(str(f))
    assert s.conta_righe() == 2
    assert s.colonne() == ["id", "citta"]
    assert s.filtra("citta", "Roma")[0]["id"] == "1"
```

```
$ pytest -v
===== 9 passed in 0.06s =====
```

Due test meritano attenzione. `test_sottoclasse_senza_carica_non_istanziabile` verifica il *contratto* dell'astrazione, non un calcolo: sta controllando che la classe astratta faccia il suo mestiere. E `tmp_path` è una comodità di pytest che ti dà una cartella temporanea pulita per ogni test, così i test sui file non lasciano sporcizia in giro.

PROVA TU · 56 La prova finale: arriva un formato nuovo

Ecco il momento della verità. Abbiamo del codice che usa le sorgenti; arriva la richiesta di leggere anche i file JSON. Quanto codice va modificato?

```
from abc import ABC, abstractmethod
import csv
import json

class DataSource(ABC):
    def __init__(self, nome):
        self.nome = nome
        self._righe = None

    @abstractmethod
    def _carica(self):
        """Ogni sorgente deve dire come si leggono i suoi dati."""

    def leggi(self):
        if self._righe is None:
            self._righe = self._carica()
        return list(self._righe)
```

```

def conta_righe(self):
    return len(self.leggi())

def colonne(self):
    righe = self.leggi()
    if len(righe) == 0:
        return []
    return list(righe[0].keys())

def anteprima(self, n=5):
    return self.leggi()[0:n]

class CsvSource(DataSource):
    def __init__(self, percorso):
        super().__init__(percorso)
        self.percorso = percorso

    def _carica(self):
        with open(self.percorso, newline="", encoding="utf-8") as f:
            return [dict(riga) for riga in csv.DictReader(f)]

class JsonSource(DataSource):
    # <-- LE UNICHE RIGHE NUOVE
    def __init__(self, percorso):
        super().__init__(percorso)
        self.percorso = percorso

    def _carica(self):
        with open(self.percorso, encoding="utf-8") as f:
            dati = json.load(f)
            if isinstance(dati, dict):
                return [dati]
            return dati

def report(sorgente):
    # questo codice non cambia mai
    print(sorgente.nome, "->", sorgente.conta_righe(), "righe")
    print("  colonne:", sorgente.colonne())
    print("  prima riga:", sorgente.anteprima(1))

# prepariamo i due file di prova
with open("vendite.csv", "w", encoding="utf-8") as f:
    f.write("data,prodotto\n")
    f.write("2026-01-03,A-100\n")
    f.write("2026-01-04,B-200\n")

with open("clienti.json", "w", encoding="utf-8") as f:
    json.dump([{"id": 1, "citta": "Roma"}, {"id": 2, "citta": "Bari"}], f)

for s in [CsvSource("vendite.csv"), JsonSource("clienti.json")]:
    report(s)

```

Che cosa compare quando lo esegui

```
vendite.csv -> 2 righe
  colonne: ['data', 'prodotto']
  prima riga: [{'data': '2026-01-03', 'prodotto': 'A-100'}]
clienti.json -> 2 righe
  colonne: ['id', 'citta']
  prima riga: [{'id': 1, 'citta': 'Roma'}]
```

Perché: la funzione `report` non nomina mai `CsvSource` né `JsonSource` : sa soltanto che riceve qualcosa che sa fare `conta_righe()` , `colonne()` e `anteprima()` . Per aggiungere il formato JSON hai scritto una classe nuova di otto righe e **non hai modificato nulla di quello che c'era già**.

Prova a cambiare: è questa la conclusione di tutto il percorso. Non un codice più elegante: un codice che *non devi riscrivere* quando cambiano le richieste. Nel tuo lavoro le sorgenti dati cambieranno di continuo, ed è il motivo per cui queste sette lezioni non erano teoria.

7.7 Dove andrà a finire questo codice

Questo componente non è un esercizio da buttare: è il filo conduttore delle prossime lezioni del corso.

Lezione	Cosa ci farai
Git e versionamento	Metterai <code>sorgenti.py</code> e <code>test_sorgenti.py</code> sotto controllo di versione, con un branch per ogni nuova sorgente.
Unit testing e TDD	Riscriverai <code>ApiSource</code> partendo dai test: prima il test, poi il codice.
Design pattern	Riconoscerai il <i>Template Method</i> che hai già scritto e aggiungerai una <i>Factory</i> che sceglie la sorgente dall'estensione del file.
API REST e JSON	Completerai <code>ApiSource</code> con <code>requests</code> , gestendo autenticazione, paginazione ed errori HTTP.
MongoDB	Aggiungerai <code>MongoSource</code> : le righe sono già dizionari, quindi l'integrazione sarà quasi immediata.
Processi ETL	<code>DataSource</code> è la E di <i>Extract</i> . Aggiungerai le classi <code>Trasformazione</code> e <code>Destinazione</code> per completare la pipeline.

IL SENSO DI TUTTO QUESTO

Un data engineer passa gran parte del tempo a spostare dati fra sistemi che cambiano di continuo. Il valore della programmazione a oggetti, nel tuo mestiere, non è accademico: è **isolare il codice che elabora i dati dal codice che sa dove i dati stanno**. Quando cambia il database, cambia una classe. Quando cambia l'API, cambia una classe. Tutto il resto continua a funzionare.

RIEPILOGO DEL CAPITOLO

- Si parte sempre dal **requisito** e dall'interfaccia, mai dalla prima classe concreta.
- La classe astratta impone **un formato di uscita uniforme**: è ciò che rende le sorgenti intercambiabili.
- Concentrare la logica comune nella base fa sì che ogni nuova sorgente costi **poche righe**.
- Il codice cliente non nomina mai le classi concrete, quindi non cambia quando ne aggiungi altre.
- Una sorgente "in memoria" (`ListSource`) rende testabile tutto il componente senza dipendere da file o rete.

Soluzioni commentate

Usa queste soluzioni **dopo** aver provato, non prima. Quando la tua soluzione è diversa da quella proposta non significa che sia sbagliata: confronta il comportamento, non le righe. Dove l'esercizio è discorsivo, qui trovi il ragionamento atteso.

Esercizio 1 — Il codice che scricchiola

1. Tre problemi.

- `ultima_lettura(s2)` solleva `IndexError`: la lista `letture` è vuota e `[-1]` non esiste. Nessuno protegge questo caso.
- `registra(s1, "caldo")` funziona senza errori: la stringa finisce fra le letture e romperà tutto più tardi, quando qualcuno calcolerà una media.
- `s3 = {"id": "X", "letture": []}`, senza la chiave `tipo`, viene accettato: l'errore comparirà solo nel punto del programma che usa `s["tipo"]`.
- In più: `s1["letture"] = None` o una chiave scritta male (`"letura"`) passano inosservate.

2. La soglia fissa. Il valore 30 ha senso per una temperatura in gradi, non per una pressione in bar: `in_allarme(s2)` darebbe risposte prive di significato. La soglia è una caratteristica *del singolo sensore*, quindi deve essere un dato dell'oggetto, non una costante scritta dentro una funzione.

3. Struttura della futura classe. Campi: `id`, `tipo`, `soglia_allarme`, `_letture`. Metodi: `registra(valore)`, `ultima_lettura()`, `media_letture()`, `in_allarme()`, `__str__`.

Esercizio 2 — Completare la classe Studente

```
class Studente:
    def __init__(self, nome, email, matricola=None):
        self.nome = nome
        self.email = email
        self.matricola = matricola
        self.voti = []

    def aggiungi_voto(self, voto):
        self.voti.append(voto)

    def numero_esami(self):
        return len(self.voti)

    def voto_massimo(self):
        return max(self.voti) if self.voti else None

    def media(self):
        return sum(self.voti) / len(self.voti) if self.voti else 0
```

```
def __str__(self):
    return (f"{self.matricola} - {self.nome} - "
            f"{self.numero_esami()} esami - media {self.media():.1f}")
```

`max()` su una lista vuota solleva `ValueError`: per questo il controllo `if self.voti` è indispensabile. Nota che in Python una lista vuota è *falsa*, quindi `if self.voti` equivale a `if len(self.voti) > 0` ed è più idiomatico.

Esercizio 3 — La classe Misurazione

```
class Misurazione:
    def __init__(self, sensore_id, valore, timestamp, unita="C"):
        self.sensore_id = sensore_id
        self.valore = valore
        self.timestamp = timestamp
        self.unita = unita

    def e_valida(self):
        return -50 <= self.valore <= 150

    def in_fahrenheit(self):
        if self.unita != "C":
            return None
        return self.valore * 9 / 5 + 32

    def __str__(self):
        return f"[{self.timestamp}] {self.sensore_id}: {self.valore} {self.unita}"

letture = [
    Misurazione("T-01", 21.5, "2026-03-01 10:00"),
    Misurazione("T-01", 999.0, "2026-03-01 10:05"),
    Misurazione("P-07", 3.2, "2026-03-01 10:10", unita="bar"),
]
for m in letture:
    print(m, "-> valida" if m.e_valida() else "-> SCARTATA")
```

I parametri con valore di default (`unita="C"`) devono stare **dopo** quelli obbligatori, altrimenti Python solleva un `SyntaxError`.

Esercizio 4 — Il carrello

```
class Carrello:
    IVA = 0.22

    def __init__(self):
        self.articoli = []           # nel costruttore: una lista PER OGGETTO

    def aggiungi(self, nome, prezzo, quantita=1):
        self.articoli.append({"nome": nome, "prezzo": prezzo, "quantita": quantita})
```

```

def totale_netto(self):
    return sum(a["prezzo"] * a["quantita"] for a in self.articoli)

def totale_lordo(self):
    return self.totale_netto() * (1 + self.IVA)

def svuota(self):
    self.articoli = []

def __str__(self):
    return f"Carrello: {len(self.articoli)} articoli, {self.totale_lordo():.2f} EUR"

c1 = Carrello(); c2 = Carrello()
c1.aggiungi("Mouse", 25.0, 2)
print(c1)          # Carrello: 1 articoli, 61.00 EUR
print(c2)          # Carrello: 0 articoli, 0.00 EUR <- indipendenti

```

Se avessi scritto `articoli = []` nel corpo della classe, `c2` mostrerebbe anche l'articolo di `c1`. La verifica finale richiesta dall'esercizio serve esattamente a smascherare quell'errore.

Esercizio 5 — Da procedurale a oggetti

```

class Sensore:
    TIPI_AMMESSI = ("temperatura", "pressione", "umidita")

    def __init__(self, id_sensore, tipo, soglia_allarme):
        if tipo not in self.TIPI_AMMESSI:
            raise ValueError(f"Tipo non ammesso: {tipo}")
        self.id_sensore = id_sensore
        self.tipo = tipo
        self.soglia_allarme = soglia_allarme      # soglia PER SENSORE
        self.letture = []

    def registra(self, valore):
        self.letture.append(valore)

    def ultima_lettura(self):
        return self.letture[-1] if self.letture else None

    def media_letture(self):
        return sum(self.letture) / len(self.letture) if self.letture else 0

    def in_allarme(self):
        ultima = self.ultima_lettura()
        return ultima is not None and ultima > self.soglia_allarme

    def __str__(self):
        stato = "ALLARME" if self.in_allarme() else "ok"
        return f"{self.id_sensore} ({self.tipo}): {self.ultima_lettura()} [{stato}]"

```

```
sensori = [  
    Sensore("T-01", "temperatura", 30),  
    Sensore("P-07", "pressione", 5),  
    Sensore("U-02", "umidita", 80),  
]  
sensori[0].registra(35); sensori[1].registra(3.2); sensori[2].registra(91)  
  
for s in sensori:  
    if s.in_allarme():  
        print(s)
```

Confronta con il codice dell'Esercizio 1: la soglia ora è un dato dell'oggetto, il caso "nessuna lettura" è gestito in un punto solo, e il tipo viene validato alla creazione invece di essere solo sperato.

Esercizio 6 — Prevedi l'output

- (a) 99 — `p3` è un alias di `p1` : modificando `p3.x` si modifica lo stesso oggetto.
- (b) 1 — `p2` è un oggetto distinto, creato da una seconda chiamata a `Punto()` .
- (c) `False` — stessi valori, ma due oggetti diversi in memoria.
- (d) `True` — `p3 = p1` ha copiato il riferimento, non l'oggetto.
- (e) `False` — senza `__eq__` , Python confronta l'identità: `==` si comporta come `is` .
- (f) 77 — `lista[0]` è `p1` : la lista contiene riferimenti, non copie.

Esercizio 7 — La trappola dell'alias

1. `comune` è un riferimento a *una sola* lista. Passandolo a tre costruttori, i campi `giocatori` delle tre squadre puntano tutti alla stessa lista: non ci sono tre liste, ce n'è una con tre etichette.

2. Le due correzioni richieste:

```
# (a) parametro opzionale, lista creata dentro il costruttore
class Squadra:
    def __init__(self, nome, giocatori=None):
        self.nome = nome
        self.giocatori = [] if giocatori is None else list(giocatori)

# (b) parametro obbligatorio, ma copiato
class Squadra:
    def __init__(self, nome, giocatori):
        self.nome = nome
        self.giocatori = list(giocatori)      # copia: nessuna condivisione
```

3. `def __init__(self, nome, giocatori=[])` non funziona per un motivo sottile e famoso: **il valore di default viene creato una sola volta, quando Python legge la definizione della funzione**, non a ogni chiamata. Quindi tutte le squadre create senza passare l'argomento condividono la *stessa identica lista*, e il bug torna identico — anzi peggiore, perché sopravvive anche fra oggetti creati in momenti lontani del programma.

```
class Squadra:
    def __init__(self, nome, giocatori=[]):      # SBAGLIATO
        self.nome = nome
        self.giocatori = giocatori

a = Squadra("A"); b = Squadra("B")
a.giocatori.append("Marco")
print(b.giocatori)          # ['Marco']
print(a.giocatori is b.giocatori)  # True
```

La regola generale: **mai usare un valore mutabile (lista, dizionario, set) come default di un parametro**. Si usa `None` e si crea l'oggetto dentro la funzione.

Esercizio 8 — Composizione: Biblioteca e Libro

```
class Libro:
    def __init__(self, titolo, autore, isbn):
        self.titolo = titolo
        self.autore = autore
        self.isbn = isbn
        self.disponibile = True

    def presta(self):
        if not self.disponibile:
            return False
```

```

        self.disponibile = False
        return True

    def restituisci(self):
        if self.disponibile:
            return False
        self.disponibile = True
        return True

    def __str__(self):
        stato = "disponibile" if self.disponibile else "in prestito"
        return f"{self.titolo} ({self.autore}) - {stato}"

class Biblioteca:
    def __init__(self, nome):
        self.nome = nome
        self.libri = []

    def aggiungi(self, libro):
        self.libri.append(libro)

    def cerca_per_autore(self, autore):
        return [l for l in self.libri if l.autore == autore]

    def disponibili(self):
        return [l for l in self.libri if l.disponibile]

    def presta_isbn(self, isbn):
        for l in self.libri:
            if l.isbn == isbn:
                return l.presta()
        return False

b = Biblioteca("Civica")
libro = Libro("Moby Dick", "Melville", "123")
b.aggiungi(libro)
b.presta_isbn("123")
print(libro.disponibile)      # False

```

Perché la variabile originale risulta non disponibile? Perché `b.libri[0]` e `libro` sono lo stesso oggetto: `aggiungi()` ha memorizzato un riferimento, non una copia. È esattamente il meccanismo del capitolo 3, e qui è un vantaggio: non esiste il rischio di due copie del libro che si contraddicono.

Esercizio 9 — Copia superficiale e profonda

3. Sì, la media dell'originale cambia. `copy.copy(corso)` crea un nuovo oggetto `Corso`, ma il suo campo `iscritti` punta alla *stessa lista* con gli *stessi* oggetti `Studente`: aggiungere un voto a `copia.iscritti[0]` significa aggiungerlo ad `anna`.

4. Con `deepcopy` l'originale non cambia: sono stati ricreati anche la lista e gli studenti dentro di essa. Verifica con `id(corso.iscritti[0]) != id(profonda.iscritti[0])`.

5. Qui la risposta è diversa: il corso originale ha ancora **due** iscritti. Iscrivere qualcuno alla copia superficiale chiama `append()` sulla lista condivisa... e infatti dipende da come è stata fatta la copia. Se `copy.copy()` ha copiato il riferimento alla lista, allora anche l'originale ne avrà tre. Prova entrambi i casi e usa `id(corso.iscritti) == id(copia.iscritti)` per stabilire con certezza se la lista è condivisa. Il punto dell'esercizio è proprio questo: **non fidarti dell'intuizione, verifica con `id()`**. Con `copy.copy()` la lista è condivisa, quindi la risposta corretta è tre; se avessi definito un `__copy__` personalizzato che copia la lista, sarebbe due.

Esercizio 10 — Il conto bancario che si difende

```
class ContoBancario:
    def __init__(self, intestatario, saldo_iniziale=0):
        self.intestatario = intestatario
        self._saldo = 0
        self._movimenti = []
        if saldo_iniziale > 0:
            self.deposita(saldo_iniziale)

    @property
    def saldo(self):
        return self._saldo

    def deposita(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da depositare deve essere positivo")
        self._saldo += importo
        self._movimenti.append(importo)
        return self._saldo

    def preleva(self, importo):
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere positivo")
        if importo > self._saldo:
            raise ValueError(f"Saldo insufficiente: disponibili {self._saldo}")
        self._saldo -= importo
        self._movimenti.append(-importo)
        return self._saldo

    def trasferisci_a(self, altro_conto, importo):
        self.preleva(importo)
        altro_conto.deposita(importo)
```

se fallisce, si ferma qui
mai altro_conto._saldo += ...

```
def estratto_conto(self):
    return list(self._movimenti)
```

Il punto chiave è `trasferisci_a()`: usa i *metodi pubblici* dell'altro oggetto, mai i suoi campi interni. Così le validazioni dell'altro conto restano attive, e se un giorno `ContoBancario` cambierà implementazione (come nel paragrafo 4.4), il trasferimento continuerà a funzionare. Nota anche l'ordine delle operazioni: prima il prelievo, che può fallire; se fallisse dopo il deposito, avresti creato denaro dal nulla.

Esercizio 11 — Property con validazione

```
class Studente:
    def __init__(self, nome, email):
        self.nome = nome          # passa dal setter -> valida subito
        self.email = email
        self._voti = []

    @property
    def nome(self):
        return self._nome

    @nome.setter
    def nome(self, valore):
        if not isinstance(valore, str) or not valore.strip():
            raise ValueError("Il nome non può essere vuoto")
        self._nome = valore.strip()

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, valore):
        if not isinstance(valore, str) or "@" not in valore:
            raise ValueError(f"Email non valida: {valore!r}")
        self._email = valore

    def aggiungi_voto(self, voto):
        if not isinstance(voto, int):
            raise TypeError("Il voto deve essere un intero")
        if not 0 <= voto <= 30:
            raise ValueError(f"Voto fuori scala: {voto}")
        self._voti.append(voto)

    def voti(self):
        return list(self._voti)

    @property
    def media(self):
        return sum(self._voti) / len(self._voti) if self._voti else 0.0

    @property
```

```
def promosso(self):
    return self.media >= 24
```

```
Studente(" ", "x")          # ValueError: Il nome non può essere vuoto
```

Il costruttore assegna `self.nome` (senza underscore) e questo attiva il setter: è il motivo per cui la validazione funziona anche alla creazione. Se avessi scritto `self._nome = nome`, avresti scavalcato il controllo — un errore frequente e difficile da notare.

Esercizio 12 — Progettare prima di scrivere

```
# 1. RESPONSABILITÀ: gestire una sequenza ordinata di brani e la posizione corrente.
# 2. OPERAZIONI: aggiungi(brano), rimuovi(titolo), durata_totale, corrente,
#               avanti(), indietro(), mescola(), __len__, __str__
# 3. DATI INTERNI: _brani (lista), _indice (int)
# 4. INVARIANTI: 0 <= _indice < len(_brani) se la playlist non è vuota;
#               _indice vale 0 (o è irrilevante) se è vuota.
```

```
import random
```

```
class Playlist:
```

```
    def __init__(self, nome):
        self.nome = nome
        self._brani = []
        self._indice = 0
```

```
    def aggiungi(self, titolo, durata):
        self._brani.append({"titolo": titolo, "durata": durata})
```

```
    def rimuovi(self, titolo):
        for i, b in enumerate(self._brani):
            if b["titolo"] == titolo:
                del self._brani[i]
                self._sistema_indice(rimosso=i)
                return True
        return False
```

```
    def _sistema_indice(self, rimosso):
        """Mantiene l'invariante sull'indice dopo una rimozione."""
        if not self._brani:
            self._indice = 0
        elif rimosso < self._indice or self._indice >= len(self._brani):
            self._indice = max(0, min(self._indice - 1, len(self._brani) - 1))
```

```
    @property
    def durata_totale(self):
        return sum(b["durata"] for b in self._brani)
```

```
    @property
    def corrente(self):
        return self._brani[self._indice] if self._brani else None
```

```

def avanti(self):
    if self._brani:
        self._indice = (self._indice + 1) % len(self._brani)
    return self.corrente

def indietro(self):
    if self._brani:
        self._indice = (self._indice - 1) % len(self._brani)
    return self.corrente

def mescola(self):
    random.shuffle(self._brani)
    self._indice = 0

def __len__(self):
    return len(self._brani)

```

La parte interessante è `_sistema_indice()`: è un metodo *privato* che esiste solo per mantenere l'invariante. Averlo isolato significa che la regola sta scritta in un punto solo, invece di essere ripetuta in ogni metodo che modifica la lista. Il modulo `%` in `avanti()` e `indietro()` fa scorrere l'indice in modo circolare senza mai uscire dai limiti.

Esercizio 13 — Scrivi i test

```

import pytest
from conto import ContoBancario

def test_conto_nuovo_ha_saldo_zero():
    assert ContoBancario("Anna").saldo == 0

def test_conto_con_saldo_iniziale():
    assert ContoBancario("Anna", 100).saldo == 100

def test_deposito_aumenta_il_saldo():
    c = ContoBancario("Anna", 100)
    c.deposita(50)
    assert c.saldo == 150

def test_prelievo_riduce_il_saldo():
    c = ContoBancario("Anna", 100)
    c.preleva(30)
    assert c.saldo == 70

def test_importo_negativo_rifiutato():
    c = ContoBancario("Anna", 100)
    with pytest.raises(ValueError):
        c.deposita(-10)

```

```

def test_prelievo_eccessivo_non_altera_il_saldo():
    c = ContoBancario("Anna", 100)
    with pytest.raises(ValueError):
        c.preleva(500)
    assert c.saldo == 100          # il vero test è QUESTA riga

def test_prelievo_esatto_azzerava_il_saldo():
    c = ContoBancario("Anna", 100)
    c.preleva(100)
    assert c.saldo == 0

def test_trasferimento_fra_conti():
    a = ContoBancario("Anna", 100)
    b = ContoBancario("Luca", 0)
    a.trasferisci_a(b, 40)
    assert a.saldo == 60
    assert b.saldo == 40

def test_saldo_non_scrivibile():
    c = ContoBancario("Anna", 100)
    with pytest.raises(AttributeError):
        c.saldo = 999

def test_estratto_conto_non_espone_la_lista_interna():
    c = ContoBancario("Anna", 100)
    c.estratto_conto().append(999999)
    assert c.saldo == 100
    assert len(c.estratto_conto()) == 1

```

Il test più importante è `test_prelievo_eccessivo_non_altera_il_saldo` : verificare che l'eccezione venga sollevata non basta, bisogna controllare che l'oggetto sia rimasto **in uno stato coerente** dopo il fallimento. Un'operazione che fallisce a metà, lasciando l'oggetto corrotto, è peggio di un'operazione che non parte affatto.

Esercizio 14 — La gerarchia dei dipendenti

```
class Apprendista(Dipendente):
    RIDUZIONE = 0.60

    def stipendio_mensile(self):
        return super().stipendio_mensile() * self.RIDUZIONE

    def scheda(self):
        return super().scheda() + " (in formazione)"

def piu_costoso(persone):
    return max(persone, key=lambda p: p.stipendio_mensile())
```

Perché funziona senza sapere i tipi? Perché `max()` chiama `p.stipendio_mensile()` su ogni elemento, e Python risolve il metodo sulla classe *reale* di ciascun oggetto. La funzione dipende solo dal fatto che tutti espongano quel metodo — cioè dall'interfaccia, non dal tipo. È il polimorfismo del capitolo 6, incontrato in anticipo.

Esercizio 15 — Overriding: estendere o sostituire

```
class NotificaEmail(Notifica):
    def __init__(self, destinatario, testo, oggetto):
        super().__init__(destinatario, testo)
        self.oggetto = oggetto

    def valida(self):
        # ESTENDE: il controllo del padre serve ancora
        super().valida()
        if "@" not in self.destinatario:
            raise ValueError(f"Email non valida: {self.destinatario}")

    def invia(self):
        # ESTENDE: riusa il formato base
        self.valida()
        return f"[EMAIL] Oggetto: {self.oggetto} | A {self.destinatario}: {self.testo}"

class NotificaSMS(Notifica):
    MAX_CARATTERI = 160

    def valida(self):
        # ESTENDE
        super().valida()
        if len(self.testo) > self.MAX_CARATTERI:
            raise ValueError(f"SMS troppo lungo: {len(self.testo)} caratteri")

    def invia(self):
        # SOSTITUISCE: formato completamente diverso
        self.valida()
        return f"[SMS a {self.destinatario}] {self.testo.upper()}"

notifiche = [
    NotificaEmail("anna@example.com", "Riunione alle 15", "Convocazione"),
```

```
        NotificaSMS("+393331234567", "Riunione alle 15"),
    ]
    for n in notifiche:
        print(n.invia())
```

Entrambe le classi *estendono* `valida()`, perché la regola del padre («il testo non è vuoto») resta valida per tutti: sostituirla significherebbe perdere un controllo. `invia()`, invece, produce un formato specifico del canale, quindi lì la sostituzione ha senso.

Esercizio 16 — "È un" oppure "ha un"?

1. **Quadrato / Rettangolo:** in geometria un quadrato è un rettangolo, ma nel codice l'ereditarietà crea guai (vedi sotto). In pratica: composizione, o due classi separate sotto una base comune `Forma`.
2. **Ordine / Cliente:** composizione. Un ordine *ha un* cliente.
3. **ContoCorrente / ContoDeposito:** ereditarietà da una base comune `Conto`. Sono varianti dello stesso concetto, usate in modo intercambiabile.
4. **Playlist / Brano:** composizione. Una playlist *contiene* brani.
5. **FileCSV / SorgenteDati:** ereditarietà: un file CSV è una sorgente dati. È il capitolo 7.
6. **Computer / ProcessorePentium:** composizione. Un computer *ha un* processore.

Il caso del quadrato. Se `Quadrato` eredita da `Rettangolo`, eredita anche `imposta_larghezza()` e `imposta_altezza()`. Ma un quadrato deve mantenere l'invariante «tutti i lati uguali»: se `imposta_larghezza(5)` cambia solo la larghezza, l'oggetto smette di essere un quadrato; se cambia entrambi i lati, allora si comporta diversamente da come un rettangolo promette. Un codice generico come `r.imposta_larghezza(5); r.imposta_altezza(4); assert r.area() == 20` funziona con i rettangoli e fallisce con i quadrati — ed è una violazione del principio di sostituzione visto nel paragrafo 5.5. Morale: **il test "è un" va applicato al comportamento, non alle definizioni da dizionario.**

Esercizio 17 — Rifattorizzare il duplicato

```
import math

class Forma:
    def __init__(self, nome):
        self.nome = nome

    def descrivi(self):
        return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"

class Rettangolo(Forma):
    def __init__(self, base, altezza):
        super().__init__("Rettangolo")
        self.base, self.altezza = base, altezza

    def area(self): return self.base * self.altezza
    def perimetro(self): return 2 * (self.base + self.altezza)
```

```

class Cerchio(Forma):
    def __init__(self, raggio):
        super().__init__("Cerchio")
        self.raggio = raggio

    def area(self): return math.pi * self.raggio ** 2
    def perimetro(self): return 2 * math.pi * self.raggio

class Triangolo(Forma):
    def __init__(self, a, b, c):
        super().__init__("Triangolo")
        self.a, self.b, self.c = a, b, c

    def area(self):
        s = self.perimetro() / 2
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))

    def perimetro(self): return self.a + self.b + self.c

```

Punto 4. `Forma.area()` non deve restituire `0` né essere lasciato fuori: deve essere dichiarato **astratto**, così una sottoclasse che se lo dimentica non può nemmeno essere istanziata. È esattamente la soluzione del paragrafo 6.3.

Esercizio 18 — Forme astratte

```

from abc import ABC, abstractmethod
import math

class Forma(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimetro(self): ...

    def descrivi(self):
        return f"{self.nome}: area {self.area():.2f}, perimetro {self.perimetro():.2f}"

class Rettangolo(Forma):
    def __init__(self, base, altezza, nome="Rettangolo"):
        super().__init__(nome)
        self.base, self.altezza = base, altezza

    def area(self): return self.base * self.altezza
    def perimetro(self): return 2 * (self.base + self.altezza)

```

```

class Quadrato(Rettangolo):
    # riusa le formule del rettangolo
    def __init__(self, lato):
        super().__init__(lato, lato, nome="Quadrato")

def riepilogo(forme):
    for f in sorted(forme, key=lambda f: f.area(), reverse=True):
        print(f.descrivi())
    print(f"Area totale: {sum(f.area() for f in forme):.2f}")

```

Il `Quadrato` qui è sicuro perché non espone metodi per modificare i lati singolarmente: senza setter, l'invariante «lati uguali» non può essere violato e il problema discusso nell'Esercizio 16 non si presenta. L'ereditarietà diventa pericolosa solo quando la sottoclasse eredita operazioni che ne rompono le regole.

Esercizio 19 — Esportatori polimorfici

```

from abc import ABC, abstractmethod
import json

class Esportatore(ABC):
    @abstractmethod
    def esporta(self, dati):
        """Restituisce la rappresentazione testuale dei dati."""

    def esporta_su_file(self, dati, percorso):
        with open(percorso, "w", encoding="utf-8") as f:
            f.write(self.esporta(dati))
        return percorso

class EsportatoreCSV(Esportatore):
    def esporta(self, dati):
        if not dati:
            return ""
        intestazione = list(dati[0].keys())
        righe = [", ".join(intestazione)]
        righe += [", ".join(str(r.get(c, "")) for c in intestazione) for r in dati]
        return "\n".join(righe)

class EsportatoreJSON(Esportatore):
    def esporta(self, dati):
        return json.dumps(dati, indent=2, ensure_ascii=False)

class EsportatoreTesto(Esportatore):
    def esporta(self, dati):
        blocchi = ["\n".join(f"{k}: {v}" for k, v in r.items()) for r in dati]
        return "\n\n".join(blocchi)

```

```

class EsportatoreMarkdown(Esportatore):          # aggiunto DOPO
    def esporta(self, dati):
        if not dati:
            return ""
        colonne = list(dati[0].keys())
        righe = ["| " + " | ".join(colonne) + " |",
                  "| " + " | ".join(["---"] * len(colonne)) + " |"]
        righe += ["| " + " | ".join(str(r.get(c, "")) for c in colonne) + " |" for r in dati]
        return "\n".join(righe)

def esporta_tutto(dati, esportatori):
    for e in esportatori:
        print(f"--- {type(e).__name__} ---")
        print(e.esporta(dati))

```

Punto 4: zero righe modificate. `EsportatoreMarkdown` si aggiunge senza toccare né la classe astratta, né gli altri esportatori, né `esporta_tutto()`, e riceve gratis `esporta_su_file()`. È la stessa proprietà che rende prezioso il componente del capitolo 7.

Esercizio 20 — Eliminare la catena di if

```

from abc import ABC, abstractmethod

class Spedizione(ABC):
    nome = "generica"

    @abstractmethod
    def costo(self, peso, distanza):
        """Costo in euro. Solleva ValueError se la spedizione non è possibile."""

    def preventivo(self, peso, distanza):
        return f"{self.nome}: {self.costo(peso, distanza):.2f} EUR"

class Standard(Spedizione):
    nome = "Standard"
    def costo(self, peso, distanza):
        return 5 + peso * 0.5

class Espresso(Spedizione):
    nome = "Espresso"
    def costo(self, peso, distanza):
        return 12 + peso * 0.8 + distanza * 0.1

class Internazionale(Spedizione):
    nome = "Internazionale"
    def costo(self, peso, distanza):
        base = 25 + peso * 1.2

```

```

        return base * 1.22 if distanza > 1000 else base

class SpedizioneDrone(Spedizione):
    nome = "Drone"
    RAGGIO_MAX = 50

    def costo(self, peso, distanza):
        if distanza > self.RAGGIO_MAX:
            raise ValueError(f"Drone non disponibile oltre {self.RAGGIO_MAX} km")
        return 8 + peso * 2

def miglior_offerta(peso, distanza, spedizioni):
    disponibili = []
    for s in spedizioni:
        try:
            disponibili.append((s.costo(peso, distanza), s))
        except ValueError:
            continue          # corriere non disponibile: si ignora
    if not disponibili:
        return None
    return min(disponibili, key=lambda coppia: coppia[0])[1]

```

La domanda finale. Formalmente sì, `SpedizioneDrone` indebolisce il contratto: le altre sottoclassi restituiscono sempre un numero, questa a volte solleva un'eccezione, e un codice scritto per una spedizione generica potrebbe non aspettarselo. La violazione però *scompare* se il contratto della classe base lo prevede fin dall'inizio — ed è ciò che fa la docstring di `costo()`: «solleva `ValueError` se la spedizione non è possibile». Da quel momento sollevare l'eccezione non è più una sorpresa, è parte della promessa, e infatti `miglior_offerta()` la gestisce esplicitamente. La lezione: **il contratto non è solo la firma del metodo, è anche cosa può andare storto**, e va documentato nella classe base.

I dieci errori più comuni

Quasi tutti i messaggi d'errore che incontrerai lavorando con le classi rientrano in questi dieci casi. Vale la pena rileggere questa pagina ogni volta che qualcosa non funziona: nove volte su dieci il colpevole è qui.

1. Dimenticare `self` nella definizione del metodo

```
class Studente:
    def saluta():
        print("ciao")
# TypeError: saluta() takes 0 positional arguments but 1 was given
```

Il messaggio confonde perché tu non hai passato argomenti: li ha passati Python, aggiungendo automaticamente l'oggetto. **Rimedio:** `def saluta(self):`.

2. Dimenticare `self.` nell'uso dei campi

```
def media(self):
    return sum(voti) / len(voti)
    return sum(self.voti) / len(self.voti)
```

Dentro un metodo, i campi dell'oggetto non sono variabili locali: si raggiungono solo attraverso `self`. Vale anche per chiamare altri metodi: `self.media()`, non `media()`.

3. Usare una lista come attributo di classe

```
class Carrello:
    articoli = []
```

Rimedio: creare sempre i tipi mutabili dentro `__init__` con `self.articoli = []`. Vedi il paragrafo 2.8.

4. Usare un valore mutabile come default di un parametro

```
def __init__(self, giocatori=[]):
def __init__(self, giocatori=None):
    self.giocatori = [] if giocatori is None else list(giocatori)
```

È lo stesso bug del punto 3 travestito. Vedi l'Esercizio 7.

5. Sbagliare gli underscore di `__init__`

```
def _init_(self, nome):      # uno solo: non viene MAI chiamato
def __init__(self, nome):    # due per lato: corretto
```

Se un oggetto sembra non avere i campi che gli hai messo, controlla subito qui: Python non segnala nulla, si limita a non chiamare il tuo metodo.

6. Dimenticare `super().__init__()` nella sottoclasse

```
class Manager(Dipendente):
    def __init__(self, nome, budget):
        self.budget = budget      # i campi del padre non esistono
# AttributeError: 'Manager' object has no attribute 'stipendio_base'
```

Rimedio: se scrivi `__init__` in una sottoclasse, la prima riga è quasi sempre `super().__init__(...)`.

7. Confondere la classe con l'istanza

```
s = Studente      # questa è la CLASSE, non un oggetto
s.nome            # AttributeError
s = Studente("Anna", "a@x.it")  # corretto: le parentesi creano l'oggetto
```

8. Chiamare una property con le parentesi (o dimenticarle su un metodo)

```
print(anna.media())  # TypeError: 'float' object is not callable (media è property)
print(anna.media)    # <bound method ...> (media è un metodo: mancano le parentesi)
```

Il messaggio «*object is not callable*» significa quasi sempre parentesi di troppo; una stampa che mostra `<bound method>` significa parentesi mancanti.

9. Restituire una struttura interna invece di una copia

```
def storico(self):
    return self._valori      # chiunque può modificarla dall'esterno
    return list(self._valori) # corretto
```

È una falla nell'incapsulamento che non produce nessun errore: si manifesta come un dato che cambia "da solo".

10. Credere che `b = a` copi l'oggetto

```
b = a
b.nome = "X"
print(a.nome)      # "X" -- è lo stesso oggetto!
```

Rimedio: `copy.copy(a)` o `copy.deepcopy(a)`, a seconda di quanto in profondità serve duplicare. Vedi il capitolo 3.

COME LEGGERE UN ERRORE PYTHON

Parti sempre dall'**ultima riga** del traceback: contiene il tipo di errore e il messaggio. Poi risali all'ultima riga che appartiene al *tuo* file: è quasi sempre lì il problema, anche quando l'errore esplode dentro una libreria. `AttributeError` significa «questo oggetto non ha quel nome» (punti 5, 6, 7);

`TypeError` riguarda numero o tipo di argomenti (punti 1, 8); `NameError` significa che un nome non esiste in quello scope (punto 2).

Cheat sheet

Definire una classe

```
class Studente:                                # nome in MaiuscoloCammello
    SOGLIA = 24                                # attributo di classe (costante)

    def __init__(self, nome, email):           # costruttore
        self.nome = nome                      # attributo di istanza (pubblico)
        self._voti = []                       # attributo interno (per convenzione)

    def aggiungi_voto(self, voto):              # metodo: self è sempre il primo
        self._voti.append(voto)

    @property                                  # si legge come un campo: s.media
    def media(self):
        return sum(self._voti) / len(self._voti) if self._voti else 0.0

s = Studente("Anna", "a@x.it")                # crea un'istanza
```

Property con validazione

```
@property
def valore(self):
    return self._valore

@valore.setter
def valore(self, nuovo):
    if nuovo < 0:
        raise ValueError("Deve essere positivo")
    self._valore = nuovo
```

Ereditarietà e overriding

```
class Manager(Dipendente):                    # Manager È un Dipendente

    def __init__(self, nome, budget):
        super().__init__(nome)                # SEMPRE come prima riga
        self.budget = budget

    def stipendio(self):
        return super().stipendio() * 1.2       # estende il comportamento
```

Classe astratta

```
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        """Le sottoclassi DEVONO implementarlo."""

    def descrivi(self):
        return f"Area: {self.area():.2f}"
```

metodo concreto: ereditato

Metodi speciali più usati

Metodo	Si attiva con	A cosa serve
<code>__init__(self, ...)</code>	<code>Classe(...)</code>	Inizializza l'oggetto appena creato
<code>__str__(self)</code>	<code>print(obj)</code> , <code>str(obj)</code>	Testo leggibile per l'utente
<code>__repr__(self)</code>	console, <code>print([obj])</code>	Rappresentazione tecnica per lo sviluppatore
<code>__eq__(self, altro)</code>	<code>obj1 == obj2</code> , <code>in</code>	Definisce quando due oggetti sono uguali
<code>__len__(self)</code>	<code>len(obj)</code>	Numero di elementi contenuti
<code>__lt__(self, altro)</code>	<code><</code> , <code>sorted()</code>	Ordinamento fra oggetti
<code>__contains__(self, x)</code>	<code>x in obj</code>	Test di appartenenza
<code>__iter__(self)</code>	<code>for x in obj</code>	Rende l'oggetto iterabile

Verifiche sui tipi

```
isinstance(m, Dipendente)    # True anche per le sottoclassi <- usa questo
type(m) is Dipendente        # True solo per il tipo esatto
issubclass(Manager, Dipendente) # relazione fra classi
type(m).__name__             # il nome della classe come stringa
a is b                       # stesso oggetto in memoria?
a == b                       # stesso valore? (serve __eq__)
id(a)                        # identità dell'oggetto
```

Copiare

```
import copy
b = a                # alias: nessuna copia
b = copy.copy(a)     # nuovo oggetto, campi interni CONDIVISI
b = copy.deepcopy(a) # nuovo oggetto, tutto duplicato
```

Convenzioni di visibilità

Scrittura	Significato
<code>nome</code>	Pubblico: fa parte dell'interfaccia, chiunque può usarlo
<code>_nome</code>	Interno: «non toccare». Python non lo impedisce, la convenzione sì
<code>__nome</code>	Name mangling: diventa <code>_Classe__nome</code> , scomodo da raggiungere dall'esterno
<code>__nome__</code>	Metodo speciale di Python: non inventarne di nuovi

Test con pytest

```
# file: test_studente.py -- si esegue con: pytest -v
import pytest
from studente import Studente

def test_studente_nuovo_ha_media_zero():
    assert Studente("Anna", "a@x.it").media == 0.0

def test_voto_fuori_scala_rifiutato():
    s = Studente("Anna", "a@x.it")
    with pytest.raises(ValueError):
        s.aggiungi_voto(45)
```

Le domande da farsi prima di scrivere una classe

1. **Responsabilità:** di cosa si occupa questa classe? (una frase sola, senza "e")
2. **Operazioni:** quali metodi pubblici deve offrire? (nomi di verbi)
3. **Dati:** quale stato deve custodire? (diventeranno campi interni)
4. **Invarianti:** quali regole devono valere sempre? (lì vanno i controlli)

Le regole d'oro

- Tutti i campi si dichiarano in `__init__`, anche quelli che valgono `None`.

- Mai una lista o un dizionario come attributo di classe o come default di un parametro.
- Nascondi lo stato, esponi le operazioni: i controlli devono stare in un punto solo.
- Non restituire mai una struttura interna mutabile: restituiscine una copia.
- Usa l'ereditarietà solo se regge il test "**è un**"; altrimenti componi e delega.
- Una lunga catena di `if` sul tipo è quasi sempre un polimorfismo mancante.
- Astrai alla **terza** ripetizione, non alla prima.
- I test verificano l'interfaccia pubblica, mai i campi interni.

E ADESSO?

- Rileggi il **capitolo 7**: quel componente è il punto di partenza delle prossime lezioni.
- Metti il tuo codice sotto **Git**, con i test accanto alle classi.
- Quando aggiungerai `ApiSource` e `MongoSource`, ti accorgerai che la parte difficile non è il codice nuovo: è averlo progettato in modo che il codice vecchio non debba cambiare.